

The SQIsign2d-push^{1/2} Signature Scheme: Algorithm Specifications and Supporting Documentation

Yu Dai¹, Debiao He², Guanju Xiao³, Gang Tang^{4,5}, Cong Peng⁵, Min Luo⁵, Jiang Zhang⁶, Xinyi Huang⁷, Zihao Yu⁵, Runqing Xu⁵, Nian Li³, and Bingbing Xia⁸

¹ School of Mathematics and Statistics, Wuhan University of Technology, Wuhan, China.

² School of Computer Science, Wuhan University, Wuhan, China

³ School of Cyber Science and Technology, Hubei University, Wuhan, China

⁴ Institute for Math & AI, Wuhan, Wuhan, China

⁵ School of Cyber Science and Engineering, Wuhan University, Wuhan, China

⁶ State Key Laboratory of Cryptology, China

⁷ College of Computer Science and Technology, Nanjing University of Aeronautics and Astronautics, Nanjing, China

⁸ Beijing Certificate Authority Co. Ltd, Beijing, China

Table of Contents

1	Introduction	4
1.1	Overview of the scheme	4
1.2	Advantages and limitations	5
2	Basic operations	6
2.1	Finite fields	6
2.2	Montgomery curves	7
2.3	Isogenies between Montgomery elliptic curves	12
2.4	Quaternion algebras over $\mathbb{Q}$	13
2.5	Quaternion orders and ideals	18
2.6	Isogenies between principally polarized abelian surfaces	20
3	The transform between isogenies and ideals	22
3.1	Deuring correspondence	22
3.2	Converting between ideals and isogenies	24
4	The SQLsign2d-push ^{1/2} Signature Scheme	25
4.1	Σ protocols and the Fiat–Shamir Transform	26
4.2	The system parameters	27
4.3	key generation	27
4.4	Signing	28
4.5	Verification	32
5	Parameters	33
5.1	Parameter requirements	33
5.2	Parameter sets	35
6	Security Reductions	36
6.1	Definitions for the FS _W A and FS _W H framework	36
6.2	Special Soundness	39
6.3	Hint-assist wHVZK [⊥]	40
6.4	Resistance to known attacks	41
7	Computing a (2, 2)-isogeny chain between products of elliptic curves	42
7.1	Generic (2, 2)-isogeny computation	44
7.2	Gluing (2, 2)-isogeny computation	46
7.3	Splitting Change of Coordinates	50
7.4	Computing a (2, 2)-isogeny chain between products of elliptic curves	52
8	Performance Analysis	56
8.1	Key and signature sizes	56
8.2	Computational cost	56
8.3	Reference implementation	56
8.4	Performance evaluation	57

Notations

Unless otherwise specified, the following symbols and notations are used in this document.

symbol	Description
λ	Classical security level of the signature scheme.
p	Size of the base field, a prime of the form $c2^{e_1}3^{e_2} - 1$.
$\mathbb{Z}/d\mathbb{Z}$	Ring of residue classes module d , where d is a positive integer.
$\mathbb{F}_p$	Finite fields with p elements.
$\mathbb{F}_{p^2}$	The quadratic extension of the finite field $\mathbb{F}_p$.
$E/\mathbb{F}_{p^2}$	An elliptic curve over $\mathbb{F}_{p^2}$.
$E \times E'$	A product of two elliptic curves E and E' .
A	An abelian variety of dimension 2 over some finite field $\mathbb{F}_q$.
$E_{A,B}$	An elliptic curve in Montgomery form with parameters $A, B \in \mathbb{F}_q$.
E_0	The elliptic curve of the form $y^2 = x^3 + x$.
E_i	The elliptic curve of the form $y^2 = x^3 + A_i x^2 + x$, where $A_i \in \mathbb{F}_{p^2}$.
E_i^j	The elliptic curve of the form $y^2 = x^3 + A_i^j x^2 + x$, where $A_i^j \in \mathbb{F}_{p^2}$.
O_E	Point at infinity of an elliptic curve E .
$\hat{O}(g)$	The time complexity $O(g\ell(g))$ with g a polylogarithmic function.
$E(\mathbb{F}_{p^2})$	The points on E with coordinates in $\mathbb{F}_{p^2}$.
$E[n]$	The n -torsion subgroup on E .
$j(E)$	The j -invariant of the elliptic curve E .
x_P resp. y_P	The x - resp. y -coordinate of a point $P \in E$.
$\varphi : E \rightarrow E'$	An isogeny between elliptic curves.
$\Phi : A \rightarrow A'$	An isogeny between abelian surface.
$\text{End}(E)$	The endomorphism ring of the elliptic curve E .
$B_{p,\infty}$	A quaternion algebra, ramified at p and ∞ .
$\mathcal{O}$	An order in $B_{p,\infty}$
$\text{trd}(\alpha)$	The trace of an element $\alpha \in B_{p,\infty}$.
$\text{nrd}(\alpha)$	The norm of an element $\alpha \in B_{p,\infty}$.
$n(I)$	The norm of an ideal $I \subset \mathcal{O}$.
$\text{ord}(P)$	The order of a point P on an elliptic curve.
$\text{adic}_2(n)$	The two adic evaluation of an integer n .

1 Introduction

We present $\text{SQIsign2d-push}^{1/2}$, a new digital signature scheme whose security relies on the hardness of computing isogenies between supersingular elliptic curves. Building on the framework of SQIsign2d-push [NO25], which is conjectured to be resistant to quantum adversaries, $\text{SQIsign2d-push}^{1/2}$ preserves the exceptionally compact signatures of its predecessor while substantially improving computational efficiency.

The name $\text{SQIsign2d-push}^{1/2}$ reflects the fact that the length of its $(2, 2)$ -isogeny chain in the signing phase is roughly half that required by SQIsign2d-push . **Consequently, compared with all variants of SQIsign2D [BDF⁺25, NOC⁺25, XLZO26, DF25], the new scheme achieves the lowest computational cost for $(2, 2)$ -isogeny evaluations during the signing stage.**

1.1 Overview of the scheme

$\text{SQIsign2d-push}^{1/2}$ is designed as a proof of knowledge, more precisely as a Σ -protocol, which is then compiled into a digital signature scheme via the Fiat-Shamir transform. The underlying Σ -protocol proves knowledge of an elliptic curve endomorphism ring. We begin by recalling the basic notions of elliptic curves and isogenies. Over a fixed finite field $\mathbb{F}_{p^2}$, there are approximately $\lfloor p/12 \rfloor$ supersingular elliptic curves up to isomorphism. An *isogeny* is a non-constant rational group homomorphism between two elliptic curves. The fundamental hardness assumption underlying isogeny-based cryptography is the following problem:

Problem 1 (isogeny path problem). Given two elliptic curves E_1 and E_2 , compute an isogeny $\varphi : E_1 \rightarrow E_2$ between them.

An *endomorphism* of an elliptic curve E is an isogeny from the curve to itself. The set of all such endomorphisms, together with the zero map, forms a ring under addition and composition; this ring is called the endomorphism ring of E and is denoted by $\text{End}(E)$.

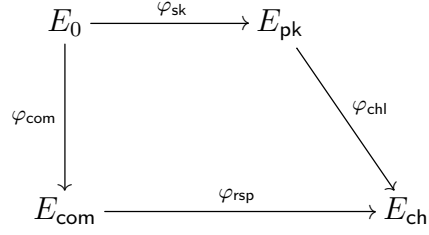
Problem 2 (endomorphism ring problem). Given an elliptic curve E , compute the endomorphism ring $\text{End}(E)$.

For supersingular elliptic curves, the endomorphism ring problem is known to be computationally equivalent to the isogeny path problem under polynomial-time reductions [EHL⁺18a, Wes22].

The underlying Σ -protocol has the following structure. The prover's public key is an elliptic curve E_{pk} , while the corresponding secret key is its endomorphism ring, $\text{End}(E_{\text{pk}})$. Given the public curve E_{pk} , the prover aims to convince a verifier that he knows $\text{End}(E_{\text{pk}})$ without revealing it. The protocol proceeds as follows:

1. **Commitment:** The prover generates a random supersingular elliptic curve together with its endomorphism ring, $(E_{\text{com}}, \text{End}(E_{\text{com}}))$, and sends E_{com} to the verifier.
2. **Challenge:** The verifier samples a random isogeny $\varphi_{\text{chl}} : E_{\text{pk}} \rightarrow E_{\text{chl}}$ and sends it to the prover.
3. **Response:** Using the secret $\text{End}(E_{\text{pk}})$ and the challenge isogeny φ_{chl} , the prover computes $\text{End}(E_{\text{chl}})$. Knowing both $\text{End}(E_{\text{com}})$ and the newly derived $\text{End}(E_{\text{chl}})$, the prover can efficiently compute a response isogeny $\varphi_{\text{rsp}} : E_{\text{com}} \rightarrow E_{\text{chl}}$ and send it to the verifier.

4. **Verification:** The verifier accepts the proof if φ_{rsp} is a valid isogeny from the commitment curve E_{com} to the challenge curve E_{chl} .



However, the naive protocol sketched above is insecure against malicious provers. An adversary could forge a commitment by simply sampling a random isogeny $\varphi_{\text{com}}^{\text{cheat}} : E_{\text{pk}} \rightarrow E_{\text{com}}$. Even without knowing $\text{End}(E_{\text{com}})$, after receiving the challenge isogeny $\varphi_{\text{chl}} : E_{\text{pk}} \rightarrow E_{\text{chl}}$, the adversary can construct a valid response by evaluating $\varphi_{\text{rsp}}^{\text{cheat}} = \varphi_{\text{chl}} \circ \widehat{\varphi}_{\text{com}}^{\text{cheat}} : E_{\text{com}} \rightarrow E_{\text{chl}}$, where $\widehat{\varphi}_{\text{com}}^{\text{cheat}}$ denotes the dual isogeny of $\varphi_{\text{com}}^{\text{cheat}}$.

This vulnerability can be addressed by requiring that the challenge φ_{chl} is not a “sub-isogeny” of the response φ_{rsp} . This structural constraint rules out the naive composition $\varphi_{\text{chl}} \circ \varphi_{\text{sk}} \circ \widehat{\varphi}_{\text{com}}$ as a valid response. As a result, the prover must compute an equivalent but distinct isogeny as the response, a task that requires knowledge of both $\text{End}(E_{\text{com}})$ and $\text{End}(E_{\text{pk}})$.

1.2 Advantages and limitations

SQIsign2d-push^{1/2} is a new variant of **SQIsign** that introduces a mechanism for evaluating an auxiliary isogeny during the response phase. This mechanism improves computational efficiency while preserving the security guarantees of the original scheme.

1. **Innovation:** **SQIsign2d-push**^{1/2} provides a strong balance between post-quantum security and exceptionally small signatures. Its signature and public-key sizes are among the smallest across post-quantum signature schemes. Moreover, compared to the existing variants of **SQIsign**, **SQIsign2d-push**^{1/2} achieves the least computational cost at the signing stage.
2. **Performance:** Although it has higher computational latency than ML-DSA and Falcon, **SQIsign2d-push**^{1/2} remains viable for real-world applications. At Level I, our 128-bit implementation performs signing and verification in approximately 64.04 ms and 12.54 ms, respectively (Intel(R) Core(TM) i7-1360P CPU@2.60Hz; see Table 3, Chapter 8). These improvements make **SQIsign2d-push**^{1/2} competitive among leading post-quantum signature candidates.
3. **Flexibility:** The scheme offers substantial parameter and design flexibility. It can scale to different post-quantum security levels by adjusting the finite field characteristic p .
4. **Compatibility:** With public-key and signature sizes of only a few hundred bytes, comparable to other post quantum signature schemes, **SQIsign2d-push**^{1/2} offers strong system-level compatibility. Thus, it is well-suit for resource-limited devices.
5. **Scalability:** The core mathematical framework is extensible to advanced cryptographic functionalities. The underlying proof-of-knowledge mechanism and quaternionic tools provide a foundation for constructing more complex primitives, such as blind signatures, ring signatures, and threshold signatures. Furthermore, the architecture supports extensions to higher-dimensional abelian varieties (such as (2, 2)-isogeny chains), leaving room for further development and performance improvements.

$\text{SQlsign2d-push}^{1/2}$ provides valuable cryptographic diversity by relying on the endomorphism ring problem, a security foundation mathematically distinct from lattice-based schemes such as ML-DSA and Falcon. Despite the inherent complexity of the $\text{SQlsign2d-push}^{1/2}$ signing procedure, the scheme achieves a level of computational efficiency that makes it practical for many real-world applications. Its main limitation, however, is the relative maturity of its cryptanalytic study: despite its number-theoretic origins in 1996 [Koh96], the problem has only recently received intensive cryptanalytic scrutiny.

2 Basic operations

In this section, we introduce the mathematical background needed for $\text{SQlsign2d-push}^{1/2}$, including supersingular elliptic curves over finite fields, isogenies between Montgomery elliptic curves, quaternion algebras over $\mathbb{Q}$, and isogenies between principally polarized abelian surfaces.

2.1 Finite fields

A finite field is a field with finitely many elements. It is equipped with addition and multiplication operations. In particular, it has additive and multiplicative identity elements, denoted by 0 and 1, respectively; every element has an additive inverse, and every nonzero element has a multiplicative inverse.

The cardinality q of a finite field must be a prime power, i.e., $q = p^r$ for some prime p and positive integer r . There is a unique finite field of cardinality q up to isomorphism, denoted by $\mathbb{F}_q$. For $q = p^r$, we call $\text{char}(\mathbb{F}_q) = p$ the characteristic of $\mathbb{F}_q$. In our scheme, the prime p is of the form $p = c \cdot 2^{e_1} \cdot 3^{e_2} - 1$ for some integers e_1 and e_2 satisfying $2^{e_1} \approx 3^{e_2}$; hence $p \equiv 3 \pmod{4}$.

2.1.1 The prime field $\mathbb{F}_p$. When the extension degree $r=1$, the finite field $\mathbb{F}_q = \mathbb{F}_p$. We represent the elements of $\mathbb{F}_p$ uniquely by the integers $\{0, \dots, p-1\}$. The algebraic operations are defined as follows:

- **Addition:** For $a, b \in \mathbb{F}_p$, the sum $c = a + b$ is given by the unique integer $c \in \{0, \dots, p-1\}$ that satisfies $c \equiv a + b \pmod{p}$.
- **Additive inverse:** For $a \in \mathbb{F}_p$, its additive inverse $-a$ is the unique integer $(-a) \in \{0, \dots, p-1\}$ that satisfies $a + (-a) \equiv 0 \pmod{p}$.
- **Multiplication:** For $a, b \in \mathbb{F}_p$, the product $c = a \cdot b$ is given by the unique integer $c \in \{0, \dots, p-1\}$ that satisfies $c \equiv a \cdot b \pmod{p}$.
- **Multiplicative inverse:** For $a \in \mathbb{F}_p$, $a \neq 0$, its multiplicative inverse a^{-1} is the unique integer $a^{-1} \in \{0, \dots, p-1\}$ that satisfies $a \cdot a^{-1} \equiv 1 \pmod{p}$. It can be computed directly as $a^{-1} = a^{p-2} \pmod{p}$.
- **Quadratic residuosity:** Let $a \in \mathbb{F}_p$, $a \neq 0$. Whether a is a square, i.e., whether there exists an element $b \in \mathbb{F}_p$ such that $a = b^2$, is determined by computing the Legendre symbol $a^{\frac{p-1}{2}}$, which equals 1 if a is a square and -1 otherwise.
- **Square root:** Let a be a square in $\mathbb{F}_p$, where $p \equiv 3 \pmod{4}$. The canonical square root of a is computed as

$$\sqrt{a} = a^{\frac{p+1}{4}} \pmod{p}. \quad (1)$$

2.1.2 The finite field $\mathbb{F}_{p^2}$. The finite field $\mathbb{F}_{p^2}$ is a quadratic extension of $\mathbb{F}_p$. Since $p \equiv 3 \pmod{4}$, the polynomial $x^2 + 1$ is irreducible over $\mathbb{F}_p$. Let i be a root of $x^2 + 1 = 0$; then $\mathbb{F}_{p^2}$ can be defined as $\mathbb{F}_{p^2} = \mathbb{F}_p(i)$. Thus, every $a \in \mathbb{F}_{p^2}$ can be represented uniquely as $a = a_0 + a_1 \cdot i$ for some $a_0, a_1 \in \mathbb{F}_p$. The basic algebraic operations over $\mathbb{F}_{p^2}$ are defined as follows:

- **Addition:** For $a = a_0 + a_1 \cdot i$, $b = b_0 + b_1 \cdot i \in \mathbb{F}_{p^2}$, the sum $c = a + b$ is given by $(a_0 + a_1) + (b_0 + b_1)i$.
- **Additive inverse:** For $a = a_0 + a_1 \cdot i \in \mathbb{F}_{p^2}$, its additive inverse is given by $-a = (-a_0) + (-a_1) \cdot i$.
- **Multiplication:** For $a = a_0 + a_1 \cdot i$, $b = b_0 + b_1 \cdot i \in \mathbb{F}_{p^2}$, the product is given by $c = c_0 + c_1 \cdot i$ with $c_0 = a_0b_0 - a_1b_1$ and $c_1 = a_0b_1 + a_1b_0$, using additions, additive inverses, and multiplications in $\mathbb{F}_p$.
- **Multiplicative inverse:** For $a = a_0 + a_1 \cdot i \in \mathbb{F}_{p^2}$, its multiplicative inverse is given by

$$a^{-1} = \frac{a_0 - a_1 \cdot i}{a_0^2 + a_1^2},$$

where $a_0^2 + a_1^2 \in \mathbb{F}_p$.

- **Quadratic residuosity:** An element $a = a_0 + a_1 \cdot i \in \mathbb{F}_{p^2}$ is a square if and only if $a^{p+1} = a_0^2 + a_1^2 \in \mathbb{F}_p$ is a square in $\mathbb{F}_p$.
- **Square root:** Let $a = a_0 + a_1 \cdot i \in \mathbb{F}_{p^2}$. If there exists an element $b \in \mathbb{F}_{p^2}$ such that $b^2 = a$, then we say that a is a square in $\mathbb{F}_{p^2}$. Following [AAA⁺25a], we define the canonical square root of a as

$$\sqrt{a} = (-i)^{\frac{1-\chi}{2}} S \cdot (a + \delta),$$

where $\delta = \sqrt{a_0^2 + a_1^2}$, $S = [2(a + \delta)]^{\frac{p-3}{4}}$, and $\chi = [2(a + \delta)]^{\frac{p-1}{2}}$.

2.2 Montgomery curves

The Montgomery form of an elliptic curve over a finite field $\mathbb{F}_q$ with $\text{char}(\mathbb{F}_q) > 3$ is defined by the equation

$$E_{A,B} : By^2 = x^3 + Ax^2 + x,$$

where $A, B \in \mathbb{F}_q$ and $B(A^2 - 4) \neq 0$. In particular, we abbreviate $E_{A,1}$ as E_A . The j -invariant of $E_{A,B}$ is given by

$$j(E_{A,B}) = \frac{256(A^2 - 3)^3}{A^2 - 4}.$$

The Montgomery curve $E_{A,B}$ over $\mathbb{F}_q$, denoted by $E_{A,B}/\mathbb{F}_q$, consists of the set of points $P = (x, y)$, defined over any extension of $\mathbb{F}_q$, that satisfy the above equation, together with the point at infinity O_E . In particular, we use $E_{A,B}(\mathbb{F}_q)$ to denote the points on $E_{A,B}$ with coordinates in $\mathbb{F}_q$.

2.2.1 Group law on Montgomery elliptic curves The set of points of a Montgomery curve forms an abelian group under the addition operation defined below. Let $P = (x_P, y_P)$ and $Q = (x_Q, y_Q)$ be points on $E_{A,B}$ with $P, Q \neq O_E$. Define $P + Q = R = (x_R, y_R)$ as follows:

- If $x_P \neq x_Q$, then

$$x_R = B\lambda^2 - A - x_P - x_Q,$$

and

$$y_R = \lambda(x_P - x_R) - y_P,$$

where $\lambda = (y_P - y_Q)/(x_P - x_Q)$.

- If $x_P = x_Q$, but $y_P \neq y_Q$, then $R = P + Q = O_E$.
- If $P = Q$ and $y_P = y_Q \neq 0$, then

$$x_R = B\lambda^2 - A - 2x_P,$$

and

$$y_R = \lambda(x_P - x_R) - y_P,$$

where $\lambda = \frac{3x_P^2 + 2Ax_P + 1}{2By_P}$.

- If $P = Q$ and $y_P = y_Q = 0$, then $R = P + Q = O_E$.

Furthermore, define $P + O_E = P$ for all $P \in E_{A,B}$.

One can verify that the above addition law on $E_{A,B}$ is commutative and associative. Moreover, every point $P = (x_P, y_P)$ has a unique additive inverse $-P = (x_P, -y_P)$ with respect to the identity element O_E .

Algorithm 1: xDBL($(X_P : Z_P)$)

Input: $(X_P : Z_P)$ where $P \in E_{A_{Pr}/C_{Pr}}$ and $(A_{24}^+ : C_{24})$.

Output: $[2]P = (X_{[2]P} : Z_{[2]P})$

```

1   $t_0 \leftarrow X_P - Z_P$ 
2   $t_1 \leftarrow X_P + Z_P$ 
3   $t_0 \leftarrow t_0^2$ 
4   $t_1 \leftarrow t_1^2$ 
5   $Z_{[2]P} \leftarrow C_{24} \cdot t_0$ 
6   $X_{[2]P} \leftarrow Z_{[2]P} \cdot t_1$ 
7   $t_1 \leftarrow t_1 - t_0$ 
8   $t_0 \leftarrow A_{24}^+ \cdot t_1$ 
9   $Z_{[2]P} \leftarrow Z_{[2]P} + t_0$ 
10  $Z_{[2]P} \leftarrow Z_{[2]P} \cdot t_1$ 
11 return  $(X_{[2]P} : Z_{[2]P})$ 

```

2.2.2 Projective x -only computations Let $\mathbb{P}_{\mathbb{F}_q}^2$ be the projective plane over $\mathbb{F}_q$. A point in $\mathbb{P}_{\mathbb{F}_q}^2$ is represented by homogeneous coordinates $(X : Y : Z)$, where not all coordinates are zero. For any nonzero $\lambda \in \mathbb{F}_q$, $(X : Y : Z)$ is equivalent to $(\lambda X : \lambda Y : \lambda Z)$.

Consider a Montgomery curve $E_{A,B}/\mathbb{F}_q$ given by the projective equation

$$BY^2Z = X^3 + AX^2Z + XZ^2.$$

When $Z \neq 0$, the projective point $P = (X : Y : Z)$ corresponds to the affine point $(x, y) = (X/Z, Y/Z)$. The point at infinity $O_{E_{A,B}}$ is represented by $(0 : 1 : 0)$ (i.e., $Z = 0$).

In optimized implementations of arithmetic on Montgomery curves, the above projective coordinates are used to avoid inversions (see [CS18]). We also use x -only arithmetic and represent points only as $P = (X_P : Z_P)$. This representation is sufficient for point doubling and addition on Montgomery curves, since the group law can be expressed solely in terms of affine x -coordinates or projective $(X : Z)$ coordinates.

To facilitate efficient computation, we use $(A_{Pr} : C_{Pr})$ to denote the equivalent class $(A_{Pr} : C_{Pr}) \sim (A : 1) \in \mathbb{P}_{\mathbb{F}_q}^1$. Furthermore, we define $(A_{24}^+ : C_{24}) \sim (A_{Pr} + 2C_{Pr} : 4C_{Pr})$, $(A_{24}^- : A_{24}^-) \sim (A_{Pr} + 2C_{Pr} : A_{Pr} - 2C_{Pr})$.

Using these projective representations, **Alg. 1** and **Alg. 3** describe point doubling and point addition, respectively, using only x -coordinates (i.e., $(X : Z)$ -only arithmetic).

Algorithm 2: xTPL($(X_P : Z_P), (A_{24}^+ : A_{24}^-)$)

Input: $(X_P : Z_P)$ and $(A_{24}^+ : A_{24}^-)$

Output: $(X_{[3]P} : Z_{[3]P})$

```

1   $t_0 \leftarrow X_P - Z_P$ ;
2   $t_2 \leftarrow t_0^2$ ;
3   $t_1 \leftarrow X_P + Z_P$ ;
4   $t_3 \leftarrow t_1^2$ ;
5   $t_4 \leftarrow t_1 + t_0$ ;
6   $t_0 \leftarrow t_1 - t_0$ ;
7   $t_1 \leftarrow t_4^2$ ;
8   $t_1 \leftarrow t_1 - t_3$ ;
9   $t_1 \leftarrow t_1 - t_2$ ;
10  $t_5 \leftarrow t_3 \cdot A_{24}^+$ ;
11  $t_3 \leftarrow t_5 \cdot t_3$ ;
12  $t_6 \leftarrow t_2 \cdot A_{24}^-$ ;
13  $t_2 \leftarrow t_2 \cdot t_6$ ;
14  $t_3 \leftarrow t_2 - t_3$ ;
15  $t_2 \leftarrow t_5 - t_6$ ;
16  $t_1 \leftarrow t_2 \cdot t_1$ ;
17  $t_2 \leftarrow t_3 + t_1$ ;
18  $t_2 \leftarrow t_2^2$ ;
19  $X_{[3]P} \leftarrow t_2 \cdot t_4$ ;
20  $t_1 \leftarrow t_3 - t_1$ ;
21  $t_1 \leftarrow t_1^2$ ;
22  $Z_{[3]P} \leftarrow t_1 \cdot t_0$ ;
23 return  $(X_{[3]P} : Z_{[3]P})$ ;
```

Algorithm 3: xADD($(X_P : Z_P), (X_Q : Z_Q), (X_{P-Q} : Z_{P-Q})$)

Input: $(X_P : Z_P), (X_Q : Z_Q), (X_{P-Q} : Z_{P-Q})$ where $P, Q \in E_{A_{Pr}/C_{Pr}}$.

Output: $(X_{P+Q} : Z_{P+Q})$ where $P + Q \in E_{A_{Pr}/C_{Pr}}$.

```

1   $t_0 \leftarrow X_P - Z_P$ 
2   $t_1 \leftarrow X_P + Z_P$ 
3   $t_2 \leftarrow X_Q - Z_Q$ 
4   $t_3 \leftarrow X_Q + Z_Q$ 
5   $t_1 \leftarrow t_1 \cdot t_2$ 
6   $t_0 \leftarrow t_0 \cdot t_3$ 
7   $t_2 \leftarrow t_1 - t_0$ 
8   $t_3 \leftarrow t_1 + t_0$ 
9   $t_2 \leftarrow t_2^2$ 
10  $t_3 \leftarrow t_3^2$ 
11  $X_{P+Q} \leftarrow Z_{P-Q} \cdot t_3$ 
12  $Z_{P+Q} \leftarrow X_{P-Q} \cdot t_2$ 
13 return  $(X_{P+Q} : Z_{P+Q})$ 
```

2.2.3 Torsion subgroups Let E be an elliptic curve defined over $\mathbb{F}_q$ of characteristic p . Given an integer n and a point $P \in E(\overline{\mathbb{F}}_q)$, we define the scalar multiplication $[n]P := \underbrace{P + P + \dots + P}_{n \text{ times}}$. The n -torsion subgroup is then defined as

$$E[n] = \{P \in E(\overline{\mathbb{F}}_q) : [n]P = O_E\}.$$

If $p \nmid n$, then $E[n] \cong \mathbb{Z}/n\mathbb{Z} \times \mathbb{Z}/n\mathbb{Z}$ and has cardinality n^2 . Furthermore, there exist non-unique points $R, S \in E[n]$ that generate $E[n]$. We call (R, S) a basis of $E[n]$.

2.2.4 The Weil pairing Let E be an elliptic curve defined over $\mathbb{F}_q$ of characteristic p , and let n be a positive integer not divisible by p . Let $\mu_n = \{x \in \overline{\mathbb{F}_q} : x^n = 1\}$ be the group of n -th roots of unity in $\overline{\mathbb{F}_q}$, and let ζ be a primitive n -th root of unity. The Weil pairing of degree n , introduced by Weil [Wei40], is a map

$$e_n : E[n] \times E[n] \rightarrow \mu_n$$

that satisfies the following properties:

1. Bilinearity: $\forall S_1, S_2, T_1, T_2 \in E[n]$, $e_n(S_1 + S_2, T_1) = e_n(S_1, T_1)e_n(S_2, T_1)$ and $e_n(S_1, T_1 + T_2) = e_n(S_1, T_1)e_n(S_1, T_2)$.
2. Non-degeneracy: $\forall T \in E[n]$, if $e_n(S, T) = 1$, then $S = O_E$; similarly, $\forall S \in E[n]$, if $e_n(S, T) = 1$, then $T = O_E$.
3. Symmetry: $\forall S, T \in E[n]$, $e_n(T, S) = e_n(S, T)^{-1}$.
4. Compatibility: $\forall S, T \in E[n]$, $\alpha \in \text{End}(E)$, $e_n(\alpha(S), \alpha(T)) = e_n(S, T)^{\deg(\alpha)}$.

2.2.5 The reduced Tate-Lichtenbaum pairing Let E be an elliptic curve defined over $\mathbb{F}_q$, and let n be an integer such that $n \mid q - 1$. The reduced Tate-Lichtenbaum pairing, introduced by Tate [Tat62], is a bilinear pairing

$$t_n : E(\mathbb{F}_q)[n] \times E(\mathbb{F}_q)/nE(\mathbb{F}_q) \rightarrow \mu_n$$

computed as $t_n(P, Q) = e_n(P, R - \Phi_q(R))$, where $R \in E(\overline{\mathbb{F}_q})$ satisfies $[n]R = Q$ and Φ_q is the q -th power Frobenius endomorphism.

2.2.6 Deterministic Basis Computation For a Montgomery curve $E_A : y^2 = x^3 + Ax^2 + x$ over $\mathbb{F}_{p^2}$, we give an algorithm **TorsionofBasis** (Alg. 4) that generates a deterministic basis of $E_A[\ell^e]$, where $\ell \in \{2, 3\}$ and e is an integer.

2.2.7 Change basis In **SQlsign2d-push**^{1/2}, we use Tate pairings to compute the matrix associated with different bases of $E[2^e]$ or $E[3^e]$. Concretely, if (P_1, P_2) and (Q_1, Q_2) are different bases of $E[2^e]$ (or $E[3^e]$), we need to compute x_1, x_2, x_3, x_4 such that

$$\begin{pmatrix} x_1 & x_2 \\ x_3 & x_4 \end{pmatrix} \cdot \begin{pmatrix} P_1 \\ P_2 \end{pmatrix} = \begin{pmatrix} Q_1 \\ Q_2 \end{pmatrix}.$$

Note that $t_n(P, P) = 1$ for any $P \in E[n]$. Then we have $t_n(P_1, Q_1) = t_n(P_1, [x_1]P_1 + [x_2]P_2) = t_n(P_1, P_1)^{x_1} \cdot t_n(P_1, P_2)^{x_2} = \zeta^{x_2}$. Applying discrete logarithms, we obtain x_2 . Similarly, we can obtain x_1, x_3, x_4 . We describe this algorithm in **Alg. 6**.

2.2.8 Supersingular elliptic curves An elliptic curve E in characteristic p is called ordinary if $E[p] \cong \mathbb{Z}_p$. It is called supersingular if $E[p] = O_E$. When $p \equiv 3 \pmod{4}$, the elliptic curve $E_0 : y^2 = x^3 + x$ over $\mathbb{F}_{p^2}$ is supersingular. Furthermore, $\#E_0(\mathbb{F}_{p^2}) = (p+1)^2$ and $E_0(\mathbb{F}_{p^2}) \cong \mathbb{Z}/(p+1)\mathbb{Z} \times \mathbb{Z}/(p+1)\mathbb{Z}$. If E_1 is an elliptic curve defined over $\mathbb{F}_{p^2}$ that is isogenous to E_0 (see the definitions in Subsection 2.3), then $E_1(\mathbb{F}_{p^2})$ and $E_0(\mathbb{F}_{p^2})$ have the same group structure.

Algorithm 4: TorsionofBasis(E_A, ℓ, e)

Input: A Montgomery curve $E_A : y^2 = x^3 + Ax^2 + x$, a small prime $\ell \in \{2, 3\}$ and an integer e .**Output:** A deterministic basis $\{P, Q\}$ of $E_A[\ell^e]$.

```

1  $a \leftarrow 0, P \leftarrow O_{E_A};$ 
2 repeat
3    $a \leftarrow a + 1;$ 
4    $x_P \leftarrow 1 + ia;$ 
5    $y_P \leftarrow (x_P^3 + Ax_P)^{\frac{p+1}{4}};$ 
6   if  $y_P^2 = x_P^3 + Ax_P$  then
7      $P \leftarrow [\frac{p+1}{\ell^e}](x_P, y_P);$ 
8 until  $[\ell^e - 1]P \neq O_{E_A}$  and  $[\ell^e]P = O_{E_A};$ 
9  $b \leftarrow a, Q \leftarrow O_{E_A};$ 
10 repeat
11    $b \leftarrow b + 1;$ 
12    $x_Q \leftarrow 1 + ib;$ 
13    $y_Q \leftarrow (x_Q^3 + Ax_Q)^{\frac{p+1}{4}};$ 
14   if  $y_Q^2 = x_Q^3 + Ax_Q$  then
15      $Q \leftarrow [\frac{p+1}{\ell^e}](x_Q, y_Q);$ 
16 until  $[\ell^e - 1]Q \neq O_{E_A}, [\ell^e - 1]Q \neq [\ell^e - 1]P$  and  $[\ell^e]Q = O_{E_A};$ 
17 return  $(P, Q);$ 

```

Algorithm 5: TorsionDesLog($(P_1, P_2), Q, n$)

Input: Integer n . Basis (P_1, P_2) of $E[n]$. Point $Q \in E[n]$.**Output:** Integers $x_1, x_2 \in \mathbb{Z}/n\mathbb{Z}$ satisfying $Q = [x_1]P_1 + [x_2]P_2$.

```

1  $\zeta \leftarrow t_n(P_1, P_2)$ 
2  $\zeta_1 \leftarrow t_n(P_2, Q)$ 
3  $\zeta_2 \leftarrow t_n(P_1, Q)$ 
4  $x_1 \leftarrow \log_\zeta(\zeta_1)$ 
5  $x_2 \leftarrow \log_\zeta(\zeta_2)$ 
6 return  $(x_1, x_2).$ 

```

Algorithm 6: ChangeofBasis($Q_1, Q_2, P_1, P_2, \ell, e$)

Input: Bases (P_1, P_2) and (Q_1, Q_2) of $E[\ell^e]$, where $\ell \in \{2, 3\}$, and e is an integer.**Output:** A change-of-basis matrix $\mathbb{M} = \begin{pmatrix} x_1 & x_2 \\ x_3 & x_4 \end{pmatrix}$ such that $Q_1 = [x_1]P_1 + [x_2]P_2$ and

$$Q_2 = [x_3]P_1 + [x_4]P_2.$$

```

1  $(x_1, x_2) \leftarrow \text{TorsionDesLog}((P_1, P_2), Q_1, \ell^e)$ 
2  $(x_3, x_4) \leftarrow \text{TorsionDesLog}((P_1, P_2), Q_2, \ell^e)$ 
3  $\mathbb{M} \leftarrow \begin{pmatrix} x_1 & x_2 \\ x_3 & x_4 \end{pmatrix}$ 
4 return  $\mathbb{M}$ 

```

2.3 Isogenies between Montgomery elliptic curves

An isogeny between two elliptic curves E_1 and E_2 over $\mathbb{F}_q$ is a non-constant rational map

$$\varphi : E_1 \rightarrow E_2$$

that is also a group homomorphism with respect to the point addition laws on E_1 and E_2 . In other words, φ maps the point at infinity O_{E_1} to O_{E_2} and satisfies $\varphi(P + Q) = \varphi(P) + \varphi(Q)$ for all $P, Q \in E_1$. If such a map exists, we say that E_1 is isogenous to E_2 .

An isogeny $\varphi : E_1 \rightarrow E_2$ can be expressed as $\varphi(x, y) = (r_1(x), r_2(x) \cdot y)$ for $(x, y) \in E_1$, where $r_1(x)$ and $r_2(x)$ are rational functions. Write

$$r_1(x) = p(x)/q(x)$$

with polynomials $p(x)$ and $q(x)$ that have no common factor. The degree of φ is defined as

$$\deg(\varphi) = \max\{\deg p(x), \deg q(x)\}.$$

An isogeny can be characterized by its kernel $\ker(\varphi) = \{P \in E_1 : \varphi(P) = O_{E_2}\}$. More precisely, given a subgroup $G \subset E$ of cardinality N , there exists a unique isogeny $\varphi : E \rightarrow E'$ up to isomorphism such that $\ker \varphi = G$ and $\deg \varphi = N$. The codomain curve E' and the isogeny can be computed explicitly using Vélu's formulas [Vél71].

2.3.1 Isogeny formulas on Montgomery elliptic curves In our signature scheme, we use isogenies of small degree $\ell \in \{2, 3, 4\}$. We recall below the corresponding isogeny formulas for Montgomery elliptic curves [AAA⁺25b, CCH⁺19].

- **2-isogenies** Let $Q = (x_Q, 0) \in E_{A,B}$ be a point of order 2. The map $\varphi : E_{A,B} \rightarrow E_{A',B'}$ is the unique 2-isogeny with kernel $\langle Q \rangle$. If $x_Q \neq 0$, then

$$(A', B') = (2 \cdot (1 - 2x_Q^2), Bx_Q),$$

and

$$\varphi(x, y) = \left(\frac{x_Q x^2 - x}{x - x_Q}, y \cdot \frac{x_Q x^2 - 2x_Q^2 x + x_Q}{(x - x_Q)^2} \right).$$

- **4-isogenies** Let $Q = (x_Q, y_Q) \in E_{A,B}$ be a point of order 4 with $x_Q \neq \pm 1$. The map $\varphi : E_{A,B} \rightarrow E_{A',B'}$ is the unique 4-isogeny with kernel $\langle Q \rangle$. Then

$$(A', B') = (4x_Q^4 - 2, -x_Q(x_Q^2 + 1) \cdot B/2),$$

and $\varphi(x, y) = (r_1(x), y \cdot r_2(x))$, where

$$r_1(x) = \frac{-(x_Q^4 x + x - 2x_Q)x(x_Q x - 1)^2}{(x - x_Q)^2(2x_Q x - x_Q^2 - 1)},$$

and

$$r_2(x) = \frac{-2x_Q^2(x_Q x - 1)(x^4(x_Q^2 + 1) - 4(x_Q^3 + x_Q)x^3 + 2(x_Q^4 + 5x_Q^2)x^2 - 4(x_Q^3 + x_Q)x + x_Q^2 + 1)}{(x - x_Q)^3(2x_Q x - x_Q^2 - 1)^2}.$$

- **3-isogenies** Let $Q = (x_Q, y_Q) \in E_{A,B}$ be a point of order 3. The map $\varphi : E_{A,B} \rightarrow E_{A',B'}$ is the unique 3-isogeny with kernel $\langle Q \rangle$. Then

$$(A', B') = ((Ax_Q - 6x_Q^3 + 6) \cdot x_Q, Bx_Q^2),$$

and

$$\varphi(x, y) = \left(\frac{x(x_Q x - 1)^2}{(x - x_Q)^2}, y \cdot \frac{(x_Q x - 1)(x_Q x^2 - 3x_Q^2 x + x + x_Q)}{(x - x_Q)^3} \right).$$

2.3.2 Projective x -only algorithms for isogeny evaluation The formulas above show that the x -coordinate of $\varphi(P)$, for $P \in E_{A,B}$, depends only on x_P and x_Q . This allows us to evaluate isogenies using only projective $(X : Z)$ coordinates. We now describe algorithms for evaluating an isogeny $\varphi : E_{A,B} \rightarrow E_{A',B'}$ of degree $\ell \in 2, 4, 3$ at a point $P = (X_P : Z_P)$ (with $x_P = X_P/Z_P$).

Given a point Q of order 2, **Alg. 7** and **Alg. 9** compute the image curve under the 2-isogeny φ with kernel $\langle Q \rangle$, while **Alg. 8** and **Alg. 10** evaluate the projective $(X : Z)$ coordinates of $\varphi(P)$. Similarly, given a point Q of order 4 or 3, **Alg. 11** and **Alg. 13** compute the image curve under the corresponding 4- or 3-isogeny φ with kernel $\langle Q \rangle$, respectively. **Alg. 12** and **Alg. 14** evaluate the $(X : Z)$ coordinates of $\varphi(P)$ using the outputs of **Alg. 11** and **Alg. 13**, respectively.

2.3.3 Computing isogeny chains Using the basic algorithms for isogeny evaluation of degrees 2, 4, 3, we can compute 2^e - and 3^e -isogenies from a given point of order 2^e or 3^e (see more details in [CCH⁺19, AAA⁺25b]).

In the following, **TwolsogenyChainSmall**(**Alg. 15**) takes as input the elliptic curve E , a small integer e , and a kernel generator R on E of order 2^e with kernel $\langle R \rangle$. It outputs the image curve under an isogeny of degree 2^e . In addition, given a list of points on E , it can output their images. **ThreelsogenyChain**(**Alg. 16**) takes as input the elliptic curve E , an integer e , and a kernel generator R on E of order 3^e . It outputs the image curve under an isogeny of degree 3^e with kernel $\langle R \rangle$. It can also evaluate the images of points on E . **TwistIsogenyChainSmall** (**Alg. 17**) takes as input the elliptic curve E , an integer L , and a kernel generator R on E of order L with kernel $\langle R \rangle$. It outputs the image curve E' under an isogeny φ from the twist of E with kernel $\langle R \rangle$. In this algorithm, we use the notation $\mathbf{p}_i\text{-iso_curve}$ to denote the algorithm that computes the image curve from a kernel generator $\langle R \rangle$ of prime order p_i . Similarly, $\mathbf{p}_i\text{-iso_eval}$ denotes the algorithm that evaluates image points under the isogeny with kernel $\langle R \rangle$. These two algorithms are analogous to $\ell\text{-iso_curve}$ and $\ell\text{-iso_eval}$ ($\ell \in \{2, 4, 3\}$) introduced above.

Algorithm 7: $2\text{-iso_curve}((X_Q : Z_Q))$

Input: $Q = (X_Q : Z_Q) \neq (0 : 1)$ of order 2 on $E_{A/C}$

Output: $(A_{24}^+ : C_{24}) \sim (A' + 2C' : 4C')$ corresponding to $E_{A'/C'} = E_{A/C}/\langle Q \rangle$

- 1 $A_{24} \leftarrow X_Q^2$
 - 2 $C_{24} \leftarrow Z_Q^2$
 - 3 $A_{24} \leftarrow C_{24} - A_{24}^+$
 - 4 **return** $(A_{24}^+ : C_{24})$
-

2.4 Quaternion algebras over $\mathbb{Q}$

Quaternion algebras over $\mathbb{Q}$ [Voi21] are 4-dimensional non-commutative algebras generated by four elements $\{1, i, j, k\}$ with the following multiplication rules:

$$i^2 = a, \quad j^2 = b, \quad ij = -ji = k,$$

where a and b are rational numbers. In **SQIsign2d-push**^{1/2}, we restrict to quaternion algebras over $\mathbb{Q}$ that are ramified only at p and ∞ , denoted by $B_{p,\infty}$. Moreover, because $p \equiv 3 \pmod{4}$, we can choose $a = -1$ and $b = -p$. Henceforth, any element $\alpha \in B_{p,\infty}$ can be represented as $\alpha = w + xi + yj + zk$ with $w, x, y, z \in \mathbb{Q}$ and $i^2 = -1, j^2 = -p$.

Algorithm 8: 2_iso_eval($(X_Q : Z_Q)$)

Input: $Q = (X_Q : Z_Q) \neq (0 : 1)$ of order 2 on $E_{A/C}$ **Output:** $(X_{P'} : Z_{P'})$ corresponding to $P' \in E_{A'/C'}$, where $E_{A'/C'}$ is 2-isogenous to $E_{A/C}$

```

1  $t_0 \leftarrow X_Q + Z_Q$ 
2  $t_1 \leftarrow X_Q - Z_Q$ 
3  $t_2 \leftarrow X_P + Z_P$ 
4  $t_3 \leftarrow X_P - Z_P$ 
5  $t_0 \leftarrow t_0 \cdot t_3$ 
6  $t_1 \leftarrow t_1 \cdot t_2$ 
7  $t_2 \leftarrow t_0 + t_1$ 
8  $t_3 \leftarrow t_0 - t_1$ 
9  $X_{P'} \leftarrow X_P \cdot t_2$ 
10  $Z_{P'} \leftarrow Z_P \cdot t_3$ 
11 return  $(X_{P'}, Z_{P'})$ 

```

Algorithm 9: 2_iso_curve_singular($(A_{24} : C_{24})$)

Input: Montgomery coefficient $(A_{24} : C_{24})$ of the curve E .**Output:** The coefficients $(A'_{24} : C'_{24})$ corresponding to the image curve $E' := E/\langle P \rangle$ under the 2-isogeny with $P = (0 : 1)$, and constants **consts** = (c_0, c_1) .

```

1  $t_0 \leftarrow A_{24} + A_{24}$ ;
2  $t_0 \leftarrow t_0 - C_{24}$ ;
3  $t_0 \leftarrow t_0 + t_0$ ;
4  $t_1 \leftarrow C_{24}^{-1}$ ;
5  $t_0 \leftarrow t_0 \cdot t_1$ ;
6  $c_1 \leftarrow t_0$ ;
7  $A'_{24} \leftarrow t_0 + t_0$ ;
8  $t_0 \leftarrow t_0^2$ ;
9  $t_0 \leftarrow t_0 - 4$ ;
10  $t_0 \leftarrow \sqrt{t_0}$ ;
11  $c_1 \leftarrow -t_0$ ;
12  $C'_{24} \leftarrow t_0 + t_0$ ;
13  $A'_{24} \leftarrow A'_{24} + C'_{24}$ ;
14  $C'_{24} \leftarrow C'_{24} + C'_{24}$ ;
15 consts  $\leftarrow (c_0, c_1)$ ;
16 return  $(A'_{24} : C'_{24}), \text{consts}$ ;

```

Algorithm 10: 2_iso_eval_singular($(X_Q : Z_Q), \text{consts}$)

Input: Projective point $Q = (X_Q : Z_Q)$ where $Q \in E$, constants **consts** returned by 2_iso_curve_singular.**Output:** Projective point $Q' = (X_{Q'} : Z_{Q'}) \in E'$ corresponding to the image of Q under the 2-isogeny defined by $P = (0 : 1)$.

```

1  $c_0, c_1 \leftarrow \text{consts}$ ;
2  $t_0 \leftarrow X_Q \cdot Z_Q$ ;
3  $t_1 \leftarrow c_0 \cdot Z_Q$ ;
4  $t_1 \leftarrow t_1 + X_Q$ ;
5  $t_1 \leftarrow t_1 \cdot X_Q$ ;
6  $X_{Q'} \leftarrow Z_Q^2$ ;
7  $X_{Q'} \leftarrow X_{Q'} + t_1$ ;
8  $Z_{Q'} \leftarrow t_0 \cdot c_1$ ;
9 return  $(X_{Q'} : Z_{Q'})$ ;

```

Algorithm 11: $4_iso_curve((X_Q : Z_Q), (X_P : Z_P))$ **Input:** $Q = (X_Q : Z_Q)$ where Q has order 4 on $E_{A/C}$, and $(X_P : Z_P)$ where $P \in E_{A/C}$ **Output:** $(A_{24}^+ : C_{24}) \sim (A' + 2C' : 4C')$ corresponding to $E_{A'/C'} = E_{A/C}/\langle Q \rangle$ and constants $(K_1, K_2, K_3) \in \mathbb{F}_{p^2}^3$

```

1  $K_2 \leftarrow X_Q - Z_Q$ 
2  $K_3 \leftarrow X_Q + Z_Q$ 
3  $k_1 \leftarrow Z_Q^2$ 
4  $K_1 \leftarrow k_1 + k_1$ 
5  $C_{24} \leftarrow K_1^2$ 
6  $K_1 \leftarrow k_1 + k_1$ 
7  $A_{24}^+ \leftarrow X_Q^2$ 
8  $A_{24}^+ \leftarrow A_{24}^+ + A_{24}^+$ 
9  $A_{24}^+ \leftarrow (A_{24}^+)^2$ 
10 return  $(A_{24}^+ : C_{24}), (K_1, K_2, K_3)$ 

```

Algorithm 12: $4_iso_eval((K_1, K_2, K_3), (X_P : Z_P))$ **Input:** Constants $(K_1, K_2, K_3) \in \mathbb{F}_{p^2}^3$ from 4_iso_curve , and $P = (X_P : Z_P)$ where $P \in E_{A/C}$ **Output:** $P' = (X_{P'} : Z_{P'})$ corresponding to $P' \in E_{A'/C'}$, where $E_{A'/C'}$ is 4-isogenous to $E_{A/C}$

```

1  $t_0 \leftarrow X_P + Z_P$ 
2  $t_1 \leftarrow X_P - Z_P$ 
3  $X_P \leftarrow t_0 \cdot K_2$ 
4  $Z_P \leftarrow t_1 \cdot K_3$ 
5  $t_0 \leftarrow t_0 \cdot t_1$ 
6  $t_0 \leftarrow t_0 \cdot K_1$ 
7  $t_1 \leftarrow X_P + Z_P$ 
8  $Z_P \leftarrow X_P - Z_P$ 
9  $t_1 \leftarrow t_1^2$ 
10  $Z_P \leftarrow Z_P^2$ 
11  $X_P \leftarrow t_0 + t_1$ 
12  $t_0 \leftarrow Z_P - t_0$ 
13  $X_{P'} \leftarrow X_P \cdot t_1$ 
14  $Z_{P'} \leftarrow Z_P \cdot t_0$ 
15 return  $(X_{P'} : Z_{P'})$ 

```

For an element $\alpha = w + xi + yj + zk \in B_{p,\infty}$, we define its conjugate $\bar{\alpha} = w - xi - yj - zk$. Its reduced trace is $\text{trd}(\alpha) = \alpha + \bar{\alpha} = 2w$, and its reduced norm is $\text{nrd}(\alpha) = \alpha\bar{\alpha} = w^2 + x^2 + p(y^2 + z^2)$.

We define the inner product of two elements $\alpha, \beta \in B_{p,\infty}$ by the map $\langle \alpha, \beta \rangle = \text{trd}(\alpha\bar{\beta})$, which is a symmetric bilinear form. Given a basis $\alpha_1, \alpha_2, \alpha_3, \alpha_4$, its Gram matrix is the symmetric matrix whose (i, j) -th entry is $\langle \alpha_i, \alpha_j \rangle$. Let $\mathbf{G}$ be the Gram matrix of the basis $(1, i, j, k)$, which is a diagonal matrix with entries $(2, 2, 2p, 2p)$. If $\alpha = (1, i, j, k)\mathbf{a}$ and $\beta = (1, i, j, k)\mathbf{b}$, then $\langle \alpha, \beta \rangle = \mathbf{a}^\top \mathbf{G} \mathbf{b}$.

Definition 1 (Reduced basis). Let $\mathbf{b}_0, \mathbf{b}_1, \dots, \mathbf{b}_d$ be a basis of a positive definite quadratic space. Define the Gram-Schmidt vectors $\bar{\mathbf{b}}_i$ as

$$\bar{\mathbf{b}}_i = \mathbf{b}_i - \sum_{j=0}^{i-1} \mu_{i,j} \bar{\mathbf{b}}_j,$$

where $\mu_{i,j} = \frac{r_{i,j}}{r_{j,j}}$ and $r_{i,j} = \langle \mathbf{b}_i, \bar{\mathbf{b}}_j \rangle$. The basis is said to be (η, δ) -reduced for parameters $1/2 < \eta < 1$ and $1/4 < \delta < 1$ if:

- (1) $|\mu_{i,j}| < \eta$ for $0 \leq j < i \leq d$, and
- (2) $\langle \bar{\mathbf{b}}_i, \bar{\mathbf{b}}_i \rangle \geq (\delta - \mu_{i,i-1}^2) \langle \bar{\mathbf{b}}_{i-1}, \bar{\mathbf{b}}_{i-1} \rangle$ for $1 \leq i < d$.

Algorithm 13: 3_iso_curve($(X_Q : Z_Q)$)**Input:** $Q = (X_Q : Z_Q)$, where Q has order 3 on $E_{A/C}$ **Output:** $(A_{24}^+ : A_{24}^-) \sim (A' + 2C' : A' - 2C')$ corresponding to $E_{A'/C'} = E_{A/C}/\langle Q \rangle$, and constants $(K_1, K_2) \in (\mathbb{F}_{p^2})^2$

```

1  $K_1 \leftarrow X_Q - Z_Q$ 
2  $t_0 \leftarrow K_1^2$ 
3  $K_2 \leftarrow X_Q + Z_Q$ 
4  $t_1 \leftarrow K_2^2$ 
5  $t_2 \leftarrow t_0 + t_1$ 
6  $t_3 \leftarrow K_1 + K_2$ 
7  $t_3 \leftarrow t_3^2$ 
8  $t_3 \leftarrow t_3 - t_2$ 
9  $t_2 \leftarrow t_1 + t_3$ 
10  $t_3 \leftarrow t_3 + t_0$ 
11  $t_4 \leftarrow t_3 + t_0$ 
12  $t_4 \leftarrow t_4 + t_4$ 
13  $t_4 \leftarrow t_1 + t_4$ 
14  $A_{24}^- \leftarrow t_2 \cdot t_4$ 
15  $t_4 \leftarrow t_1 + t_2$ 
16  $t_4 \leftarrow t_4 + t_4$ 
17  $t_4 \leftarrow t_0 + t_4$ 
18  $A_{24}^+ \leftarrow t_3 \cdot t_4$ 
19 return  $(A_{24}^+ : A_{24}^-), (K_1, K_2) \in (\mathbb{F}_{p^2})^2$ 

```

Algorithm 14: 3_iso_eval($(K_1, K_2), P = (X_P : Z_P)$)**Input:** Constants $(K_1, K_2) \in \mathbb{F}_{p^2}^2$ from 3_iso_curve, and $P = (X_P : Z_P)$ where $P \in E_{A/C}$ **Output:** $(X_{P'} : Z_{P'})$ corresponding to $P' \in E_{A'/C'}$, where $E_{A'/C'}$ is 3-isogenous to $E_{A/C}$

```

1  $t_0 \leftarrow X_P + Z_P$ 
2  $t_1 \leftarrow X_P - Z_P$ 
3  $t_0 \leftarrow K_1 \cdot t_0$ 
4  $t_1 \leftarrow K_2 \cdot t_1$ 
5  $t_2 \leftarrow t_0 + t_1$ 
6  $t_0 \leftarrow t_1 - t_0$ 
7  $t_2 \leftarrow t_2^2$ 
8  $t_0 \leftarrow t_0^2$ 
9  $X_{P'} \leftarrow X_P \cdot t_2$ 
10  $Z_{P'} \leftarrow Z_P \cdot t_0$ 
11 return  $(X_{P'} : Z_{P'})$ 

```

2.4.1 Quaternion lattices The four elements $\alpha_1, \alpha_2, \alpha_3, \alpha_4 \in B_{p,\infty}$ are $\mathbb{Q}$ -linearly independent if $x_1\alpha_1 + x_2\alpha_2 + x_3\alpha_3 + x_4\alpha_4 = 0$ with $x_i \in \mathbb{Q}$ implies $x_1 = x_2 = x_3 = x_4 = 0$. In this case, $\Lambda = \alpha_1\mathbb{Z} + \alpha_2\mathbb{Z} + \alpha_3\mathbb{Z} + \alpha_4\mathbb{Z}$ is a quaternion lattice. Since $\{1, i, j, k\}$ is a basis of $B_{p,\infty}$, we have

$$(\alpha_1, \alpha_2, \alpha_3, \alpha_4) = (1, i, j, k)L,$$

where L is an invertible matrix over $\mathbb{Q}$. We call L the basis matrix of Λ . Given two quaternion lattices Λ_1 and Λ_2 with basis matrices L_1 and L_2 , respectively, the basic operations are as follows.

- **Equality:** The lattices Λ_1 and Λ_2 are equal if and only if L_1 and L_2 have the same Hermite normal form (HNF) [Coh93, § 2.4.2].
- **Sum:** The basis matrix of $\Lambda_1 + \Lambda_2$ is the HNF of the matrix $[L_1 | L_2]$.
- **Intersection:** Compute the dual lattices Λ_1^* and Λ_2^* of Λ_1 and Λ_2 , respectively. Then the intersection $\Lambda_1 \cap \Lambda_2$ is the dual of $\Lambda_1^* + \Lambda_2^*$.
- **Multiplication:** The product $\Lambda_1 \Lambda_2$ can be computed by $A_1 L_2 + A_2 L_2 + A_3 L_2 + A_4 L_2$, where A_i is the i -th column of L_1 .

Algorithm 15: TwolosogenyChainSmall($E, R = (X_R : Z_R), \{P_i\}_{i \in \mathcal{I}}, e$)

Input: An elliptic curve E with coefficient $(A_{24} : C_{24})$. A point $R = (X_R : Z_R) \in E$ of order 2^e . A list $\{P_i\}_{i \in \mathcal{I}}$ on E . A small integer e .

Output: The image curve E' under an isogeny $\varphi : E_0 \rightarrow E'$ of kernel $\langle R \rangle$. A list $\{Q_i\}_{i \in \mathcal{I}} = \{\varphi(P_i)\}_{i \in \mathcal{I}}$.

```

1   $(A'_{24} : C'_{24}) \leftarrow (A_{24} : C_{24})$ ;
2  for  $i \leftarrow 0$  to  $e - 1$  do
3       $K \leftarrow R$ ;
4      for  $j \leftarrow 0$  to  $e - i - 2$  do
5           $K \leftarrow \text{xDBL}(K, (A_{24} : C_{24}))$ ;
6      if  $i = 0$  then
7          if  $[2]K \neq 0_E$  then
8              raise Exception: "TwolosogenyChainSmall failed: wrong order"
9      if  $K = (0 : 1)$  then
10          $(A'_{24} : C'_{24}), \text{consts} \leftarrow 2\_iso\_curve\_singular((A'_{24} : C'_{24}))$ ;
11          $R \leftarrow 2\_iso\_eval\_singular(R, \text{consts})$ ;
12          $\{Q_i\} \leftarrow \{2\_iso\_eval\_singular(P_i, \text{consts})\}_{i \in \mathcal{I}}$ ;
13     else
14          $(A'_{24} : C'_{24}) \leftarrow 2\_iso\_curve(K)$ ;
15          $R \leftarrow 2\_iso\_eval(K, R)$ ;
16          $\{Q_i\}_{i \in \mathcal{I}} \leftarrow \{2\_iso\_eval(K, P_i)\}_{i \in \mathcal{I}}$ ;
17 return  $(A'_{24} : C'_{24}), \{Q_i\}$ ;

```

- **Containment:** Given an element $\alpha = (1, i, j, k)\mathbf{a} \in B_{p, \infty}$, we have $\alpha \in \Lambda$ if the vector X satisfying $LX = \mathbf{a}$ has integer entries.
- **Inclusion:** The lattice Λ_1 is contained in a lattice Λ_2 if all 4 basis vectors of Λ_1 are contained in Λ_2 , or equivalently if $\Lambda_1 + \Lambda_2 = \Lambda_2$.
- **Index:** When $\Lambda_1 \subseteq \Lambda_2$, the index of Λ_1 in Λ_2 is the order of the finite quotient group Λ_2/Λ_1 , which is equal to $|\det(L_1)/\det(L_2)|$.
- **Basis reduction:** A lattice is reduced when its basis is reduced with respect to the chosen inner product. One can use the L2 algorithm of [NS09], which takes as input a lattice basis and its associated Gram matrix and computes an (η, δ) -reduced basis for the same lattice.

Algorithm 16: ThreelosogenyChain($E, (X_R : Z_R), \{P_i\}_{i \in \mathcal{I}}, e$)

Input: An elliptic curve E with constants $(A_{24}^+ : A_{24}^-)$. A projective point $R = (X_R : Z_R) \in E$ of order 3^e . A list $\{P_i\}_{i \in \mathcal{I}}$ of points on E . An integer e .

Output: The image curve E' under an isogeny $\varphi : E_0 \rightarrow E'$ of kernel $\langle R \rangle$. A list $\{Q_i\}_{i \in \mathcal{I}} = \{\varphi(P_i)\}_{i \in \mathcal{I}}$

```

1  for  $i = e - 1$  to  $0$  do
2       $T \leftarrow R$ ;
3      for  $j = 1$  to  $i$  do
4           $T \leftarrow \text{xTPL}(T, (A_{24}^+ : A_{24}^-))$ 
5       $((A_{24}^+ : A_{24}^-), (K_1, K_2)) \leftarrow 3\_iso\_curve(T)$ ;
6      if  $i \neq 0$  then
7           $R \leftarrow 3\_iso\_eval((K_1, K_2), R)$ ;
8          for  $P_k, k \in \mathcal{I}$  do
9               $Q_k = (X_k : Z_k) \leftarrow 3\_iso\_eval((K_1, K_2), (X_k : Z_k))$ 
10 return  $(A_{24}^+ : A_{24}^-), \{Q_i\}_{i \in \mathcal{I}}$ ;

```

Algorithm 17: TwistIsogenyChainSmall($E, l, R, \{P_i\}_{i \in \mathcal{I}}$)

Input: An elliptic curve E . An integer L . A kernel generator $R \in E$ of order L .
A list $\{P_i\}_{i \in \mathcal{I}}$ on the twist curve of E .

Output: The image curve E' under an isogeny φ from the twist of E of kernel $\langle R \rangle$. A list $\{Q_i\}_{i \in \mathcal{I}} = \{\varphi(P_i)\}_{i \in \mathcal{I}}$

```

1  Decompose  $L$  to  $L = \prod_{i=1}^t p_i^{e_i}$ ;
2  for  $i = 1$  to  $t$  do
3      for  $j = e_i - 1$  to  $1$  do
4           $T \leftarrow R$ ;
5           $T \leftarrow [\prod_{\ell=i+1}^t p_\ell^{e_\ell}]T$ ;
6           $T \leftarrow [p_i^j]T$ ;
7           $E \leftarrow \text{p\_iso\_curve}(E, T)$ ;
8           $R \leftarrow \text{p\_iso\_eval}(R)$ ;
9          for  $k \in \mathcal{I}$  do
10              $Q_k \leftarrow \text{p\_iso\_eval}(E, P_k)$ 
11 return  $E, \{Q_i\}_{i \in \mathcal{I}}$ 

```

2.5 Quaternion orders and ideals

An order $\mathcal{O} \subseteq B_{p,\infty}$ is a subring of $B_{p,\infty}$ that is also a lattice of rank 4. Every element in an order $\mathcal{O}$ is said to be integral, since it has reduced trace and reduced norm in $\mathbb{Z}$. Given a basis $\langle \alpha_1, \alpha_2, \alpha_3, \alpha_4 \rangle$ of $\mathcal{O}$, the discriminant of $\mathcal{O}$ is defined as $\text{disc}(\mathcal{O}) = \det(\langle \alpha_i, \alpha_j \rangle)_{i,j \in \{1,2,3,4\}}$. The discriminant of an order is an integer and is independent of the choice of basis. An order is called maximal if it is not properly contained in any other order. The discriminant of a maximal order in $B_{p,\infty}$ is p^2 .

Let $\mathcal{O}$ be a maximal order. A left (right) integral ideal I of $\mathcal{O}$ is a sublattice of $\mathcal{O}$ satisfying $\mathcal{O}I \subseteq I$. Define the left order $\mathcal{O}_L(I)$ and the right order $\mathcal{O}_R(I)$ of I by

$$\mathcal{O}_L(I) = \{x \in B_{p,\infty} : xI \subseteq I\}, \quad \mathcal{O}_R(I) = \{x \in B_{p,\infty} : Ix \subseteq I\}.$$

Moreover, $\mathcal{O}_L(I) = \mathcal{O}$, and $\mathcal{O}_R(I)$ is also a maximal order, in which case we say that I connects $\mathcal{O}_L(I)$ and $\mathcal{O}_R(I)$. The reduced norm of I can be defined as $n(I) = \gcd(\{\text{nrd}(\alpha) : \alpha \in I\})$. Any ideal I can be written as $I = \mathcal{O}\alpha + \mathcal{O}\text{nrd}(I)$ for some $\alpha \in I$.

If the two ideals I and J satisfy $\mathcal{O}_R(I) = \mathcal{O}_L(J)$, then we can define the product IJ by the product of I and J as lattices. It follows that IJ is also an ideal and $\mathcal{O}_L(IJ) = \mathcal{O}_L(I)$ and $\mathcal{O}_R(IJ) = \mathcal{O}_R(J)$. The ideal norm is multiplicative with respect to ideal products.

Two orders $\mathcal{O}_1$ and $\mathcal{O}_2$ are equivalent if and only if there exists an element $\alpha \in B_{p,\infty}^\times$ such that $\mathcal{O}_1 = \alpha^{-1}\mathcal{O}_2\alpha$. Two left $\mathcal{O}$ -ideals I_1 and I_2 are equivalent if there exists $\beta \in B_{p,\infty}^\times$ such that $I_2 = I_1\beta$. Moreover, $\mathcal{O}_R(I_1)$ and $\mathcal{O}_R(I_2)$ are equivalent since $\beta\mathcal{O}_R(I_1) = \mathcal{O}_R(I_2)\beta$.

We can represent ideals by their lattices and compute equality, sum, intersection, inclusion, and multiplication using the corresponding lattice operations. Given $\alpha \in B_{p,\infty}$ and an integer N with $N \mid \text{nrd}(\alpha)$, one can compute $\mathcal{O}\alpha$ and $\mathcal{O}N$ by lattice multiplication, and the lattice sum $\mathcal{O}\alpha + \mathcal{O}N$ gives the ideal $I = \mathcal{O}\langle \alpha, N \rangle$ with norm N . Conversely, given an ideal I with norm N , we can find its generator by sampling arbitrary elements in I until finding one such that $\gcd(\text{nrd}(\alpha), N^2) = N$.

Every ideal in a maximal order has an inverse. Let $\bar{I} = \{\alpha : \bar{\alpha} \in I\}$ be the conjugate ideal of I . Since $I\bar{I} = n(I)\mathcal{O}_L(I)$ and $\bar{I}I = n(I)\mathcal{O}_R(I)$, the inverse of I is $I^{-1} = \frac{1}{n(I)}\bar{I}$.

Note that I^{-1} is not an ideal but a rank-4 lattice. The left and right orders of an ideal I can be computed using its inverse, since $II^{-1} = \mathcal{O}_L(I)$ and $I^{-1}I = \mathcal{O}_R(I)$.

For an ideal $I \subseteq B_{p,\infty}$, we need to compute an equivalent ideal J with bounded norm. The ideal $J = \chi_I(\alpha) = I \frac{\bar{\alpha}}{n(I)}$ is equivalent to I and has $n(J) = \text{nr}d(\bar{\alpha})/n(I)$. Let $\{\alpha_1, \alpha_2, \alpha_3, \alpha_4\}$ be a reduced basis of I . By randomly sampling constants $c_i \in \mathbb{Z}$, we can construct a primitive element $\beta = \sum_{i=1}^4 c_i \alpha_i$ such that the norm of $J = \chi_I(\alpha)$ is expected to be $\approx \sqrt{p}$ heuristically. **Alg.** 18 can output an equivalent ideal J satisfying $\ell \nmid n(J)$ with a certain probability.

Algorithm 18: RandomEquivalentIdeal _{ℓ} (I)

Input: I , a left $\mathcal{O}$ -ideal (bound.coeff is a system parameter).

Output: found: 1 if we find the target ideal, and 0 otherwise. $J \sim I$ has small norm.

```

1 counter  $\leftarrow$  0;
2 Compute a LLL-reduced basis  $\alpha_1, \alpha_2, \alpha_3, \alpha_4$  of  $I$ ;
3 if  $\ell \mid \gcd(\text{nr}d(\alpha_1), \text{nr}d(\alpha_2), \text{nr}d(\alpha_3), \text{nr}d(\alpha_4))$  then
4   return 0,  $\perp$ ;
5 while counter < bound.coeff do
6   counter  $\leftarrow$  counter + 1;
7   Sample  $c_1, c_2, c_3, c_4$  uniformly at random from  $[-b, \dots, b]$  for a bound  $b$ ;
8    $c \leftarrow \gcd(c_1, c_2, c_3, c_4)$ ;
9    $m \leftarrow \text{adic}_2(c)$ ;
10   $c_1 \leftarrow c_1/2^m, c_2 \leftarrow c_2/2^m, c_3 \leftarrow c_3/2^m, c_4 \leftarrow c_4/2^m$ ;
11   $\alpha \leftarrow c_1\alpha_1 + c_2\alpha_2 + c_3\alpha_3 + c_4\alpha_4$ ;
12   $J = I \frac{\bar{\alpha}}{n(I)}$ ;
13  if  $\text{nr}d(J) \bmod 3 \neq 0$  then
14    return 1,  $J$ ;
```

2.5.1 Solving norm equations The key generation and signing procedures in **SQIsign2d-push**^{1/2} often need to solve norm equations in the maximal order $\mathcal{O}_0 = \mathbb{Z} + i\mathbb{Z} + \frac{i+j}{2}\mathbb{Z} + \frac{1+k}{2}\mathbb{Z}$.

Cornacchia's algorithm Cornacchia's algorithm finds an integer solution to the quadratic equation $x^2 + ny^2 = m$, provided that one exists for the given m and n . Its efficiency depends heavily on the difficulty of computing a square root modulo m , which essentially reduces to factoring m . To preserve this efficiency, we assume that the algorithm first checks whether m is prime or nearly prime using trial factorization. If m meets this condition, trial factorization returns the factorization of m and the algorithm proceeds; otherwise, Cornacchia's algorithm outputs $\perp$. Given an integer m , we provide pseudocode for solving the quadratic equation $x^2 + ny^2 = m$.

Representing integers by the maximal order $\mathcal{O}_0$ In **SQIsign2d-push**^{1/2}, we often need to solve norm equations in the maximal order $\mathcal{O}_0 = \mathbb{Z} + i\mathbb{Z} + \frac{i+j}{2}\mathbb{Z} + \frac{1+k}{2}\mathbb{Z}$. Specifically, for a given integer $M > p$, we require an element $\alpha \in \mathcal{O}_0$ such that $\text{nr}d(\alpha) = M$. Write $\alpha = w + xi + y\frac{i+j}{2} + z\frac{1+k}{2} \in \mathcal{O}_0$. Its reduced norm is $\text{nr}d(\alpha) = (w+z/2)^2 + (x+y/2)^2 + p/4(y^2 + z^2)$. Multiplying by 4 gives the equivalent equation

$$4M = (2w + z)^2 + (2x + y)^2 + p(y^2 + z^2).$$

To solve this equation, we first sample y uniformly from $[-m, m]$ with $m = \lfloor \sqrt{4M/p} \rfloor$, and then sample z uniformly from $[-m', m']$ where $m' = \lfloor \sqrt{4M/p - y^2} \rfloor$. For each such pair (y, z) , the remaining unknowns w, x must satisfy $(2w + z)^2 + (2x + y)^2 = 4M - p(y^2 + z^2)$. This is a binary quadratic equation that can be solved using Cornacchia's algorithm.

Algorithm 19: Cornacchia(n, m)

Input: $n, m \in \mathbb{N}$
Output: $\perp$ or $x, y \in \mathbb{Z}$ such that $x^2 + ny^2 = m$

```

1 fac  $\leftarrow$  TrialFactorization( $m$ );
2 if  $fac = \perp$  then
3    $\perp$  return  $\perp$ ;
4 Determine whether  $-n$  is a square modulo  $m$  using fac;
5 if  $-n$  is not a square modulo  $m$  then
6    $\perp$  return  $\perp$ ;
7 Compute  $r_{-2}$  as a square root of  $-n \pmod{m}$  using fac;
8 Set  $r_{-1} = m$  and  $i = -1$ ;
9 repeat
10    $i \leftarrow i + 1$ ;
11    $r_i \leftarrow r_{i-2} \bmod r_{i-1}$ ;
12 until  $r_i^2 < m < r_{i-1}^2$ ;
13 if  $m - r_i^2 \not\equiv 0 \pmod{n}$  then
14   Try again with another square root of  $-n \pmod{m}$ ;
15   if no such square root exists then
16      $\perp$  return  $\perp$ ;
17  $x \leftarrow r_i$ ;
18  $y \leftarrow \sqrt{(m - x^2)/n}$ ;
19 if  $y \notin \mathbb{Z}$  then
20   Try again with another square root of  $-n \pmod{m}$ ;
21   if no such square root exists then
22      $\perp$  return  $\perp$ ;
23 return  $x, y$ ;
```

We give the pseudocode in the following algorithm, $\text{FullRepresentInteger}_{\mathcal{O}_0}$. The algorithm $\text{FullRepresentInteger}_{\mathcal{O}_0}$ is heuristically believed to return well-distributed solutions in $\mathcal{O}_0$.

2.6 Isogenies between principally polarized abelian surfaces

In $\text{SQsign2d-push}^{1/2}$, to efficiently compute isogenies between elliptic curves of non-smooth degree, we introduce two-dimensional isogenies between principally polarized abelian surfaces (PPAS). A PPAS is a natural generalization of an elliptic curve to two dimensions. Over an algebraically closed field, every PPAS is isomorphic to one of the following [OU73]:

1. a product of elliptic curves $E_1 \times E_2$;
2. the Jacobian $\text{Jac}(C)$ of a genus-2 hyperelliptic curve C .

For the first type, the arithmetic on the Cartesian product $A := E_1 \times E_2$ follows directly from that of elliptic curves. Indeed, the addition of two points $(P_1, P_2), (Q_1, Q_2) \in A$ is defined as

$$(P_1, P_2) \oplus (Q_1, Q_2) := (P_1 \oplus_1 Q_1, P_2 \oplus_2 Q_2),$$

where $\oplus_1$ and $\oplus_2$ are the group laws on E_1 and E_2 , respectively. The group law $\oplus$ has identity $O_A = (O_{E_1}, O_{E_2})$.

We now briefly describe the second type of PPAS. Every genus-2 hyperelliptic curve defined over $\mathbb{F}_{p^2}$ can be written as

$$C : y^2 = f(x),$$

where f is squarefree and $\deg(f) = 5$ or 6 , provided $p > 5$. Unlike elliptic curves, the curve C does not form a group under point addition. Instead, we construct its Jacobian $\text{Jac}(C)$

Algorithm 20: FullRepresentInteger_{O₀}(M)

Input: An integer $M > p$
Output: $\alpha = w + xi + y \frac{i+j}{2} + z \frac{1+k}{2}$ with $\text{nrd}(\alpha) = M$ or raise an exception

```

1 Set counter  $\leftarrow 0$  and bound  $\leftarrow \lfloor \sqrt{4M/p} \rfloor^2$ ;
2 repeat
3    $m \leftarrow \lfloor \sqrt{4M/p} \rfloor$ ;
4   Sample a random integer  $y \in [-m, m]$ ;
5    $m' \leftarrow \lfloor \sqrt{4M/p - y^2} \rfloor$ ;
6   Sample a random integer  $z \in [-m', m']$ ;
7    $M' \leftarrow 4M - p(y^2 + z^2)$ ;
8   if Cornacchia(1,  $M'$ ) =  $\perp$  then
9     counter = counter + 1;
10    go back to sampling of  $(y, z)$ ;
11    $(w, x) \leftarrow \text{Cornacchia}(1, M')$ ;
12   if  $w \not\equiv z \pmod{2}$  or  $x \not\equiv y \pmod{2}$  then
13     counter = counter + 1;
14     go back to sampling of  $(y, z)$ ;
15    $\alpha \leftarrow \frac{w-z}{2} + \frac{x-y}{2}i + y \frac{i+j}{2} + z \frac{1+k}{2}$ ;
16 until  $\alpha$  is found or counter  $\geq$  bound;
17 return  $\alpha$  or FullRepresentation failed;

```

[Mil86], which is the group associated to C . Note that for an elliptic curve E , we have $\text{Jac}(E) \cong E$, which is why the construction of the Jacobian can usually be avoided. The group law on $A := \text{Jac}(C)$ with identity O_A can be computed using Cantor's algorithms [Can87].

Let A_1 and A_2 be principally polarized abelian surfaces over a finite field. An isogeny $\Phi : A_1 \rightarrow A_2$ is a surjective morphism of algebraic groups with a finite kernel. In dimension $g = 1$ (elliptic curves), the theory is relatively straightforward: taking the quotient by any finite subgroup uniquely determines another elliptic curve, which inherently carries a canonical principal polarization. In higher dimensions, however, ensuring that the quotient variety inherits a principal polarization imposes strict structural constraints. To understand these constraints, recall that a principal polarization on A_1 induces an isomorphism between A_1 and its dual $\widehat{A}_1$. For any integer N , this isomorphism endows the N -torsion subgroup $A_1[N]$ with a non-degenerate alternating symplectic form known as the *Weil pairing*, denoted by $e_N(\cdot, \cdot)$. Consequently, let K be a subgroup of $A_1[N]$. The necessary and sufficient condition for the quotient A_1/K to canonically inherit a principal polarization is that K must be a maximally isotropic subgroup with respect to the N -Weil pairing.

Building upon this framework, an isogeny Φ is defined as an (N, N) -isogeny if its kernel $K \subset A_1[N]$ is isomorphic to $(\mathbb{Z}/N\mathbb{Z})^2$. Such a kernel can be generated by a basis of two linearly independent N -torsion points, say $P, Q \in A_1[N]$. To satisfy the aforementioned isotropy condition, these generators must exhibit a trivial N -Weil pairing, meaning $e_N(P, Q) = 1$. When this pairing condition is met, the kernel is maximally isotropic, and the resulting quotient abelian surface $A_2 \cong A_1/\langle P, Q \rangle$ is guaranteed to inherit the principal polarization from A_1 .

Kani's lemma gives an connections between non-smooth isogenies of elliptic curves and some smooth isogenies of PPAS.

Theorem 1 (Kani's Lemma [Kan97]). *Let d_1, d_2 be pairwise coprime integers and $D = d_1 + d_2$. Let E_1, E_2, E_3 , and E_4 be elliptic curves connected by the following commu-*

tative diagram of isogenies:

$$\begin{array}{ccc} E_3 & \xrightarrow{\varphi_2} & E_2 \\ \varphi_1 \downarrow & \nearrow f & \downarrow \varphi'_1 \\ E_1 & \xrightarrow{\varphi'_2} & E_4 \end{array}$$

where $\varphi'_1 \circ \varphi_2 = \varphi'_2 \circ \varphi_1$, $f = \varphi_2 \circ \hat{\varphi}_1$ and $\deg(\varphi_i) = \deg(\varphi'_i) = d_i$ for $i = 1, 2$. Then, the map

$$\Phi = \begin{pmatrix} \hat{\varphi}_1 & -\hat{\varphi}_2 \\ \varphi'_2 & \varphi'_1 \end{pmatrix} : E_1 \times E_2 \rightarrow E_3 \times E_4$$

is a (D, D) -isogeny with respect to the natural product polarizations on $E_1 \times E_2$ and $E_3 \times E_4$, and has kernel $\{([d_2]P, f(P)) \mid P \in E_1[D]\}$.

Explicitly, given $E_1, E_2, d_1, d_2, D = 2^e$ and $\{f(P_1), f(P_2)\}$ with $\langle P_1, P_2 \rangle = E_1[2^e]$, we can compute the $(2^e, 2^e)$ -isogeny $\Phi : E_1 \times E_2 \rightarrow E_3 \times E_4$. Moreover, for $Q \in E_1$, the first component of $\Phi(Q, O_{E_2})$ is $\hat{\varphi}_1(Q)$ or $\hat{\varphi}_2(Q)$ (up to a post-composition isomorphism). We can determine which one is the case by computing the Weil pairing as follows. Let (Q_1, Q_2) be a basis of $E_1[2^{e'}]$ for $e' \geq 2$ and $(R_1, S_1) = \Phi(Q_1, O_{E_2})$, $(R_2, S_2) = \Phi(Q_2, O_{E_2})$. Then, we have $e_{2^{e'}}(R_1, R_2) = e_{2^{e'}}(Q_1, Q_2)^{d_1}$ if $R_1 = \varphi_1(\hat{Q}_1)$, and $e_{2^{e'}}(R_1, R_2) = e_{2^{e'}}(Q_1, Q_2)^{d_2}$ if $R_1 = \varphi_2(\hat{Q}_1)$. Since $d_1 \equiv -d_2 \pmod{2^e}$ and $\gcd(d_1, d_2) = 1$, we have $d_1 \not\equiv d_2 \pmod{2^{e'}}$.

We give an **Alg.** 47 to compute a $(2^e, 2^e)$ -isogenies $\Phi : E_1 \times E_2 \rightarrow E_3 \times E_4$ between elliptic curve products $E_1 \times E_2$ and $E_3 \times E_4$ in Section 7.

3 The transform between isogenies and ideals

In this section, we introduce the Deuring correspondence, with a particular focus on algorithms that convert between isogenies and ideals.

3.1 Deuring correspondence

An endomorphism of an elliptic curve E is an isogeny from E to itself. For an elliptic curve E defined over $\mathbb{F}_{p^2}$, the Frobenius map $\pi : (x, y) \mapsto (x^{p^2}, y^{p^2})$ is an inseparable endomorphism. Its characteristic polynomial is $x^2 - tx + p^2$, where t denotes the trace of π . It is well known that E is supersingular if $p \mid t$ and ordinary otherwise. Moreover, the j -invariant of any supersingular elliptic curve over $\overline{\mathbb{F}_{p^2}}$ lies in $\mathbb{F}_{p^2}$ and is called a supersingular j -invariant.

The set $\text{End}(E)$ of all endomorphisms of E (including the zero map) over $\overline{\mathbb{F}_{p^2}}$ forms a ring under addition and composition (as multiplication). For a supersingular elliptic curve E over $\mathbb{F}_{p^2}$, Deuring [Deu41] proved that $\text{End}(E)$ is isomorphic to a maximal order in the quaternion algebra $B_{p,\infty}$. This correspondence in fact gives an equivalence of categories between supersingular elliptic curves and left ideals of a maximal order $\mathcal{O}$ in $\mathcal{B}_{p,\infty}$.

Given a supersingular elliptic curve E over $\mathbb{F}_{p^2}$, we assume that its endomorphism ring is isomorphic to a maximal order $\mathcal{O}$ in $B_{p,\infty}$. The explicit correspondence between isogenies and ideals is given via kernel ideals [Wat69]. For an integral left $\mathcal{O}$ -ideal I , we define the kernel of I as

$$E[I] = \{P \in E(\overline{\mathbb{F}_{p^2}}) : \alpha(P) = O_E \text{ for all } \alpha \in I\}$$

which is a finite group, and we associate the isogeny φ_I with kernel $E[I]$, i.e., $\varphi_I : E \rightarrow E/E[I]$. Conversely, given an isogeny φ , the corresponding kernel ideal is

$$I_\varphi = \{\alpha \in \mathcal{O} : \alpha(P) = O_E \text{ for all } P \in \ker(\varphi)\}.$$

In **SQIsign2d-push**^{1/2}, we always use a prime $p \equiv 3 \pmod{4}$. In this case, the curve

$$E_0 : y^2 = x^3 + x$$

is supersingular, and its endomorphism ring is isomorphic to

$$\mathcal{O}_0 = \mathbb{Z} + i\mathbb{Z} + \frac{i+j}{2}\mathbb{Z} + \frac{1+k}{2}\mathbb{Z}.$$

The isomorphism is given by sending i to the automorphism $(x, y) \mapsto (-x, \sqrt{-1}y)$ and j to the Frobenius endomorphism $(x, y) \mapsto (x^p, y^p)$ on E_0 .

For the remainder of this document, we fix this choice of E_0 , $\mathcal{O}_0$, and the isomorphism $\varepsilon : \text{End}(E_0) \cong \mathcal{O}_0$. Let $\varphi : E_0 \rightarrow E$ be a d -isogeny. Then the isogeny φ can be associated to a left $\mathcal{O}_0$ -ideal I_φ of norm d , and its right order $(\mathcal{O}_R(I_\varphi) =) \mathcal{O}$ is isomorphic to $\text{End}(E)$. In particular, φ induces an isomorphism from $\text{End}(E)$ to $\mathcal{O}$ defined by

$$\alpha \mapsto \frac{1}{\deg \varphi} \varepsilon(\hat{\varphi} \circ \alpha \circ \varphi).$$

3.1.1 Pushforward and pullback Let $\varphi_1 : E \rightarrow E_1$ and $\varphi_2 : E \rightarrow E_2$ be two separable isogenies with coprime degrees. Then $\varphi_1(\ker \varphi_2)$ and $\varphi_2(\ker \varphi_1)$ are subgroups of E_1 and E_2 of orders $\deg \varphi_2$ and $\deg \varphi_1$, respectively. The pushforward of φ_2 through φ_1 is the isogeny $\psi_2 = [\varphi_1]_* \varphi_2$ defined by $\ker \psi_2 = \varphi_1(\ker \varphi_2)$. Similarly, the pushforward of φ_1 through φ_2 is $\psi_1 = [\varphi_2]_* \varphi_1$ with $\ker \psi_1 = \varphi_2(\ker \varphi_1)$. These isogenies fit into the commutative diagram shown in Fig. 1. Moreover, one can verify that $\hat{\varphi}_1(\ker \psi_2) = \ker \varphi_2$, which justifies calling φ_2 the pullback of ψ_2 through φ_1 , denoted $\varphi_2 = [\varphi_1]^* \psi_2$; likewise, $\varphi_1 = [\varphi_2]^* \psi_1$.

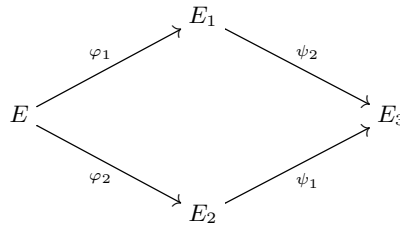


Fig. 1: Commutative diagram of pushforward and pullback isogenies

Assume that the isogenies φ_1 and φ_2 correspond to the left ideals I_1 and I_2 , respectively. Then the pushforward isogeny $\psi_2 = [\varphi_1]_* \varphi_2$ corresponds to the ideal [DFKL⁺20, Lemma 3]

$$[I_1]_* I_2 = \frac{1}{n(I_1)} \bar{I}_1 (I_1 \cap I_2),$$

and similarly $\psi_1 = [\varphi_2]_* \varphi_1$ corresponds to $[I_2]_* I_1$. Note that $I_1 [I_1]_* I_2 = I_1 \cap I_2$ if $\gcd(n(I_1), n(I_2)) = 1$. Moreover, if the isogenies ψ_1 and ψ_2 correspond to the ideals J_1 and J_2 , respectively, then the pullback isogenies $\varphi_2 = [\varphi_1]^* \psi_2$ and $\varphi_1 = [\varphi_2]^* \psi_1$ correspond to the ideals $[I_1]^* [J_2] = [\bar{I}_1]_* J_2$ and $[I_2]^* [J_1] = [\bar{I}_2]_* J_1$ respectively.

3.2 Converting between ideals and isogenies

Let E be a supersingular elliptic curve defined over $\mathbb{F}_{p^2}$. Assume that its endomorphism ring is isomorphic to a maximal order $\mathcal{O} = \mathbb{Z}\alpha_1 + \mathbb{Z}\alpha_2 + \mathbb{Z}\alpha_3 + \mathbb{Z}\alpha_4$.

3.2.1 Ideals to isogenies Given an ideal $I = \mathcal{O}\alpha + \mathcal{O}N$ with $N \mid \text{nrd}(\alpha)$, we can write $\alpha = x_1\alpha_1 + x_2\alpha_2 + x_3\alpha_3 + x_4\alpha_4$, where $x_i \in \mathbb{Z}$ and $\gcd(x_1, x_2, x_3, x_4) = 1$. Let $E[N] = \langle P, Q \rangle \subseteq E(\mathbb{F}_{p^2})$. Using the isomorphism $\varepsilon : \text{End}(E) \rightarrow \mathcal{O}$, we precompute the actions of $\alpha_1, \alpha_2, \alpha_3, \alpha_4$ on the basis of $E[N]$. In particular,

$$[\alpha_1](P, Q) = (P, Q)\mathbb{M}_1, [\alpha_2](P, Q) = (P, Q)\mathbb{M}_2,$$

$$[\alpha_3](P, Q) = (P, Q)\mathbb{M}_3, [\alpha_4](P, Q) = (P, Q)\mathbb{M}_4,$$

and $\mathbb{M}_\alpha = x_1\mathbb{M}_1 + x_2\mathbb{M}_2 + x_3\mathbb{M}_3 + x_4\mathbb{M}_4 \in M^{2 \times 2}(\mathbb{Z}/N\mathbb{Z})$. Then we compute a vector $(a, b)^\top \in \ker \mathbb{M}_\alpha$, which gives the corresponding generator $[a]P + [b]Q$ of the desired isogeny.

We denote by **KernelTolsogeny** an algorithm that takes as input an elliptic curve point and outputs the separable isogeny whose kernel is generated by that point, for instance using the appropriate choice of Vélú or $\sqrt{\text{élú}}$ [BFLS20].

Algorithm 21: IdealTolsogenyEvaluation $_N(I, P, Q, R, S)$

Input: I : a left $\mathcal{O}$ -ideal with $n(I) \mid N$; $P, Q \in E[N]$; $R, S \in E(\mathbb{F}_{p^2})$ with order coprime to N .

Output: $\varphi_I(R), \varphi_I(S)$

- 1 Compute α such that $I = \mathcal{O}\langle \alpha, \text{nrd}(I) \rangle$;
 - 2 Let $\mathbf{B} = [\alpha_1, \alpha_2, \alpha_3, \alpha_4]$ be a basis of $\mathcal{O}$;
 - 3 Compute $\mathbf{v}_\alpha := [x_1, x_2, x_3, x_4]^\top \in \mathbb{Z}^4$ satisfying $\mathbf{B}\mathbf{v}_\alpha = \alpha$;
 - 4 $\mathbf{M}_\alpha := x_1\mathbb{M}_1 + x_2\mathbb{M}_2 + x_3\mathbb{M}_3 + x_4\mathbb{M}_4$;
 - 5 Compute $[a, b]^\top \in \ker \mathbf{M}_\alpha$;
 - 6 $K := [a]P + [b]Q$;
 - 7 $\varphi_I = \text{KernelTolsogeny}(K)$;
 - 8 **return** $\varphi_I(R), \varphi_I(S)$;
-

Remark 1. In **SQlsign2d-push**^{1/2}, we often apply **IdealTolsogenyEvaluation** for elliptic curve E_0 with N a smooth integer, and α can be computed by **FullRepresentationInteger** $_{\mathcal{O}_0}$.

3.2.2 Isogenies to ideals Let $\varphi : E \rightarrow E/E[I]$ be an isogeny with $\ker \varphi = \langle P \rangle$ of order N . First, compute $\alpha_1(P), \alpha_2(P), \alpha_3(P), \alpha_4(P)$. Then $E[N] = \langle \alpha_i(P), \alpha_j(P) \rangle$ for some $i, j \in \{1, 2, 3, 4\}$. Using discrete logarithms via Weil pairings, we obtain a, b such that $\alpha_k(P) = a\alpha_i(P) + b\alpha_j(P)$ for $k \neq i, j$. Define $\alpha = a\alpha_i + b\alpha_j - \alpha_k$; then $I = \langle \alpha, N \rangle$ is the kernel ideal of φ .

Algorithm 22: KernelTolideal $_N(P)$

Input: A point P of order N in $E[N]$

Output: The ideal I corresponding to the isogeny with kernel $\langle P \rangle$

- 1 Compute $\varepsilon : \text{End}(E) \hookrightarrow B_{p,\infty}$ and set $\mathcal{O} = \varepsilon(\text{End}(E))$;
 - 2 Compute a basis $\alpha_1, \alpha_2, \alpha_3, \alpha_4$ of $\mathcal{O}$ where each α_i has norm coprime to N ;
 - 3 Find i, j such that $\alpha_i(P), \alpha_j(P)$ is a basis of $E[N]$;
 - 4 Take $k \neq i, j$ and compute $a, b \in \mathbb{Z}/N\mathbb{Z}$ such that $\alpha_k(P) = a\alpha_i(P) + b\alpha_j(P)$;
 - 5 Compute $\alpha = \alpha_k - a\alpha_i - b\alpha_j$;
 - 6 **return** $\mathcal{O}\langle \alpha, N \rangle$;
-

3.2.3 Generating two isogeny paths Let $p = c \cdot 2^{e_1} \cdot 3^{e_2} - 1$ with $c \in \mathbb{N}^*$ small. Let (P, Q) be a basis of $E_0[2^{e_1}3^{e_2}]$, $(P_0, Q_0) = [3^{e_2}](P, Q)$ a basis of $E_0[2^{e_1}]$, $(U_0, V_0) = [2^{e_1}](P, Q)$ a basis of $E_0[3^{e_2}]$. The algorithm **FastDoublePath** [DLRW24] is used to generate two isogeny paths $\Phi, \Phi' : E_0 \rightarrow E$ of degree dividing $2^{2e_1} \approx p$ and $3^{2e_2} \approx p$ respectively. This algorithm can be divided into the following steps:

1. Find $\gamma \in \mathcal{O}_0$ of norm $2^{2e_1}3^{2e_2}$. Decompose $\gamma = \widehat{\rho}_2 \circ \rho_1$, where ρ_1 and ρ_2 are isogenies $E_0 \rightarrow E'$ of degree $2^{e_1}3^{e_2}$. The kernel of ρ_1 and ρ_2 can be computed by Lemma 1 and **Alg. 23**.
2. Decompose ρ_1 and ρ_2 into $\rho_1 = \widehat{\theta}'_1 \circ \theta_1$ and $\rho_2 = \widehat{\theta}_2 \circ \theta'_2$, where θ_i are isogenies of degree 2^{e_1} and θ'_i are isogenies of degree 3^{e_2} .
3. Compute the pushforward isogenies $[\theta'_1]_*\theta_2$ and $[\theta_2]_*\theta'_1$.
4. Isogenies $\Phi := [\theta'_1]_*\theta_2 \circ \theta_1$ and $\Phi' := [\theta_2]_*\theta'_1 \circ \theta'_2$ are desired.

Lemma 1. [DLRW24] *Let $\rho : E \rightarrow E'$ be a cyclic isogeny decomposed into $\rho = \theta \circ \rho_1$. Then we have:*

1. $\ker(\rho_1) = \ker(\rho) \cap E[d_1]$ with $d_1 := \deg(\rho_1)$.
2. If ρ is a cyclic endomorphism, then the kernel ideal of ρ_1 is $K_1 = \mathcal{O}\rho + \mathcal{O}d_1$, where $\mathcal{O} := \text{End}(E)$.

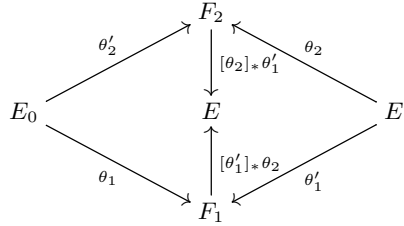


Fig. 2: Commutative diagram of **DoublePath**

Algorithm 23: $\text{KernelofDecomp}(\rho, d_1, d_2, (P_1, Q_1), (P_2, Q_2))$

Input: A cyclic isogeny $\rho : E \rightarrow E'$ of degree d . Degrees d_1 and d_2 of isogenies ρ_1 and ρ_2 such that $\rho = \rho_2 \circ \rho_1$. A basis (P_1, Q_1) of $E[d_1]$ and a basis (P_2, Q_2) of $E'[d_2]$.

Output: A generator G_1 of $\ker(\rho_1)$ and a generator G_2 of $\ker(\widehat{\rho}_2)$.

- 1 $(x_1, x_2) \leftarrow \text{TorsionDesLog}((P_1, Q_1), \rho(P), d)$;
 - 2 $(x_3, x_4) \leftarrow \text{TorsionDesLog}((P, Q), \rho(Q), d)$;
 - 3 $(x'_1, x'_2) \leftarrow \text{TorsionDesLog}((P_2, Q_2), \widehat{\rho}(P), d)$;
 - 4 $(x'_3, x'_4) \leftarrow \text{TorsionDesLog}((P_2, Q_2), \widehat{\rho}(Q), d)$;
 - 5 Find nonzero $a, b \in \mathbb{Z}/d_1\mathbb{Z}$ such that $ax_2 + bx_4 = 0$ and $ax_1 + bx_3 = 0$;
 - 6 Find nonzero $a', b' \in \mathbb{Z}/d_2\mathbb{Z}$ such that $a'x_2 + b'x_4 = 0$ and $a'x_1 + b'x_3 = 0$;
 - 7 $G_1 \leftarrow [a]P_1 + [b]Q_1$;
 - 8 $G_2 \leftarrow [a']P_2 + [b']Q_2$;
 - 9 **return** G_1, G_2
-

Remark 2. **DoublePath** for $\text{mode} = 0$ will be used in Key Generation of **SQIsign2d-push**^{1/2}. **DoublePath** for $\text{mode} = 1$ will be used in Commitment of **SQIsign2d-push**^{1/2}.

4 The **SQIsign2d-push**^{1/2} Signature Scheme

In this section, we elaborate in detail on the steps of the **SQIsign2d-push**^{1/2} signature scheme, including the algorithms for key generation, signing, and verification. This scheme is an improved version of **SQIsign2d-push**[NO25].

Algorithm 24: DoublePath(mode)

Input: A bit mode
Output: If mode = 0: an element $\gamma \in \mathcal{O}_0$ of norm $2^{2e_1}3^{2e_2}$, two kernel ideals J and J' , a basis (P_1, Q_1) of $E[2^{e_1}]$, and a basis (U_1, V_1) of $E[3^{e_2}]$ where E is isogenous to E_0 ; Otherwise: an elliptic curve E_m isogenous to E_0 , two kernel subgroups $K_0 \subseteq E_0$ and $K_1 \subseteq E_m$, a basis (P_m, Q_m) of $E_m[2^{e_1}]$, (P_1, Q_1) and J' .

```

1  $\gamma \leftarrow \text{FullRepresentInteger}_{\mathcal{O}_0}(2^{2e_1}3^{2e_2});$ 
2  $(G_1, G_2) \leftarrow \text{KernelofDecomp}(\gamma, 2^{e_1}3^{e_2}, 2^{e_1}3^{e_2}, (P, Q), (P, Q));$ 
3 Compute  $\rho_i : E_0 \rightarrow E'$  of kernel  $\mathcal{G}_i = \langle G_i \rangle$ ; // Decompose  $\gamma = \hat{\rho}_2 \circ \rho_1$ 
4  $(P_2, Q_2) \leftarrow \text{TorsionofBasis}(E', 2, e_1);$ 
5  $(P_3, Q_3) \leftarrow \text{TorsionofBasis}(E', 3, e_2);$ 
6  $(K_1, K'_1) \leftarrow \text{KernelofDecomp}(\rho_1, 2^{e_1}, 3^{e_2}, (P_0, Q_0), (P_3, Q_3));$ 
7  $(K_2, K'_2) \leftarrow \text{KernelofDecomp}(\rho_2, 3^{e_2}, 2^{e_1}, (U_0, V_0), (P_2, Q_2));$ 
8 Compute isogenies  $\theta_1 : E_0 \rightarrow F_1$  with kernel  $\mathcal{H}_1 = \langle K_1 \rangle$  and  $\theta'_1 : E' \rightarrow F_1$  with kernel  $\mathcal{H}'_1 = \langle K'_1 \rangle$ ;
   // Decompose  $\rho_1 = \hat{\theta}'_1 \circ \theta_1$ 
9 Compute isogenies  $\theta_2 : E' \rightarrow F_2$  with kernel  $\mathcal{H}_2 = \langle K_2 \rangle$  and  $\theta'_2 : E_0 \rightarrow F_2$  with kernel  $\mathcal{H}'_2 = \langle K'_2 \rangle$ ;
   // Decompose  $\rho_2 = \hat{\theta}_2 \circ \theta'_2$ 
10  $E_m \leftarrow F_2, P_m \leftarrow \theta'_2(P_0), Q_m \leftarrow \theta'_2(Q_0);$ 
11 Compute isogenies  $[\theta'_1]_*\theta_2 : F_1 \rightarrow E$  with kernel  $\theta'_1(K_2)$  and  $[\theta_2]_*\theta'_1 : E_m \rightarrow E$  with kernel
    $\theta_2(K'_1) = \langle K_m \rangle$ ;
12 Let  $\Phi := [\theta'_1]_*\theta_2 \circ \theta_1$  and  $\Phi' := [\theta_2]_*\theta'_1 \circ \theta'_2$  be isogenies from  $E_0$  to  $E$ ;
13 Compute  $P_1 = \Phi(P_0), Q_1 = \Phi(Q_0), U_1 = \Phi'(P_0)$  and  $V_1 = \Phi'(Q_0)$ ;
14 Let  $J := \mathcal{O}_0\gamma + \mathcal{O}_02^{2e_1}$  be the kernel ideal of  $\Phi$  and  $J' := \mathcal{O}_0\bar{\gamma} + \mathcal{O}_03^{2e_2}$  be the kernel ideal of  $\Phi'$ ;
15 if mode = 0 then
16   return  $(E, \gamma, J, J', P_1, Q_1, U_1, V_1)$ 
17 if mode = 1 then
18   return  $(E, K'_2, K_m, P_m, Q_m, P_1, Q_1, J')$ 

```

4.1 Σ protocols and the Fiat–Shamir Transform

The signature scheme $\text{SQLsign2d-push}^{1/2}$ is obtained from a three-move identification protocol of Σ -protocol type. At a high level, the prover demonstrates knowledge of a valid secret witness for the public key by producing a commitment, answering a random challenge, and returning a response that is consistent with both the commitment and the public key.

Let $\mathcal{R} \subseteq \mathcal{X} \times \mathcal{W}$ be the relation of valid statement-witness pairs underlying our construction. For $x \in \mathcal{X}$ and $w \in \mathcal{W}$, the condition $(x, w) \in \mathcal{R}$ means that w is a valid witness for the public statement x . In our setting, the statement x corresponds to a public key, and the witness w corresponds to the associated secret key. We denote by

$$\mathcal{L}_{\mathcal{R}} := \{x \in \mathcal{X} : \exists w \in \mathcal{W} \text{ such that } (x, w) \in \mathcal{R}\}$$

the language of valid public statements.

Definition 2 (Σ -protocol). Let $\mathcal{R} \subseteq \mathcal{X} \times \mathcal{W}$ be a relation and let $\mathcal{C}$ be challenge space. A Σ -protocol for $\mathcal{R}$ is a three-move interactive protocol consists of three probabilistic polynomial-time algorithms (P_1, P_2, V) , where V is deterministic and we assume that P_1 and P_2 share the same state. For any $(x, w) \in \mathcal{R}$, the protocol proceeds as follows:

- The prover computes a commitment $\text{com} \leftarrow P_1(x, w)$ and sends com to the verifier.
- The verifier samples a challenge string $\text{chl} \xleftarrow{\$} \mathcal{C}$ and sends chl to the prover.
- The prover computes $\text{rsp} \leftarrow P_2(x, w, \text{chl}, \text{com})$ and sends rsp to the verifier who will run $V(x, \text{com}, \text{chl}, \text{rsp})$ and outputs 1 if the transcript $(\text{com}, \text{chl}, \text{rsp})$ is valid and 0 otherwise.

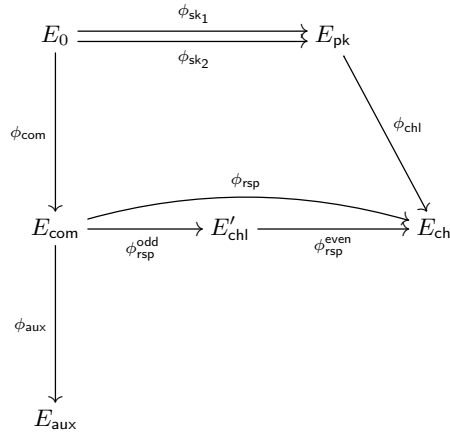
The above Σ -protocol is the interactive identification primitive underlying our signature scheme: the prover convinces the verifier that it knows a witness for the public statement x , without revealing the witness itself.

Fiat–Shamir transformation. To obtain a non-interactive signature scheme, we apply the Fiat–Shamir transform and replace the verifier’s random challenge by the output of a cryptographic hash function $H : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$. More precisely, on input a message m , the signer first computes $\text{com} \leftarrow P_1(x, w)$ then sets $\text{chl} := H(\text{com}, m)$ and finally computes $\text{rsp} \leftarrow P_2(x, w, \text{chl}, \text{com})$. Given a statement x , a message m , and a signature σ , the verifier parses σ , recomputes the challenge as $\text{chl} := H(\text{com}, m)$ and accepts if and only if the corresponding verification procedure succeeds.

4.2 The system parameters

We first describe our Σ -protocol underlying **SQIsign2d-push**^{1/2}. The Σ -protocol is based on the following diagram.

Fig. 3: The sketch of **SQIsign2d-push**^{1/2}.



The main system parameters of **SQIsign2d-push**^{1/2} is as follows.

- The prime p has the form $p = c \cdot 2^{e_1} \cdot 3^{e_2} - 1$, where $2^{e_1} \approx 3^{e_2} \approx \lambda$, and the integer c is chosen as small as possible.
- The integer $p - 1$ has a cofactor $L = \prod \ell_i^{m_i}$ such that $L \geq (\log p)^2$.
- The initial curve E_0 is defined by the equation $y^2 = x^3 + x$.
- $\langle P_0, Q_0 \rangle$ is a basis of $E_0[2^{e_1}]$, and $\langle U_0, V_0 \rangle$ is a basis of $E_0[3^{e_2}]$.
- The degree of the challenge isogeny is defined as $D_{\text{chl}} = 3^{e_2}$.
- The degree of the commitment isogeny is defined as $D_{\text{com}} = 3^{2e_2}$.
- The degree of the response isogeny is bounded by $D_{\text{rsp}} = 2^{e_{\text{rsp}}}$ where $e_{\text{rsp}} \leq e_1 - 1 \approx \log_2(\sqrt{p})$.

4.3 key generation

The key generation algorithm **SQIsign2d-push**^{1/2}.**KeyGen** is a procedure to generate a pair of public key pk and secret key sk . The public key pk is a random elliptic curve E_{pk} such that there exist two equivalent isogenies $\phi_{\text{sk1}} : E_0 \rightarrow E_{\text{pk}}$ and $\phi_{\text{sk2}} : E_0 \rightarrow E_{\text{pk}}$ of degree 2^{2e_1} and 3^{2e_2} respectively, which can be obtained by performing **Alg.** 24. The secret key sk consists of

- The public elliptic curve E_{pk} .
- The two points $P_{\text{pk}} = \phi_{\text{sk}2}(P_0)$, $Q_{\text{pk}} = \phi_{\text{sk}2}(Q_0)$.
- The ideal $I_{\text{sk}1}$ corresponding to the secret isogeny $\phi_{\text{sk}1}$.
- The quaternion element $\gamma \in \mathcal{O}_0$ of norm $2^{2e_1} \cdot 3^{2e_2}$ such that $I_{\text{sk}1} \cdot \overline{I_{\text{sk}2}} = \gamma \mathcal{O}_0$, where $I_{\text{sk}2}$ corresponds to $\phi_{\text{sk}2}$.
- The change-of-basis matrix $\mathbb{M}_{\text{sk}1}$ such that

$$(\tilde{U}_{\text{pk}}, \tilde{V}_{\text{pk}}) = (U_{\text{pk}}, V_{\text{pk}}) \cdot \mathbb{M}_{\text{sk}1} = (\phi_{\text{sk}1}(U_0), \phi_{\text{sk}1}(V_0)) \cdot \mathbb{M}_{\text{sk}1},$$

where $(\tilde{U}_{\text{pk}}, \tilde{V}_{\text{pk}})$ is a deterministic basis of $E_{\text{pk}}[3^{e_2}]$.

Algorithm 25: $\text{SQlsign2d-push}^{1/2}.\text{KeyGen}$

Output: Secret key sk and public key pk

- 1 $(E_{\text{pk}}, \gamma, I_{\text{sk}1}, I_{\text{sk}2}, P_{\text{pk}}, Q_{\text{pk}}, U_{\text{pk}}, V_{\text{pk}}) \leftarrow \text{DoublePath}(0)$;
 - 2 $\tilde{U}_{\text{pk}}, \tilde{V}_{\text{pk}} \leftarrow \text{TorsionofBasis}(E_{\text{pk}}, 3, e_2)$
 - 3 $\mathbb{M}_{\text{sk}1} \leftarrow \text{ChangeofBasis}(\tilde{U}_{\text{pk}}, \tilde{V}_{\text{pk}}, U_{\text{pk}}, V_{\text{pk}}, 3, e_2)$
 - 4 $\text{sk} \leftarrow (E_{\text{pk}}, P_{\text{pk}}, Q_{\text{pk}}, \gamma, I_{\text{sk}1}, \mathbb{M}_{\text{sk}1})$
 - 5 $\text{pk} \leftarrow E_{\text{pk}}$
 - 6 **return** sk, pk ;
-

4.4 Signing

$\text{SQlsign2d-push}^{1/2}$ is a digital signature scheme derived from an isogeny-based Σ -protocol. The signing procedure consists of three phases: *commitment*, *challenge*, and *response*. During the commitment phase, it computes a random D_{com} -isogeny $\phi_{\text{com}} : E_0 \rightarrow E_{\text{com}}$ and the corresponding ideal I_{com} . The main task of the challenge phase is to generate a D_{chl} -isogeny $\phi_{\text{chl}} : E_{\text{pk}} \rightarrow E_{\text{chl}}$ and the corresponding ideal I_{chl} . The kernel of ϕ_{chl} is determined by the challenge value chl , which is generated by a hash function on the inputs E_{pk} , E_{com} , and the message msg . The response phase is the core of the signing procedure, which needs to find an ideal I_{rsp} equivalent to $\overline{I_{\text{com}}} \cdot I_{\text{sk}1} \cdot I_{\text{chl}}$ such that $D_{\text{rsp}} < 2^{e_{\text{rsp}}}$ and obtain the corresponding isogeny $\phi_{\text{rsp}} : E_{\text{com}} \rightarrow E_{\text{chl}}$. Below, we provide more details on each phase of the signing procedure.

4.4.1 Commitment. At the commitment stage, we first sample a random endomorphism $\alpha \in \mathcal{O}_0$ of norm $2^{2e_1} \cdot 3^{2e_2}$ by **Alg. 20**. Then, the codomain of the commitment isogeny $\phi_{\text{com}} : E_0 \rightarrow E_{\text{com}}$ can also be obtain by **Alg. 24**. In fact, the commitment isogeny ϕ_{com} can be decomposed as $\phi_{\text{com}} = \phi_m \circ \phi_0$, where the degree of ϕ_m and ϕ_0 are both 3^{e_2} . The kernels of ϕ_0 and ϕ_m , and the images of ϕ_0 and ϕ_{com} evaluated on the basis $\langle P_0, Q_0 \rangle$ can be also obtained when performing **DoublePath**. The ideal I_{com} can be represented as $I_{\text{com}} = \mathcal{O}_0 \langle \alpha, D_{\text{com}} \rangle$.

4.4.2 Challenge. The challenge stage is quite simple, which only needs to obtain the challenge value chl by hashing the public key, the commitment curve and the message msg into $\mathbb{Z}_{D_{\text{chl}}}$. This value is used to determine the kernel of $\phi_{\text{chl}} : E_{\text{pk}} \rightarrow E_{\text{chl}}$. In more detail, $\text{Ker} \langle \phi_{\text{chl}} \rangle$ is given as $\langle \tilde{U}_{\text{pk}} + [\text{chl}] \tilde{V}_{\text{pk}} \rangle$, where $\langle \tilde{U}_{\text{pk}}, \tilde{V}_{\text{pk}} \rangle$ is a deterministic basis of $E_{\text{pk}}[D_{\text{chl}}]$.

Algorithm 26: GenerateAuxiliaryIsogeny

Input: K_0 : the kernel of ϕ_0 , K_m : the kernel of ϕ_m , $\langle P_0, Q_0 \rangle$: the basis of $E_0[2^{e_1}]$, E_m : the middle curve of the commitment isogeny, $\langle P_m, Q_m \rangle$: the basis of $E_m[2^{e_1}]$ where $P_m = \phi_0(P_0)$ and $Q_m = \phi_0(Q_0)$, E_{com} : the commitment curve, q_{aux} : the degree of the random auxiliary isogeny, e'_{rsp} : the number of $(2, 2)$ -isogeny

Output: E_{aux} : the codomain of the auxiliary isogeny $\phi_{\text{aux}} : E_{\text{com}} \rightarrow E_{\text{aux}}$, $\langle P_{\text{aux}}, Q_{\text{aux}} \rangle$: the image of $\langle P_{\text{com}}, Q_{\text{com}} \rangle$ under ϕ_{aux}

```

1   $\alpha \leftarrow \text{FullRepresentInteger}_{\mathcal{O}_0}(q_{\text{aux}} \cdot (2^{e'_{\text{rsp}}} - q_{\text{aux}}) \cdot 2^{e_1 - e'_{\text{rsp}}} \cdot L)$ 
2   $\text{cof} \leftarrow L^{-1} \bmod 2^{e_1}$ 
3   $P_\alpha \leftarrow [\text{cof}]\varepsilon(\alpha)(P_0)$ 
4   $Q_\alpha \leftarrow [\text{cof}]\varepsilon(\alpha)(Q_0)$ 
5   $K_\alpha \leftarrow \varepsilon(\alpha)(K_0)$ 
6   $R_0 \leftarrow E_0[2^{e_1 - e'_{\text{rsp}}}] \cap \ker(\varepsilon(\bar{\alpha}))$ 
7   $R_1 \leftarrow E_0[L] \cap \ker(\varepsilon(\bar{\alpha}))$ 
8   $F_0, \{R'_1, P'_\alpha, Q'_\alpha, K'_\alpha\} \leftarrow \text{TwolsogenyChainSmall}(E_0, R_0, \{R_1, P_\alpha, Q_\alpha, K_\alpha\}, e_1 - e'_{\text{rsp}})$ 
9   $E''_0, \{P''_0, Q''_0, K''_0\} \leftarrow \text{TwistIsogenyChainSmall}(F_0, R'_1, \{P'_\alpha, Q'_\alpha, K'_\alpha\}, L)$ 
10  $E''_m, \{P''_m, Q''_m\} \leftarrow \text{ThreelsogenyChain}(E''_0, K''_0, \{P''_0, Q''_0\}, e_2)$ 
11  $R_m \leftarrow [(2^{e_1 - e'_{\text{rsp}}}) \cdot (2^{e'_{\text{rsp}}} - q_{\text{aux}})]P_m$ ,  $S_m \leftarrow [(2^{e_1 - e'_{\text{rsp}}}) \cdot (2^{e'_{\text{rsp}}} - q_{\text{aux}})]Q_m$ 
12  $E'_m, F'_m, P'_m, Q'_m, K'_m \leftarrow$ 
     $2\text{X}2\text{sogenyChain}((R_m, P''_m), (S_m, Q''_m), e'_{\text{rsp}}, \{(P_m, O_{E''_m}), (Q_m, O_{E''_m}), (K_m, O_{E''_m})\})$ 
13  $E_{\text{aux}}, P_{\text{aux}}, Q_{\text{aux}} \leftarrow \text{ThreelsogenyChain}(E'_m, K'_m, \{P'_m, Q'_m\}, e_2)$ 
14 return  $E_{\text{aux}}, P_{\text{aux}}, Q_{\text{aux}}$ 

```

4.4.3 Response. Upon obtaining chl , the first step at the response stage is to compute the kernel $\langle U_{\text{pk}} + [\text{chl}]\tilde{V}_{\text{pk}} \rangle$ and codomain of ϕ_{chl} as above. Let $(a_0, b_0) \in \mathbb{Z}_{D_{\text{chl}}} \times \mathbb{Z}_{D_{\text{chl}}}$ such that

$$(a_0, b_0) = \mathbb{M}_{\text{sk}1} \cdot (1, \text{chl})^\top \bmod D_{\text{chl}}.$$

Then, the kernel of ϕ_{chl} can be represented by $\tilde{U}_{\text{pk}}$ and $\tilde{V}_{\text{pk}}$ as

$$\begin{aligned} \tilde{U}_{\text{pk}} + [\text{chl}]\tilde{V}_{\text{pk}} &= (\tilde{U}_{\text{pk}}, \tilde{V}_{\text{pk}})(1, \text{chl})^\top \\ &= (U_{\text{pk}}, V_{\text{pk}}) \cdot \mathbb{M}_{\text{sk}1} \cdot (1, \text{chl})^\top \\ &= [a_0]U_{\text{pk}} + [b_0]V_{\text{pk}}. \end{aligned}$$

Hence, the point $[a_0]U_0 + [b_0]V_0$ is preimage of $\tilde{U}_{\text{pk}} + [\text{chl}]\tilde{V}_{\text{pk}}$ under the isogeny $\phi_{\text{sk}1}$. Using the **KernelToIdeal** algorithm, we can find the ideal $I'_{\text{chl}} \subset \mathcal{O}_0$ corresponding to the kernel $\langle [a_0]U_0 + [b_0]V_0 \rangle$. Since the challenge ideal $I_{\text{chl}} = [I_{\text{sk}1}]_* I'_{\text{chl}}$, the ideal corresponding to the isogeny $\phi_{\text{chl}} \circ \phi_{\text{sk}2} \circ \hat{\phi}_{\text{com}}$ is given as

$$I = \overline{I_{\text{com}}} \cdot I_{\text{sk}2} \cdot I_{\text{chl}} = \overline{I_{\text{com}}} \cdot I_{\text{sk}1} \cdot I_{\text{chl}} \cdot \bar{\gamma}/n(I_{\text{sk}1}).$$

At the response stage, one of important step is to construct a response ideal I_{rsp} that satisfying the following conditions:

- (1). The response ideal I_{rsp} is equivalent to I with norm $n(I_{\text{rsp}}) < D_{\text{rsp}}$.
- (2). We can not extract a scalar $[2^\ell]$ from the corresponding response isogeny for any $\ell > 0$.
- (3). The norm $n(I_{\text{rsp}})$ is coprime to 3.

To construct such a ideal, we first sample uniformly a quaternion element $\alpha_{\text{rsp}} \in \overline{I_{\text{com}}} \cdot I_{\text{sk}1} \cdot I_{\text{chl}}$ such that $\text{nrd}(\alpha_{\text{rsp}}) < D_{\text{rsp}} \cdot n(I)$, which can be done using the LLL algorithm. We can treat α_{rsp} as an element of $\mathcal{O}_0$ and write it as $\alpha_{\text{rsp}} = a_0 + a_1i + a_2\frac{i+j}{2} + a_3\frac{1+k}{2}$, where $a_i \in \mathbb{Z}$. Let ℓ be the two adic evaluation of $\gcd(a_1, a_2, a_3, a_4)$. Then, we can update α_{rsp} as

$$\alpha_{\text{rsp}} = \tilde{a}_0 + \tilde{a}_1i + \tilde{a}_2\frac{i+j}{2} + \tilde{a}_3\frac{1+k}{2},$$

where $\tilde{a}_i = a_i/2^\ell$. It is straightforward to see that α_{rsp} is still in $\mathcal{O}_0$, and thus $\tilde{I} = I \cdot \bar{\alpha}_{\text{rsp}}/n(I)$ is equivalent to I . If $n(\tilde{I})$ is coprime to 3, we then set I_{rsp} as $\tilde{I}$; otherwise, we resample a quaternion element α_{rsp} until the condition meets. We summarize the details of computing $\tilde{I}$ in **Alg. 18**. As discussed in [NO25, Appendix C], the failure probability of **Alg. 18** is below 2^{-18} .

Based on the above analysis, the response isogeny ϕ_{rsp} can be written as

$$\phi_{\text{rsp}} = \phi_{\text{chl}} \circ \phi_{\text{sk2}} \circ \varepsilon(\bar{\alpha}_{\text{rsp}}) \circ \hat{\phi}_{\text{com}}/3^{5e_2}.$$

Let $\langle P_{\text{com}}, Q_{\text{com}} \rangle$ be the image of $\langle P_0, Q_0 \rangle$ under the commitment isogeny ϕ_{com} . Let $\langle P_{\text{rsp}}, Q_{\text{rsp}} \rangle$ be the image of $\langle P_{\text{com}}, Q_{\text{com}} \rangle$ under the response isogeny ϕ_{rsp} . Then,

$$\begin{aligned} (P_{\text{rsp}}, Q_{\text{rsp}}) &= [3^{-5e_2}] \phi_{\text{chl}} \circ \phi_{\text{sk2}} \circ \varepsilon(\bar{\alpha}_{\text{rsp}}) \circ \hat{\phi}_{\text{com}}(P_{\text{com}}, Q_{\text{com}}) \\ &= [3^{-3e_2}] \phi_{\text{chl}} \circ \phi_{\text{sk2}} \circ \varepsilon(\bar{\alpha}_{\text{rsp}})(P_0, Q_0) \\ &= [3^{-3e_2}] \phi_{\text{chl}}(P_{\text{pk}}, Q_{\text{pk}}) \mathbb{M}_{\bar{\alpha}} \\ &= [3^{-3e_2}](P_{\text{chl}}, Q_{\text{chl}}) \mathbb{M}_{\bar{\alpha}}, \end{aligned}$$

where $\mathbb{M}_{\bar{\alpha}}$ is the change-of-basis matrix such that

$$\varepsilon(\bar{\alpha}_{\text{rsp}})(P_0, Q_0) = (P_0, Q_0) \mathbb{M}_{\bar{\alpha}}.$$

The response isogeny ϕ_{rsp} can be decomposed as $\phi_{\text{rsp}} = \phi_{\text{rsp}}^{\text{even}} \circ \phi_{\text{rsp}}^{\text{odd}}$, where $\phi_{\text{rsp}}^{\text{odd}} : E_{\text{com}} \rightarrow E'_{\text{chl}}$ has an odd degree q_{rsp} and $\phi_{\text{rsp}}^{\text{even}} : E'_{\text{chl}} \rightarrow E_{\text{chl}}$ has an even degree $2^{n_{\text{rsp}}}$ for some non-negative integer n_{rsp} . It is clear that the integer $n_{\text{rsp}} = \text{adic}_2(n(I_{\text{rsp}}))$. We define that

$$e'_{\text{rsp}} = e_{\text{rsp}} + v, \quad q_{\text{aux}} = 2^{e'_{\text{rsp}}} - q_{\text{rsp}},$$

where v is an integer such that $\gcd(q_{\text{aux}}, 3) = 1$. Clearly, it is sufficient to select $v = 0$ or 1 . Since $e_{\text{rsp}} \leq e_1 - 1$, we have $e'_{\text{rsp}} \leq e_1$, which implies we can always find a $2^{e'_{\text{rsp}}}$ -torison group on $E(\mathbb{F}_{p^2})$. The computational cost at the response stage largely comes from generating a random q_{aux} -isogeny $\phi_{\text{aux}} : E_{\text{com}} \rightarrow E_{\text{aux}}$ and computing $\phi_{\text{aux}}(P_{\text{com}})$ and $\phi_{\text{aux}}(Q_{\text{com}})$. The procedure of this computation is presented in **Alg. 26**. We now illustrate it in detail.

Step 1: Constructing a random $q_{\text{aux}} \cdot q_{\text{rsp}}$ -isogeny starting from E_0 . Firstly, one can sample an endomorphism $\alpha_0 \in \mathcal{O}_0$ with norm $n(\alpha_0) = q_{\text{aux}} \cdot q_{\text{rsp}} \cdot 2^{e_1 - e'_{\text{rsp}}} \cdot L$. Indeed, since $n(\alpha_0) \approx p \log^2 p$, it follows from [BDF⁺25, Remark 2.6] that the target endomorphism can be sampled by executing the **FullRepresentInteger** $_{\mathcal{O}_0}$ algorithm, which succeeds with overwhelming probability. Then, one can obtain a $2^{e_1 - e'_{\text{rsp}}}$ -isogeny $\tau_0 : E_0 \rightarrow F_0$ with kernel $E_0[2^{e_1 - e'_{\text{rsp}}}] \cap \ker(\varepsilon(\bar{\alpha}_0))$ and an L -isogeny $\tau_1 : E_0 \rightarrow H_0$ with kernel $E_0[L] \cap \ker(\varepsilon(\bar{\alpha}_0))$. Since $\gcd(2^{e_1 - e'_{\text{rsp}}}, L) = 1$, using the *push-forward* technique, the above two isogenies induces a $2^{e_1 - e'_{\text{rsp}}} \cdot L$ -isogeny $\tau : E_0 \rightarrow E''_0$, where $\tau = [\tau_0]_* \tau_1 \circ \tau_0$. Hence, there exists a $q_{\text{aux}} \cdot q_{\text{rsp}}$ -isogeny $\rho_0 : E_0 \rightarrow E''_0$ such that

$$\varepsilon(\alpha_0) = \hat{\tau} \circ \rho_0,$$

wich implies that

$$[2^{e_1 - e'_{\text{rsp}}} \cdot L] \rho_0 = \tau \circ \varepsilon(\alpha_0).$$

Step 2: Constructing a q_{aux} -isogeny starting from E_m . Recall that the commitment isogeny ϕ_{com} can be decomposed as $\phi_{\text{com}} = \phi_m \circ \phi_0$, where $\phi_0 : E_0 \rightarrow E_m$ and $\phi_m : E_m \rightarrow E_{\text{com}}$ are two 3^{e_2} -isogenies with kernels K_0 and K_m , respectively. The images

of the pair (P_0, Q_0) under the maps ϕ_0 and ϕ_{com} are denoted as (P_m, Q_m) and $(P_{\text{com}}, Q_{\text{com}})$, respectively. Since $\gcd(\deg([\deg \tau]\rho_0), 3^{e_2}) = 1$, the point $K_0'' = \tau \circ \varepsilon(\alpha_0)(K_0) \in E_0''$ also has order 3^{e_2} . Thus, we can generate a 3^{e_2} -isogeny: $\phi_0'' : E_0'' \rightarrow E_m''$ with kernel $\langle K_0'' \rangle$. As shown in **Fig. 4**, there exists a $q_{\text{aux}} \cdot q_{\text{rsp}}$ -isogeny: $\rho_m : E_m \rightarrow E_m''$ such that

$$\rho_m \circ \phi_0 = \phi_0'' \circ \rho_0.$$

Thus, the images of (P_m, Q_m) under the isogeny ρ_m can be computed as

$$\rho_m(P_m, Q_m) = \rho_m \circ \phi_0(P_0, Q_0) = \phi_0'' \circ \rho_0(P_0, Q_0).$$

Write ρ_m as $\varphi_m' \circ \varphi_m$ such that $\deg(\varphi_m) = q_{\text{aux}}$ and $\deg(\varphi_m') = q_{\text{rsp}}$. It is obvious that the codomain of φ_m , denoted by E_m' , is our target isogeny. It actually can be obtained by computing the $(2^{e'_{\text{rsp}}}, 2^{e'_{\text{rsp}}})$ -isogeny Φ with kernel

$$\begin{aligned} & \langle ([2^{e_1 - e'_{\text{rsp}}} \cdot q_{\text{rsp}}]P_m, [2^{e_1 - e'_{\text{rsp}}}]\phi_0'' \circ \rho_0(P_0), ([2^{e_1 - e'_{\text{rsp}}} \cdot q_{\text{rsp}}]Q_m, [2^{e_1 - e'_{\text{rsp}}}]\phi_0'' \circ \rho_0(Q_0)) \rangle \\ &= \langle ([2^{e_1 - e'_{\text{rsp}}} \cdot q_{\text{rsp}}]P_m, [1/L]\phi_0'' \circ \tau \circ \varepsilon(\alpha_0)(P_0), ([2^{e_1 - e'_{\text{rsp}}} \cdot q_{\text{rsp}}]Q_m, [1/L]\phi_0'' \circ \tau \circ \varepsilon(\alpha_0)(Q_0)) \rangle. \end{aligned}$$

The $(2^{e'_{\text{rsp}}}, 2^{e'_{\text{rsp}}})$ -isogeny Φ can be represented by the following matrix

$$\Phi = \begin{pmatrix} \varphi_m & -\widehat{\varphi_m'} \\ [\varphi_m]^* \varphi_m' & [\varphi_m']^* \varphi_m \end{pmatrix} : E_m \times E_m'' \rightarrow E_m' \times F_m',$$

for some elliptic curve F_m' .

Step 3: Constructing the target q_{aux} -isogeny $\phi_{\text{aux}} : E_{\text{com}} \rightarrow E_{\text{aux}}$. Evaluating the two-dimensional isogeny Φ on $(K_m, O_{E_m''})$, we first obtain the point $K_m' = \varphi_m(K_m) \in E_m'$. Using the kernel $\langle K_m' \rangle$, we then generate a 3^{e_2} -isogeny $\phi_m' : E_m' \rightarrow E_{\text{aux}}$ such that $\phi_m' \circ \varphi_m = \phi_{\text{aux}} \circ \phi_m$. Finally, the images of the points P_{com} and Q_{com} under the auxiliary isogeny ϕ_{aux} , denoted by P_{aux} and Q_{aux} , are given as

$$\begin{aligned} (P_{\text{aux}}, Q_{\text{aux}}) &= \phi_{\text{aux}}(P_{\text{com}}, Q_{\text{com}}) \\ &= \phi_{\text{aux}} \circ \phi_m(P_m, Q_m) \\ &= \phi_m' \circ \varphi_m(P_m, Q_m). \end{aligned}$$

In total, the signing of **SQIsign2d-push**^{1/2} consists of

- The elliptic curve E_{aux} ;
- The challenge value chl ;
- The integer n_{rsp} ;
- The integer $v \in \{0, 1\}$;
- The matrix $\mathbb{M}_{\text{rsp}}$ such that

$$\phi_{\text{rsp}} \circ \hat{\phi}_{\text{aux}}(\tilde{P}_{\text{aux}}, \tilde{Q}_{\text{aux}}) = ([q_{\text{aux}}]\tilde{P}_{\text{chl}}, [q_{\text{aux}}]\tilde{Q}_{\text{chl}})\mathbb{M}_{\text{rsp}},$$

where $(\tilde{P}_{\text{aux}}, \tilde{Q}_{\text{aux}})$ and $(\tilde{P}_{\text{chl}}, \tilde{Q}_{\text{chl}})$ are the two deterministic basis of $E_{\text{aux}}[2^{e_1}]$ and $E_{\text{chl}}[2^{e_1}]$, respectively.

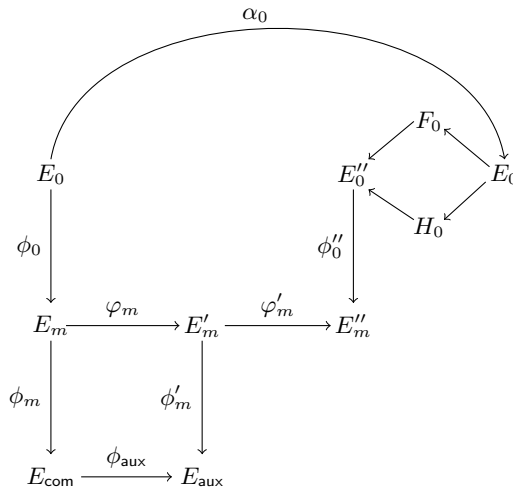
Using the **ChangeofBasis** algorithm (see **Alg. 6**), we compute the two matrixes $\mathbb{M}_{\text{chl}}$ and $\mathbb{M}_{\text{aux}}$ such that

$$(\tilde{P}_{\text{aux}}, \tilde{Q}_{\text{aux}}) = (P_{\text{aux}}, Q_{\text{aux}}) \cdot \mathbb{M}_{\text{aux}}, \quad (P_{\text{rsp}}, Q_{\text{rsp}}) = (\tilde{P}_{\text{chl}}, \tilde{Q}_{\text{chl}}) \cdot \mathbb{M}_{\text{chl}}.$$

Then the matrix $\mathbb{M}_{\text{rsp}} = \mathbb{M}_{\text{chl}} \cdot \mathbb{M}_{\text{aux}} \bmod 2^{e_1}$ as

$$\begin{aligned}
 \phi_{\text{rsp}} \circ \hat{\phi}_{\text{aux}}(\tilde{P}_{\text{aux}}, \tilde{Q}_{\text{aux}}) &= \phi_{\text{rsp}} \circ \hat{\phi}_{\text{aux}}(P_{\text{aux}}, Q_{\text{aux}}) \cdot \mathbb{M}_{\text{aux}} \\
 &= \phi_{\text{rsp}} \circ \hat{\phi}_{\text{aux}} \circ \phi_{\text{aux}}(P_{\text{com}}, Q_{\text{com}}) \mathbb{M}_{\text{aux}} \\
 &= \phi_{\text{rsp}}([q_{\text{aux}}]P_{\text{com}}, [q_{\text{aux}}]Q_{\text{com}}) \mathbb{M}_{\text{aux}} \\
 &= ([q_{\text{aux}}]P_{\text{rsp}}, [q_{\text{aux}}]Q_{\text{rsp}}) \cdot \mathbb{M}_{\text{aux}} \\
 &= ([q_{\text{aux}}]\tilde{P}_{\text{chl}}, [q_{\text{aux}}]\tilde{Q}_{\text{chl}}) \cdot \mathbb{M}_{\text{chl}} \cdot \mathbb{M}_{\text{aux}}.
 \end{aligned}$$

Fig. 4: The sketch of **Alg. 26**.



4.5 Verification

In the $\text{SQIsign2d-push}^{1/2}.\text{Verify}$ algorithm, the verifier needs to check that

- (a) the kernel of $\hat{\phi}_{\text{rsp}} \circ \phi_{\text{chl}}$ does not contain $E_{\text{pk}}[3]$;
- (b) the output of the $\text{SQIsign2d-push}^{1/2}.\text{Sign}$ algorithm is valid.

The condition (a) can guarantee the special soundness of $\text{SQIsign2d-push}^{1/2}$ (See Section 6). Let K_3 be a generator of $E_{\text{chl}}[3] \cap \ker(\hat{\phi}_{\text{chl}})$. Then the condition (a) is equivalent to $\hat{\phi}_{\text{rsp}}(K_3) \neq O_{E_{\text{com}}}$. The image $\hat{\phi}_{\text{rsp}}(K_3)$ can be obtained via a 2-dimensional evaluation.

In terms of the condition (b), the verifier first recovers the deterministic basis $(\tilde{U}_{\text{pk}}, \tilde{V}_{\text{pk}})$ of $E_{\text{pk}}[3^{e_2}]$, followed by generating the challenge isogeny $\phi_{\text{chl}} : E_{\text{pk}} \rightarrow E_{\text{chl}}$ with kernel $\langle \tilde{U}_{\text{pk}} + [\text{chl}]\tilde{V}_{\text{pk}} \rangle$. Then, the verifier computes the deterministic basis $\langle \tilde{P}_{\text{chl}}, \tilde{Q}_{\text{chl}} \rangle$ of $E_{\text{chl}}[2^{e_1}]$. To reconstruct the correct evaluation under the response isogeny, it is necessary to compute $(P_{\text{chl}}, Q_{\text{chl}}) = (\tilde{P}_{\text{chl}}, \tilde{Q}_{\text{chl}}) \cdot \mathbb{M}_{\text{rsp}}$. On the other side, the verifier also requires recovering the deterministic basis $\langle \tilde{P}_{\text{aux}}, \tilde{Q}_{\text{aux}} \rangle$ of $E_{\text{aux}}[2^{e_1}]$.

The response isogeny ϕ_{rsp} can be decomposed as $\phi_{\text{rsp}} = \phi_{\text{rsp}}^{\text{even}} \circ \phi_{\text{rsp}}^{\text{odd}}$, where $\deg(\phi_{\text{rsp}}^{\text{even}}) = 2^{n_{\text{rsp}}}$ and $\deg(\phi_{\text{rsp}}^{\text{odd}}) = q_{\text{rsp}}$. If $n_{\text{rsp}} \neq 0$, then $\deg(\phi_{\text{rsp}})$ is even and we denote by E'_{chl} the codomain of $\phi_{\text{rsp}}^{\text{odd}}$. Thus, the kernel of the dual isogeny $\hat{\phi}_{\text{rsp}}^{\text{even}} : E_{\text{chl}} \rightarrow E'_{\text{chl}}$ is given as $\langle [2^{e'_{\text{rsp}}}]P_{\text{chl}}, [2^{e'_{\text{rsp}}}]Q_{\text{chl}} \rangle$. Let $\hat{\phi}_{\text{rsp}}^{\text{even}}(P_{\text{chl}}, Q_{\text{chl}}) = (P'_{\text{chl}}, Q'_{\text{chl}})$. The images of $([2^{n_{\text{rsp}}}]\tilde{P}_{\text{aux}}, [2^{n_{\text{rsp}}}]\tilde{Q}_{\text{aux}})$

under $\phi_{\text{rsp}}^{\text{odd}} \circ \hat{\phi}_{\text{aux}}$ can be represented as

$$\begin{aligned} \phi_{\text{rsp}}^{\text{odd}} \circ \hat{\phi}_{\text{aux}}([2^{n_{\text{rsp}}}] \tilde{P}_{\text{aux}}, [2^{n_{\text{rsp}}}] \tilde{Q}_{\text{aux}})^{\top} &= \hat{\phi}_{\text{rsp}}^{\text{even}} \circ \phi_{\text{rsp}}^{\text{even}} \circ \phi_{\text{rsp}}^{\text{odd}} \circ \hat{\phi}_{\text{aux}}(\tilde{P}_{\text{aux}}, \tilde{Q}_{\text{aux}}) \\ &= \hat{\phi}_{\text{rsp}}^{\text{even}} \circ \phi_{\text{rsp}} \circ \hat{\phi}_{\text{aux}}(\tilde{P}_{\text{aux}}, \tilde{Q}_{\text{aux}}) \\ &= \hat{\phi}_{\text{rsp}}^{\text{even}}([q_{\text{aux}}] \tilde{P}_{\text{chl}}, [q_{\text{aux}}] \tilde{Q}_{\text{chl}}) \mathbb{M}_{\text{rsp}} \\ &= ([q_{\text{aux}}] P'_{\text{chl}}, [q_{\text{aux}}] Q'_{\text{chl}}). \end{aligned}$$

Let $(P'_{\text{aux}}, Q'_{\text{aux}}) = ([2^{(e_1 - e'_{\text{rsp}} + n_{\text{rsp}})}] \tilde{P}_{\text{aux}}, [2^{(e_1 - e'_{\text{rsp}} + n_{\text{rsp}})}] \tilde{Q}_{\text{aux}})$. The main computational cost of the verifier is to compute the following $(2^{e'_{\text{rsp}} - n_{\text{rsp}}}, 2^{e'_{\text{rsp}} - n_{\text{rsp}}})$ -isogeny :

$$\Phi : E_{\text{aux}} \times E'_{\text{chl}} \rightarrow E'_{\text{com}} \times F'$$

with kernel $\langle (\tilde{P}'_{\text{aux}}, [q_{\text{rsp}}] \phi_{\text{rsp}}^{\text{odd}} \circ \hat{\phi}_{\text{aux}}(\tilde{P}'_{\text{aux}})), (\tilde{Q}'_{\text{aux}}, [2^{e_1 - e'_{\text{rsp}}}] \tilde{Q}'_{\text{chl}}) \rangle$ for some elliptic curve F' , where

$$\begin{aligned} [q_{\text{rsp}}^{-1}] \phi_{\text{rsp}}^{\text{odd}} \circ \hat{\phi}_{\text{aux}}(\tilde{P}'_{\text{aux}}, \tilde{Q}'_{\text{aux}}) &= [q_{\text{rsp}}^{-1} \cdot 2^{e_1 - e'_{\text{rsp}}}] \hat{\phi}_{\text{rsp}}^{\text{even}} \circ \phi_{\text{rsp}}^{\text{even}} \circ \phi_{\text{rsp}}^{\text{odd}} \circ \hat{\phi}_{\text{aux}}(\tilde{P}_{\text{aux}}, \tilde{Q}_{\text{aux}}) \\ &= [q_{\text{rsp}}^{-1} \cdot 2^{e_1 - e'_{\text{rsp}}}] \hat{\phi}_{\text{rsp}}^{\text{even}} \circ \phi_{\text{rsp}} \circ \hat{\phi}_{\text{aux}}(\tilde{P}_{\text{aux}}, \tilde{Q}_{\text{aux}}) \\ &= [q_{\text{rsp}}^{-1} \cdot 2^{e_1 - e'_{\text{rsp}}}] \hat{\phi}_{\text{rsp}}^{\text{even}}([q_{\text{aux}}] \tilde{P}_{\text{chl}}, [q_{\text{aux}}] \tilde{Q}_{\text{chl}}) \mathbb{M}_{\text{rsp}} \\ &= [-2^{e_1 - e'_{\text{rsp}}}] (P'_{\text{chl}}, Q'_{\text{chl}}). \end{aligned}$$

The final step of the verifier is to check that $\text{HASH}(\text{pk} || j(E'_{\text{com}}) || \text{msg})$ equals chl.

5 Parameters

In this section, we describe the parameter requirements and list the parameter sets for instantiations of **SQIsign2d-push**^{1/2} across different security levels.

5.1 Parameter requirements

The selected parameters consist of the security parameter λ , the prime p , and the hash function **HASH**. They are selected according to the following principles:

- (1) The prime p is of the form $p = c \cdot 2^{e_1} \cdot 3^{e_2} - 1$, where $2^{e_1} \approx 3^{e_2}$ and c is a positive integer chosen as small as possible.
- (2) The integer $p - 1$ has a smooth cofactor L satisfying that $L \geq (\log_2(p))^2$.
- (3) Given a classical security parameter λ , the size of the prime p must be at least 2λ bits, because the best known attacks on classical computers have complexity $O(\sqrt{p})$. This parameter also provides approximately $\lambda/2$ -bits post-quantum security level, since the best known attacks on quantum computers have complexity $O(p^{1/4})$.
- (4) The hash function **HASH** is used to generate the challenge value chl of size $\log_2(3^{e_2})$ bits, where $\log_2(3^{e_2})$ is always slightly smaller than λ in our setting. Following the method proposed in **SQIsign Version 2.0.1** [AAA⁺25b], we use the trick of *grinding*: to compensate for the $\lambda - \lceil \log_2(3^{e_2}) \rceil$ missing bits, the hash function is constructed by iterating $\lceil 2^\lambda / 3^{e_2} \rceil$ times a standard cryptographic hash function.

Algorithm 27: SQlsign2d-push^{1/2}.Sign

Input: Secret signing key sk and message msg **Output:** Signature sig

```

1 do
2   // Commitment
3    $(K_0, K_m, E_m, P_m, Q_m, P_{com}, Q_{com}, I_{com}) \leftarrow \text{DoublePath}(1)$ 
4   // Challenge
5    $chl \leftarrow \text{HASH}(pk || j(E_{com}) || msg)$ 
6   // Response
7    $I'_{chl} \leftarrow \text{KerToldeal}(a_0, a_1)$ 
8    $I_{chl} \leftarrow [I_{sk1}]_* I'_{chl}$ 
9    $I \leftarrow \overline{I_{com}} \cdot I_{sk1} \cdot I_{chl} \cdot \tilde{\gamma} / n(I_{sk1})$ 
10   $\text{found}, I_{rsp} \leftarrow \text{RandomEquivalentIdeal}_3(I)$ 
11  while(found=1)
12    Compute a matrix  $\mathbb{M}_{\tilde{\alpha}}$  such that  $\varepsilon(\tilde{\alpha}_{rsp})(P_0, Q_0) = (P_0, Q_0)\mathbb{M}_{\tilde{\alpha}}$ 
13     $d_{rsp} \leftarrow n(I_{rsp})$ 
14     $n_{rsp} \leftarrow \text{adic}_2(d_{rsp})$ 
15     $q_{rsp} \leftarrow d_{rsp} / 2^{n_{rsp}}$ 
16     $q_{aux} \leftarrow 2^{e_{rsp}} - q_{rsp}$ 
17     $v \leftarrow 0$ 
18    if  $q_{aux} \equiv 0 \pmod{3}$  then
19       $v \leftarrow 1$ 
20     $e'_{rsp} \leftarrow e_{rsp} + v$ 
21     $q_{aux} \leftarrow 2^{e'_{rsp}} - q_{rsp}$ 
22     $R \leftarrow U_{chl} + [chl]V_{chl}$ 
23     $E_{chl}, P_{chl}, Q_{chl} \leftarrow \text{ThreelsogenyChain}(E_{pk}, R, \{P_{pk}, Q_{pk}\})$ 
24     $P_{rsp}, Q_{rsp} \leftarrow ([3^{-3e_2}]P_{chl}, [3^{-3e_2}]Q_{chl}) \cdot \mathbb{M}_{\tilde{\alpha}}$ 
25     $E_{aux}, P_{aux}, Q_{aux} \leftarrow \text{GenerateAuxiliarySogeny}(K_0, K_m, P_0, Q_0, E_m, P_m, Q_m, E_{com}, q_{aux}, e'_{rsp})$ 
26     $\tilde{P}_{aux}, \tilde{Q}_{aux} \leftarrow \text{TorsionofBasis}(E_{aux}, 2, e_1)$ 
27     $\tilde{P}_{chl}, \tilde{Q}_{chl} \leftarrow \text{TorsionofBasis}(E_{chl}, 2, e_1)$ 
28     $\mathbb{M}_{aux} \leftarrow \text{ChangeofBasis}(\tilde{P}_{aux}, \tilde{Q}_{aux}, P_{aux}, Q_{aux}, 2, e_1)$ 
29     $\mathbb{M}_{chl} \leftarrow \text{ChangeofBasis}(P_{rsp}, Q_{rsp}, \tilde{P}_{chl}, \tilde{Q}_{chl}, 2, e_1)$ 
30     $\mathbb{M}_{rsp} \leftarrow \mathbb{M}_{chl} \cdot \mathbb{M}_{aux} \pmod{2^{e_1}}$ 
31  return  $E_{aux}, \mathbb{M}_{rsp}, chl, n_{rsp}, v$ ;

```

Algorithm 28: SQIsign2d-push^{1/2}.Verify

Input: the signing $(E_{\text{aux}}, \mathbb{M}_{\text{rsp}}, \text{chl}, n_{\text{rsp}}, v)$ and message msg
Output: verified a boolean indicating whether the verification passed

```

1   $e'_{\text{rsp}} \leftarrow e_{\text{rsp}} + v$ 
2   $n_1 \rightarrow e_1 - e'_{\text{rsp}}$ 
3   $\tilde{U}_{\text{pk}}, \tilde{V}_{\text{pk}} \leftarrow \text{TorsionofBasis}(E_{\text{pk}}, 3, e_2)$ 
4   $R \leftarrow \tilde{U}_{\text{pk}} + [\text{chl}] \tilde{V}_{\text{pk}}$ 
5   $E_{\text{chl}} \leftarrow \text{ThreeIsogenyChain}(E_{\text{pk}}, R, \emptyset)$ 
6   $\tilde{P}_{\text{aux}}, \tilde{Q}_{\text{aux}} \leftarrow \text{TorsionofBasis}(E_{\text{aux}}, 2, e_1)$ 
7   $\tilde{P}_{\text{chl}}, \tilde{Q}_{\text{chl}} \leftarrow \text{TorsionofBasis}(E_{\text{chl}}, 2, e_1)$ 
8   $P_{\text{chl}}, Q_{\text{chl}} \leftarrow (\tilde{P}_{\text{chl}}, \tilde{Q}_{\text{chl}}) \cdot \mathbb{M}_{\text{rsp}}$ 
9   $K_3 \leftarrow E_{\text{chl}}[3] \cap \ker(\hat{\phi}_{\text{chl}})$ 
10 if  $n_{\text{rsp}} \neq 0$  then
11   if  $\mathbb{M}_{\text{rsp}}[0][0] \bmod 2 \neq 1$  or  $\mathbb{M}_{\text{rsp}}[0][1] \bmod 2 \neq 1$  then
12      $R \leftarrow [2^{e_{\text{rsp}} - n_{\text{rsp}}}] P_{\text{chl}}$ 
13   else
14      $R \leftarrow [2^{e_{\text{rsp}} - n_{\text{rsp}}}] Q_{\text{chl}}$ 
15    $E'_{\text{chl}}, \{K'_3, P'_{\text{chl}}, Q'_{\text{chl}}\} \leftarrow \text{TwoIsogenyChainSmall}(E_{\text{chl}}, R, \{K_3, P_{\text{chl}}, Q_{\text{chl}}\}, n_{\text{rsp}})$ 
16 else
17    $E'_{\text{chl}} \leftarrow E_{\text{chl}}, K'_3 \leftarrow K_3, P'_{\text{chl}} \leftarrow P_{\text{chl}}, Q'_{\text{chl}} \leftarrow Q_{\text{chl}}$ 
18    $\tilde{P}'_{\text{aux}} \leftarrow [2^{n_{\text{rsp}} + n_1}] \tilde{P}_{\text{aux}}, \tilde{Q}'_{\text{aux}} \leftarrow [2^{n_{\text{rsp}} + n_1}] \tilde{Q}_{\text{aux}}$ 
19    $\tilde{P}'_{\text{chl}} \leftarrow [-2^{n_1}] \tilde{P}_{\text{chl}}, \tilde{Q}'_{\text{chl}} \leftarrow [-2^{n_1}] \tilde{Q}_{\text{chl}}$ 
20    $E_1, E_2, \{K'_3\} \leftarrow \text{2X2IsogenyChain}((\tilde{P}'_{\text{aux}}, P'_{\text{chl}}), (\tilde{Q}'_{\text{aux}}, Q'_{\text{chl}}), e'_{\text{rsp}} - n_{\text{rsp}}, \{(K'_3, O_{E_{\text{chl}}})\},)$ 
21    $\text{chl}' \leftarrow \text{HASH}(\text{pk} || j(E_1) || \text{msg})$ 
22 return  $(\text{chl} = \text{chl}' \text{ and } K'_3 \neq O_{E_{\text{com}}})$ 

```

5.2 Parameter sets

In this section, we list the parameter sets of **SQIsign2d-push** that achieve at least the 128-, 256-, and 512-bit classical security levels, as well as the 80-, 128-, and 256-bit post-quantum security levels.

Level-1:

$$\begin{aligned}
\lambda &= 128, \\
p &= 2^{131} \cdot 3^{78} - 1, \\
&= 0x62D7F37F9815E5FCA7ECC14EC3FA83C7FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF \\
L &= 5 \cdot 53 \cdot 173 \cdot 863.
\end{aligned}$$

The parameter of Level-1 achieves the 128-bit classical security level.

Level-2:

$$\begin{aligned}
\lambda &= 160, \\
p &= 2^{191} \cdot 3^{118} - 1 \\
&= 0x412508A24FC64C3F96D653C9A6B76DF21D47FD4347F03CC7FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF \\
&\quad \text{FFFFFFFFFFFFFFFFFFFFF}, \\
L &= 5 \cdot 7 \cdot 59 \cdot 937.
\end{aligned}$$

The parameter of Level-2 achieves the 80-bit quantum security level.

Level-3:

$$\begin{aligned}\lambda &= 256, \\ p &= 2^{263} \cdot 3^{156} - 1, \\ &= 0 \times 4C54193130BB665F234A5C454B6E0020B8D8A1C1C552969B4C46EF90ACEC987FFFFFFFFFFFFFFFF \\ &\quad FF, \\ L &= 11 \cdot 17 \cdot 59 \cdot 947.\end{aligned}$$

The parameter of Level-3 achieves both the 256-bit classical and the 128-bit quantum security levels.

Level-4:

$$\begin{aligned}\lambda &= 512, \\ p &= 2^{527} \cdot 3^{324} - 1, \\ &= 0 \times 1711870EF7F80BEEFB63DB18B7F78420F7F5F35DA2851D6F76A630B11240922C3DB0A77209271BE624 \\ &\quad BE3871C05D1E821F9F5307444D71F07E95920C8DF7D80A87FF \\ &\quad FF \\ &\quad FFFFFFFFFFFFFFFFFF, \\ L &= 401 \cdot 691 \cdot 2153.\end{aligned}$$

The parameter of Level-4 achieves both the 512-bit classical and the 256-bit quantum security levels.

6 Security Reductions

We analyze the security of $\text{SQlsign2d-push}^{1/2}$ by following the security framework used for SQlsign2d-push . Note that the signing algorithm may fail to produce a response, that is, the response procedure may output $\perp$ when no suitable response ideal of the required norm is found. Thus, as in SQlsign2d-push the Fiat–Shamir transform used for the signature scheme is a Fiat–Shamir transform with aborts [Lyu09]. At the same time, the zero-knowledge simulation is handled in the hint-assisted weak honest-verifier zero-knowledge framework. Consequently, the relevant proof framework combines Fiat–Shamir with aborts and Fiat–Shamir with hints.

6.1 Definitions for the FSwA and FSwH framework

We recall the definitions needed to combine Fiat–Shamir with aborts (FSwA) and Fiat–Shamir with hints (FSwH). These definitions follow the SQlsign2d-push , with notation adapted to $\text{SQlsign2d-push}^{1/2}$.

Definition 3 (Commitment min-entropy). *The commitment min-entropy of Σ is*

$$\text{MinEnt}(\Sigma) := \max_{(x,w) \in R} \max_{com'} \Pr[com = com' \mid com \leftarrow P_1(x, w)].$$

Definition 4 (Response failure probability). *The maximal response failure probability of Σ is*

$$P_{\Sigma}^{\perp} := \max_{(x,w) \in R} \Pr \left[P_2(x, w, chl) = \perp \mid com \leftarrow P_1(x, w), chl \xleftarrow{\$} \mathcal{C} \right].$$

In our protocol this probability is dominated by the probability that the response-ideal search fails to find a suitable equivalent cyclic ideal of the required norm.

Definition 5 (Hint distribution). *Let $\mathcal{R} \subseteq \mathcal{X} \times \mathcal{W}$ be a relation. A hint distribution $\mathcal{H}$ for $\mathcal{R}$ is a collection of distributions $\mathcal{H} = \{\mathcal{H}_x\}_{x \in \mathcal{X}}$, where $\mathcal{H}_x$ is a distribution over a set HintSet_x of efficiently representable hints associated with x .*

Definition 6 (Hard relation with hints). *Let $\mathcal{H}$ be a hint distribution for $\mathcal{R}$. We say that $(\mathcal{R}, \mathcal{H})$ is a hard relation with hints if, for every PPT algorithm $\mathcal{A}$ and every $q = \text{poly}(\lambda)$,*

$$\text{Adv}_{(\mathcal{R}, \mathcal{H})}^{\text{hint-rel}}(\mathcal{A}, q) := \Pr \left[(x, w^*) \in R \mid \begin{array}{l} (x, w) \leftarrow \text{Gen}_{\mathcal{R}}(1^{\lambda}), \\ h_1, \dots, h_q \leftarrow H_x, \\ w^* \leftarrow \mathcal{A}(x, h_1, \dots, h_q) \end{array} \right].$$

The relation $(\mathcal{R}, \mathcal{H})$ is hard with hints if this advantage is negligible in λ .

Definition 7 ($\mathcal{H}$ -hint-IMP-PA). *Let $\mathcal{H}$ be a hint distribution for $\mathcal{R}$. We say that Σ is secure under $\mathcal{H}$ -hint-assisted passive impersonation attacks, or $\mathcal{H}$ -hint-IMP-PA, if for every $q = \text{poly}(\lambda)$ and every PPT two-stage algorithm $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$,*

$$\text{Adv}_{\Sigma, \mathcal{H}}^{\text{hint-IMP-PA}}(\mathcal{A}, q) := \Pr[\mathcal{H}\text{-hint-IMP-PA}_{\Sigma}(\mathcal{A}, q) = 1]$$

is negligible. The $\mathcal{H}$ -hint-IMP-PA $_{\Sigma}$ game is defined as follows.

Game: $\mathcal{H}\text{-hint-IMP-PA}_{\Sigma}(\mathcal{A}, q)$

1. $(x, w) \leftarrow \text{Gen}_{\mathcal{R}}(1^{\lambda})$.
2. $h_1, \dots, h_q \leftarrow H_x$.
3. $(com^*, st^*) \leftarrow \mathcal{A}_1(x, h_1, \dots, h_q)$.
4. $chl^* \xleftarrow{\$} \mathcal{C}$.
5. $chl^* \leftarrow \mathcal{A}_2(st^*, chl^*)$.
6. **return** $V(x, com^*, chl^*, rsp^*)$.

Definition 8 (IMP-PA $^{\perp}$). *We say that Σ is secure under passive impersonation attacks with aborts, or IMP-PA $^{\perp}$, if for every PPT two stage algorithm $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$,*

$$\text{Adv}_{\Sigma}^{\text{IMP-PA}^{\perp}}(\mathcal{A}) := \Pr[\text{IMP-PA}_{\Sigma}^{\perp}(\mathcal{A}) = 1]$$

is negligible. The IMP-PA $_{\Sigma}^{\perp}$ game is defined as follows.

Game: IMP-PA $_{\Sigma}^{\perp}(\mathcal{A})$

1. $(x, w) \leftarrow \text{Gen}_{\mathcal{R}}(1^{\lambda})$.
2. $(com^*, st^*) \leftarrow \mathcal{A}_1^{\text{OTrans}_{x,w}^{\perp}}(x)$.
3. $chl^* \xleftarrow{\$} \mathcal{C}$.
4. $rsp^* \leftarrow \mathcal{A}_2(st^*, chl^*)$.
5. **return** $V(x, com^*, chl^*, rsp^*)$.

Oracle: $\text{OTrans}_{x,w}^{\perp}$

1. $rsp \leftarrow \perp$.
2. **while** $rsp = \perp$ **do**
 - 2.1. $(com, st) \leftarrow P_1(x, w)$.
 - 2.2. $chl \xleftarrow{\$} \mathcal{C}$.
 - 2.3. $rsp \leftarrow P_2(x, w, st, chl)$.
3. **return** (com, chl, rsp) .

Definition 9 (Hint-assisted wHVZK with aborts). Let $\mathcal{H}$ be a hint distribution for $\mathcal{R}$. We say that Σ is $\mathcal{H}$ -hint-assisted weak honest-verifier zero-knowledge with aborts, or $\mathcal{H}$ -hint-assisted w-HVZK $^\perp$, if there exists a PPT simulator $\mathcal{S}$ such that, for every $q = \text{poly}(\lambda)$ and every PPT algorithms $\mathcal{A}$, the following advantage is negligible:

$$\text{Adv}_{\Sigma, \mathcal{H}, \mathcal{S}}^{\text{hint-wHVZK}^\perp}(\mathcal{A}, q) := \left| \Pr[1 \leftarrow \mathcal{A}(x, \pi) \mid (x, \pi) \leftarrow \text{Real}_\Sigma^\perp(1^\lambda, q)] - \Pr[1 \leftarrow \mathcal{A}(x, \pi) \mid (x, \pi) \leftarrow \text{HintSim}_\Sigma(1^\lambda, q, \mathcal{S})] \right|,$$

where the two distribution $\text{Real}_\Sigma^\perp(1^\lambda, q)$ and $\text{HintSim}_\Sigma(1^\lambda, q, \mathcal{S})$ are defined as below. The real distribution uses only successful transcripts, sampled by the oracle $\text{OTrans}_{x,w}^\perp$; equivalently, it repeatedly samples the commitment and challenge until the response algorithm does not return $\perp$.

Hint-assisted wHVZK$^\perp$	
<u>$\text{Real}_\Sigma^\perp(1^\lambda, q)$:</u>	<u>$\text{HintSim}_\Sigma(1^\lambda, q, \mathcal{S})$:</u>
1. $(x, w) \leftarrow \text{Gen}_\mathcal{R}(1^\lambda)$.	1. $(x, w) \leftarrow \text{Gen}_\mathcal{R}(1^\lambda)$.
2. for $i = 1$ to q do	2. for $i = 1$ to q do
2.1. $\pi_i \leftarrow \text{OTrans}_{x,w}^\perp$.	2.1. $h_i \leftarrow \mathcal{H}_x$.
3. return $(x, \pi = (\pi_1, \dots, \pi_q))$.	2.2. $\pi_i \leftarrow \mathcal{S}(x, h_i)$.
	3. return $(x, \pi = (\pi_1, \dots, \pi_q))$.

Now we have the following two lemmas used in `SQLsign2DPush`.

Lemma 2. Let Σ be a Σ -protocol. For any PPT adversary $\mathcal{A}$ attacking the EUF-CMA security of $\text{SIG}^\perp[\Sigma]$ and any integer $B \geq 1$, there exists a PPT two-stage algorithm $\mathcal{B}$ such that

$$\text{Adv}_{\text{SIG}^\perp[\Sigma]}^{\text{EUF-CMA}}(\mathcal{A}) \leq (1 + q)\text{Adv}_\Sigma^{\text{IMP-PA}^\perp}(\mathcal{B}) + (1 + q + Bs)Bs \cdot \text{MinEnt}(\Sigma) + s(P_\Sigma^\perp)^B,$$

where q and s are upper bounds on the numbers of random-oracle and signing queries made by $\mathcal{A}$, respectively. Moreover, $\mathcal{B}$ makes at most s queries to $\text{OTrans}^\perp$.

Lemma 3. Let $(\mathcal{R}, \mathcal{H})$ be a hard relation with hints, and let Σ be a Σ -protocol for $\mathcal{R}$. Let $\mathcal{S}$ be a $\mathcal{H}$ -assisted simulator for Σ . For any PPT two-stage algorithm $\mathcal{A}$ against $\text{IMP-PA}^\perp$ of Σ , there exist a PPT two-stage algorithm $\mathcal{B}$ and a PPT distinguisher $\mathcal{D}$ such that

$$\text{Adv}_\Sigma^{\text{IMP-PA}^\perp}(\mathcal{A}) \leq \text{Adv}_{\Sigma, \mathcal{H}}^{\text{hint-IMP-PA}}(\mathcal{B}, s) + \text{Adv}_{\Sigma, \mathcal{H}, \mathcal{S}}^{\text{hint-wHVZK}^\perp}(\mathcal{D}, s),$$

where s is an upper bound on the number of queries that $\mathcal{A}$ makes to $\text{OTrans}^\perp$.

Lemma 4. Assume that Σ is special-sound with respect to $\mathcal{R}$. Then, for any PPT two-stage algorithm $\mathcal{A}$ against $\mathcal{H}$ -hint-IMP-PA, there exists an expected polynomial-time algorithm $\mathcal{B}$ such that

$$\text{Adv}_{\Sigma, \mathcal{H}}^{\text{hint-IMP-PA}}(\mathcal{A}, s) \leq \text{Adv}_{(\mathcal{R}, \mathcal{H})}^{\text{hint-rel}}(\mathcal{B}, s) + \frac{1}{\#\mathcal{C}}.$$

Theorem 2. *Let $(\mathcal{R}, \mathcal{H})$ be a relation with hints. Let Σ be a Σ -protocol against $\mathcal{R}$ with challenge space $\mathcal{C}$, where the response algorithm may output $\perp$. Suppose that Σ is special-sound with respect to $\mathcal{R}$, and let $\mathcal{S}$ be an $\mathcal{H}$ -assisted simulator for Σ . Let $\text{SIG}^\perp[\Sigma]$ be the signature scheme obtained from Σ by the Fiat–Shamir transform with aborts. Then for any PPT adversary $\mathcal{A}$ attacking the EUF-CMA security of $\text{SIG}^\perp[\Sigma]$ and any integer $B \geq 1$, there exist algorithms $\mathcal{B}$ and $\mathcal{D}$ such that*

$$\begin{aligned} \text{Adv}_{\text{SIG}^\perp[\Sigma]}^{\text{EUF-CMA}}(\mathcal{A}) \leq (1 + q) & \left(\text{Adv}_{(\mathcal{R}, \mathcal{H})}^{\text{hint-rel}}(\mathcal{B}, s) + \text{Adv}_{\Sigma, \mathcal{H}, \mathcal{S}}^{\text{hint-wHVZK}^\perp}(\mathcal{D}, s) + \frac{1}{\#\mathcal{C}} \right) \\ & + (1 + q + Bs)Bs \cdot \text{MinEnt}(\Sigma) + s(\mathcal{P}_\Sigma^\perp)^B, \end{aligned}$$

where q and s are upper bounds on the numbers of random-oracle and signing queries made by $\mathcal{A}$, respectively.

Proof. Apply Lemma 2 to reduce EUF-CMA security of the Fiat–Shamir-with-aborts signature to IMP-PA[⊥] security. Then apply Lemma 3 to replace the real non-aborting transcript oracle by the hint-assisted simulator. Finally, apply Lemma 4, which uses special soundness to reduce hinted passive impersonation to finding a witness for the relation in the presence of hints, up to the unavoidable challenge-guessing term $1/\#\mathcal{C}$. $\square$

Thus, to prove the signature is EUF-CMA secure it is sufficient to prove the following for a hard relation with hints $(\mathcal{R}, \mathcal{H})$.

1. special soundness with respect to a hard relation with hints;
2. hint-assisted wHVZK[⊥];
3. an exponentially large challenge space;
4. high commitment entropy;
5. a response-failure probability bounded by a constant strictly smaller than one.

6.2 Special Soundness

From now on, we denote our Σ -protocol by $\Sigma_{\text{Push}^{1/2}}$. We begin by establishing that $\Sigma_{\text{Push}^{1/2}}$ satisfies special soundness with respect to the relation defined as

$$\mathcal{R}_{\text{OneEnd}} = \{(E, \alpha) \mid E : \text{supersingular curve over } \mathbb{F}_{p^2}, \alpha \in \text{End}(E) \setminus \mathbb{Z}\}.$$

Theorem 3. $\Sigma_{\text{Push}^{1/2}}$ satisfies the special-soundness with respect to the relation $\mathcal{R}_{\text{OneEnd}}$.

Proof. The proof of the soundness of our protocol is similar to that of **SQIsign2DPush** [NO25, Theorem 2]. For the sake of the completeness of the document, we provide a proof.

Consider two accepting transcripts of $\Sigma_{\text{Push}^{1/2}}$ for the statement E_{pk} , sharing the same commitment E_{com} but with distinct challenges $\text{chl} \neq \text{chl}'$:

$$(E_{\text{com}}, \text{chl}, \text{rsp} = (E_{\text{aux}}, P_{\text{chl}}, Q_{\text{chl}}, n_1)) \text{ and } (E_{\text{com}}, \text{chl}', \text{rsp}' = (E'_{\text{aux}}, P'_{\text{chl}}, Q'_{\text{chl}}, n'_1)).$$

From the challenges, one obtains the isogenies $\varphi_{\text{chl}} : E_{\text{pk}} \rightarrow E_{\text{chl}}$ and $\varphi'_{\text{chl}} : E_{\text{pk}} \rightarrow E'_{\text{chl}}$. Likewise, the responses yield efficient representations of $\varphi_{\text{rsp}} : E_{\text{com}} \rightarrow E_{\text{chl}}$ and $\varphi'_{\text{rsp}} : E_{\text{com}} \rightarrow E'_{\text{chl}}$. Consequently, we obtain an efficiently computable endomorphism

$$\beta = \widehat{\varphi}_{\text{chl}} \circ \varphi_{\text{rsp}} \circ \widehat{\varphi}'_{\text{rsp}} \circ \varphi'_{\text{chl}} \in \text{End}(E_{\text{pk}}).$$

We claim that β is not a scalar endomorphism. Suppose, to the contrary, that $\beta = [\nu]$ for some integer ν . Then $\nu^2 = \deg \beta = d \cdot d' \cdot 3^{2e_2}$, where $d = \deg \varphi_{\text{rsp}}$ and $d' = \deg \varphi'_{\text{rsp}}$.

Write $d = 3^m \tilde{d}$ and $d' = 3^n \tilde{d}'$ with $\tilde{d}, \tilde{d}'$ coprime to 3. Without loss of generality, assume $m \geq n$. Since $\nu^2 = dd' \cdot 3^{2e_2} = \tilde{d}\tilde{d}' \cdot 3^{m+n+2e_2}$, it follows that $\nu = \nu' \cdot 3^{(m+n)/2+e_2}$, where $\nu' = \sqrt{\tilde{d}\tilde{d}'} \in \mathbb{Z}$. Note that ν' is coprime to 3 because both $\tilde{d}$ and $\tilde{d}'$ are. From the above, we derive

$$[\nu'] \hat{\varphi}_{\text{rsp}} \circ \varphi_{\text{chl}} = [3^{(m-n)/2} \tilde{d}] \hat{\varphi}'_{\text{rsp}} \circ \varphi'_{\text{chl}}.$$

Let K'_{chl} generate $\ker \varphi'_{\text{chl}}$. Applying the previous equality to K'_{chl} , we get

$$\hat{\varphi}_{\text{rsp}} \circ \varphi_{\text{chl}}(K'_{\text{chl}}) = [(\nu')^{-1} 3^{(m-n)/2} \tilde{d}] \hat{\varphi}'_{\text{rsp}} \circ \varphi'_{\text{chl}}(\tilde{K}_{\text{chl}}) = O.$$

Since $\ker(\hat{\varphi}_{\text{rsp}})$ contains no 3-torsion point of E_{chl} (i.e., it does not contain $E_{\text{chl}}[3]$), it follows that $\varphi_{\text{chl}}(K'_{\text{chl}}) = O$. Thus $\ker \varphi_{\text{chl}} = \ker \varphi'_{\text{chl}}$, which contradicts the fact that the challenges are distinct. Therefore, β must be non-scalar. $\square$

6.3 Hint-assist wHVZK $^\perp$

We now prove that the underlying Σ -protocol of $\text{SQIsign2d-push}^{1/2}$ is Hint-assist wHVZK $^\perp$.

We first define the following auxiliary distributions. Let $C = \mathbb{Z}/3^{e_2}\mathbb{Z}$ be the challenge space.

The challenge distribution. For a supersingular curve $E/\mathbb{F}_{p^2}$, define $\text{DChl}(E)$ as follows.

1. Sample $\text{chl} \leftarrow C$.
2. Compute the deterministic basis $(\tilde{U}, \tilde{V})$ of $E[3^{e_2}]$.
3. Compute the isogeny $\phi_{\text{chl}} : E \rightarrow E_{\text{chl}}$ with kernel $\langle \tilde{U} + [\text{chl}]\tilde{V} \rangle$.
4. Return $(\text{chl}, \phi_{\text{chl}})$.

The response-isogeny distribution. For two supersingular curves E, E' over $\mathbb{F}_{p^2}$, define $\text{DRsplso}(E, E')$ to be the distribution obtained by sampling a isogeny $\phi : E \rightarrow E'$ according to the same distribution as the response isogeny in an honest non-aborting execution of the response algorithm, conditioned on the pair (E, E') . In particular, if $\deg(\phi) = 2^n q$ with q odd, then $\deg(\phi) < 2^{e_{\text{rsp}}}$, the cyclic component has no non-trivial scalar factor, and $3 \nmid q$. If no such isogeny is sampled, the algorithm returns $\perp$.

The auxiliary-isogeny distribution. Let E be the codomain of a commitment isogeny $\psi : E_0 \rightarrow E$ generated by $\text{DoublePath}(1)$. Given integers $q_{\text{rsp}}, n_{\text{rsp}}$ and $v \in \{0, 1\}$, define

$$e'_{\text{rsp}} = e_{\text{rsp}} - n_{\text{rsp}} + v, \quad q_{\text{aux}} = 2^{e'_{\text{rsp}}} - q_{\text{rsp}}.$$

We choose v so that $q_{\text{aux}} > 0$ and $3 \nmid q_{\text{aux}}$. The distribution $\text{DAuxIsog}(E, \psi, q_{\text{aux}}, q_{\text{rsp}})$ is obtained by running the auxiliary-isogeny generation procedure of $\text{SQIsign2d-push}^{1/2}$, namely the procedure based on the relation

$$\text{nrd}(\alpha) = q_{\text{aux}} q_{\text{rsp}} 2^{e_1 - e'_{\text{rsp}}} L,$$

and on the push-forward of the resulting isogeny through the two halves of the commitment isogeny. It returns an isogeny $\phi_{\text{aux}} : E \rightarrow E_{\text{aux}}$ of degree q_{aux} together with an efficient representation sufficient to evaluate ϕ_{aux} and its dual on 2^{e_1} -torsion points.

The simulated hint distribution. For a public curve E , define $H_{\text{sim}}(E)$ as follows.

1. Sample $(\text{chl}, \phi_{\text{chl}} \leftarrow \text{DChl}(E))$, and let E_{chl} be the codomain of ϕ_{chl} .
2. Run the commitment-generation algorithm **DoublePath**(1) and obtain a commitment isogeny $\psi = \phi_{\text{com}} : E_0 \rightarrow E_{\text{com}}$.
3. Sample $\phi_{\text{rsp}} \leftarrow \text{DRsplSog}(E_{\text{com}}, E_{\text{chl}})$. If $\phi_{\text{rsp}} = \perp$, restart from Step 1.
4. Write $\deg(\phi_{\text{rsp}}) = 2^{n_{\text{rsp}}} q_{\text{rsp}}$ with q_{rsp} odd. Choose $v \in \{0, 1\}$ such that, for $e'_{\text{rsp}} = e_{\text{rsp}} - n_{\text{rsp}} + v$, the integer $q_{\text{aux}} = 2^{e'_{\text{rsp}}} - q_{\text{rsp}}$ is positive and prime to 3.
5. Sample $\phi_{\text{aux}} \leftarrow \text{DAuxIsog}(E_{\text{com}}, \psi, q_{\text{aux}}, q_{\text{rsp}})$.
6. Return $h = (\text{chl}, \phi_{\text{rsp}}, \phi_{\text{aux}}, v)$.

Theorem 4. *The protocol $\Sigma_{\text{Push}^{1/2}}$ is H_{sim} -hint-assisted $w\text{HVZK}^\perp$.*

Proof. We construct a simulator $\mathbf{S}(E_{\text{pk}}, h) \rightarrow (E_{\text{com}}, \text{chl}, \text{rsp})$ as follows. Let $h = (\text{chl}, \phi_{\text{rsp}}, \phi_{\text{aux}}, v) \leftarrow H_{\text{sim}}(E_{\text{pk}})$. Let E_{com} be the domain of both ϕ_{rsp} and ϕ_{aux} , let E_{chl} be the codomain of ϕ_{rsp} , and let E_{aux} be the codomain of ϕ_{aux} .

The simulator computes the degree of ϕ_{rsp} . Since $\deg(\phi_{\text{rsp}}) < 2^{e_{\text{rsp}}} \leq 2^{e_1}$, this degree is determined by the action on the 2^{e_1} -torsion, or equivalently by the 2^{e_1} -Weil pairing. Write $\deg(\phi_{\text{rsp}}) = 2^{n_{\text{rsp}}} q_{\text{rsp}}$, where q_{rsp} is odd. Set $e'_{\text{rsp}} = e_{\text{rsp}} - n_{\text{rsp}} + v$ and $q_{\text{aux}} = 2^{e'_{\text{rsp}}} - q_{\text{rsp}}$. Let $(\tilde{P}_{\text{aux}}, \tilde{Q}_{\text{aux}})$ and $(\tilde{P}_{\text{chl}}, \tilde{Q}_{\text{chl}})$ be the deterministic bases of $E_{\text{aux}}[2^{e_1}]$ and $E_{\text{chl}}[2^{e_1}]$, respectively. The simulator computes the matrix $M_{\text{rsp}} \in M_2(\mathbb{Z}/2^{e_1}\mathbb{Z})$ defined by $\phi_{\text{rsp}} \circ \hat{\phi}_{\text{aux}}(\tilde{P}_{\text{aux}}, \tilde{Q}_{\text{aux}}) = ([q_{\text{aux}}]\tilde{P}_{\text{chl}}, [q_{\text{aux}}]\tilde{Q}_{\text{chl}})M_{\text{rsp}}$. It then outputs $\text{rsp} = (E_{\text{aux}}, M_{\text{rsp}}, n_{\text{rsp}}, v)$ and the transcript $(E_{\text{com}}, \text{chl}, \text{rsp})$.

By the definition of H_{sim} , the curve E_{com} is distributed exactly as an honest commitment curve, the challenge chl is distributed exactly as in an honest execution, the response isogeny ϕ_{rsp} is distributed as the response isogeny in a successful honest response, and the auxiliary isogeny ϕ_{aux} is generated by the same auxiliary-isogeny distribution as in the real protocol. The matrix M_{rsp} is then a deterministic function of $(\phi_{\text{rsp}}, \phi_{\text{aux}}, v)$ and of the deterministic torsion bases. Therefore the simulated transcript distribution is identical to the real non-aborting transcript distribution. Hence, for every PPT distinguisher $\mathcal{A}$, $\text{Adv}_{\Sigma_{\text{Push}^{1/2}}, H_{\text{sim}}, \mathbf{S}}^{\text{hint-wHVZK}^\perp}(\mathcal{A}) = 0$. $\square$

6.4 Resistance to known attacks

Recall that the parameters of **SQIsign2d-push**^{1/2} is restricted by the following conditions:

- The prime $p = c \cdot 2^{e_1} \cdot 3^{e_2} - 1$ with $2^{e_1} \approx 3^{e_2}$ and c is as small as possible.
- The degree of the commitment isogeny is $D_{\text{com}} = 3^{2e_2} \approx 2\lambda$.
- The degree of the challenge isogeny is $D_{\text{chl}} = 3^{e_2} \approx \lambda$.
- The degree of the respond isogeny is bounded by $D_{\text{rsp}} \leq 2^{e_1} \approx \sqrt{p}$.

6.4.1 The hard problem in isogeny-based cryptography The security of isogeny-based cryptosystems relies on the hardness of *Problem 2* (the endomorphism ring problem). In the case of **SQIsign2d-push**^{1/2}, if an adversary can break *Problem 2*, then they can find an isogeny between E_0 and E_{pk} , which is equivalent to the two secret isogenies ϕ_{sk1} and ϕ_{sk2} . Since *Problem 2* and the one-endomorphism problem are equivalent [PW24], an adversary can solve it by computing a single non-scalar endomorphism. Given a random supersingular elliptic curve E over $\mathbb{F}_{p^2}$, the best known algorithm for computing one non-scalar endomorphism of E runs in time $\tilde{O}(p^{1/2})$ on a classical computer [EHL⁺18b].

Using a quantum computer and Grover's algorithm, the time complexity can be reduced to $\tilde{O}(p^{1/4})$. In **SQLsign2d-push**^{1/2}, the prime p is chosen so that $p \approx 2^\lambda$; therefore, the complexity of the best known attack on *Problem 2* is $\tilde{O}(p^{1/2})$ classically and $\tilde{O}(p^{1/4})$ quantumly.

6.4.2 Key recovery In **SQLsign2d-push**^{1/2}, the two secret isogenies $\phi_{\text{sk1}} : E_0 \rightarrow E_{\text{pk}}$ and $\phi_{\text{sk2}} : E_0 \rightarrow E_{\text{pk}}$ have degrees $\deg(\phi_{\text{sk1}}) = 2^{2e_1} \approx 2^{2\lambda}$ and $\deg(\phi_{\text{sk2}}) = 3^{2e_2} \approx 2^{2\lambda}$, respectively. This means that the distribution of E_{pk} is close to uniform over the supersingular curves. Hence, finding an isogeny between E_0 and E_{pk} is equivalent to *Problem 2*. Another way to recover an equivalent secret key is to find the commitment isogeny between E_0 and E_{com} . In detail, given a valid signature, one can compute the commitment curve E_{com} and the signature isogeny between E_{com} and E_{chl} . Using the commitment isogeny, the challenge isogeny, and the signature isogeny, it is easy to construct an isogeny from E_0 to E_{pk} . The commitment isogeny, however, has the same degree as ϕ_{sk2} , so the distribution of E_{com} is also close to uniform. Therefore, computing an isogeny between E_0 and E_{com} is again equivalent to *Problem 2*.

6.4.3 Strong unforgeability **SQLsign2d-push**^{1/2} can not provide strong unforgeability security. Given a valid signature on a message, one can efficiently produce a second distinct valid signature on the same message by manipulating the auxiliary isogeny. Replacing the auxiliary isogeny with any other isogeny of the same degree yields a valid signature. The role of the auxiliary isogeny in the signature is only to enable a two-dimensional representation of the response, but it does not contribute to the security of the protocol. In other words, two-dimensional representations are inherently not unique: given such a representation, in most instances it is easy to find a different representation of the same isogeny. For this reason, **SQLsign2d-push**^{1/2} cannot achieve strong unforgeability.

7 Computing a (2, 2)-isogeny chain between products of elliptic curves

For **SQLsign2d-push**^{1/2}, we specialize to the case of $(2^e, 2^e)$ -isogenies $\Phi : E_1 \times E_2 \rightarrow E_3 \times E_4$ between elliptic curve products $E_1 \times E_2$ and $E_3 \times E_4$ defined over $\mathbb{F}_{p^2}$. Given generators P and Q of $\ker(\Phi)$, we demonstrate the decomposition of Φ into a chain of successive $(2, 2)$ -isogenies:

$$E_1 \times E_2 \xrightarrow{\Phi_1} A_1 \xrightarrow{\Phi_2} A_2 \rightarrow \cdots \rightarrow A_{e-2} \xrightarrow{\Phi_{e-1}} A_{e-1} \xrightarrow{\Phi_e} E_3 \times E_4.$$

The evaluation of each intermediate isogeny in this chain fundamentally reduces to determining the theta null point of its respective codomain. For computational purposes, points on a principally polarized abelian surface (PPAS) are represented via level-2 theta coordinates [DMPR25], serving as a higher-dimensional analogue to the x -coordinate representation on Montgomery curves. A naïve method for computing a chain of $(2, 2)$ -isogenies proceeds as follows (see [AAA⁺25b, section 8.5]):

- Compute the gluing isogeny $\Phi_1 : E_1 \times E_2 \rightarrow A_1$ using the 8-torsion points $[2^{e-3}]P$ and $[2^{e-3}]Q$ lying above $\ker(\Phi_1) = \langle [2^{e-1}]P, [2^{e-1}]Q \rangle$ using **GluingCodomain**, and compute $\Phi_1(P), \Phi_1(Q)$ using **GluingEval**.

- For all $i \in \{2, \dots, e-2\}$, compute the generic isogeny $\Phi_i : A_{i-1} \rightarrow A_i$ using the 8-torsion points $[2^{e-i-2}](\Phi_{i-1} \circ \dots \circ \Phi_1(P))$ and $[2^{e-i-2}](\Phi_{i-1} \circ \dots \circ \Phi_1(Q))$ lying above $\ker(\Phi_i)$ (see **GenericCodomainWith8Torsion**).
- Compute the generic isogeny $\Phi_{e-1} : A_{e-2} \rightarrow A_{e-1}$ given the 4-torsion points $\Phi_{e-2} \circ \dots \circ \Phi_1(P)$ and $\Phi_{e-2} \circ \dots \circ \Phi_1(Q)$ lying above $\ker(\Phi_{e-1})$ (see **GenericCodomainWith4Torsion**).
- Compute the generic isogeny $\Phi_e : A_{e-1} \rightarrow E_3 \times E_4$ given the 2-torsion kernel points $\Phi_{e-1} \circ \dots \circ \Phi_1(P)$ and $\Phi_{e-1} \circ \dots \circ \Phi_1(Q)$ (see **GenericCodomain**).
- Compute the coordinate transformation from the theta structure on $E_3 \times E_4$ (naturally induced by Φ_e) to a product theta structure. This allows the image points to be explicitly expressed in $(X : Z)$ -Montgomery coordinates on the individual components E_3 and E_4 (see **SplittingIsomorphism**).

We highlight the following algorithms that are depicted:

- **ThetaDBL**: on input of a point P (in theta coordinates) and the constants **consts**, outputs $[2]P$ (in theta coordinates).
- **GenericCodomainWith8Torsion**: given theta coordinates of 8-torsion points T_1'', T_2'' on domain surface A , this algorithm outputs the dual theta null point $(\alpha : \beta : \gamma : \delta)$, its inverse $(\alpha^{-1} : \beta^{-1} : \gamma^{-1} : \delta^{-1})$, and the theta null point of the image B of the isogeny $\Phi : A \rightarrow B$ with $\ker(\Phi) = \langle [4]T_1'' \rangle \oplus \langle [4]T_2'' \rangle$.
- **GenericCodomainWith4Torsion**: given theta coordinates of a 4-torsion point T_1' on domain surface A satisfying $[2]T_1' \in \ker(\Phi)$, and the theta null point of A , this algorithm outputs the dual theta null point $(\alpha : \beta : \gamma : \delta)$, its inverse $(\alpha^{-1} : \beta^{-1} : \gamma^{-1} : \delta^{-1})$, and the theta null point of the codomain B of $\Phi : A \rightarrow B$.
- **GenericCodomain**: given the theta null point O_A of domain A , computes the dual theta null point $(\alpha : \beta : \gamma : \delta)$, its inverse $(\alpha^{-1} : \beta^{-1} : \gamma^{-1} : \delta^{-1})$, and the theta null point O_B of the codomain B of $(2, 2)$ -isogeny $\Phi : A \rightarrow B$.
- **GenericEval**: given a point P (in theta coordinates) on domain surface A and the point $(\alpha^{-1} : \beta^{-1} : \gamma^{-1} : \delta^{-1})$, outputs the theta coordinates of $\Phi(P)$.
- **GluingCodomain**: given theta coordinates of 8-torsion points T_1'', T_2'' on domain product surface, this algorithm outputs the dual theta null point $(\alpha : \beta : \gamma : 0)$, its "inverse" $(\alpha^{-1} : \beta^{-1} : \gamma^{-1} : 0)$, the theta null point of the image surface A of the isogeny $\Phi : E_1 \times E_2 \rightarrow A$ with $\ker(\Phi) = \langle [4]T_1'' \rangle \oplus \langle [4]T_2'' \rangle$, J the dual of the theta point $\Phi(T_1'')$ and a change-of-basis matrix **N** (re-used for evaluation).
- **GluingEval**: given a point $P \in E_1 \times E_2$, T_1'' a 8-torsion point such that $[4]T_1'' \in \ker(\Phi)$, J the dual of the theta point $\Phi(T_1'')$ on A , and the change-of-basis matrix **N** computed during **GluingCodomain**, outputs the theta coordinates of $\Phi(P)$.
- **GluingEvalSpecial**: given a point $P \in E_1 \times E_2$ of the form $(P_1, 0)$ or $(0, P_2)$, $(\alpha^{-1} : \beta^{-1} : \gamma^{-1} : 0)$ the "inverse" of the dual theta null point over A , and the change-of-basis matrix **N** computed during **GluingCodomain**, outputs the theta coordinates of $\Phi(P)$.
- **SplittingIsomorphism**: given theta null point O_A on surface $A \cong E_1 \times E_2$, computes the isomorphism whose action on O_A gives the theta null point associated with the product theta structure.

We consider a principally polarized abelian surface (PPAS) A endowed with a system of level-2 theta coordinates induced by the theta null point $(a : b : c : d)$. We introduce the Hadamard operator and the squaring operator acting on theta coordinates as follows:

$$\begin{aligned} \mathcal{H}(x, y, z, w) &:= (x + y + z + w, x - y + z - w, x + y - z - w, x - y - z + w), \\ \mathcal{S}(x, y, z, w) &:= (x^2, y^2, z^2, w^2). \end{aligned}$$

These operations cost 8 additions and 4 squarings, respectively (using recursive formulae for the Hadamard operator). The Hadamard operator maps theta coordinates to the *dual* theta coordinates. This dual system is determined by the dual theta null point $(a' : b' : c' : d')$, where $(a', b', c', d') := \mathcal{H}(a, b, c, d)$. A level-2 theta null point encodes a canonical 2-isogeny $A \rightarrow B$. We call the dual of the theta null point of B the *dual isogenous* theta null point and denote its coordinates by $(\alpha : \beta : \gamma : \delta)$. We have the relation

$$(\alpha^2 : \beta^2 : \gamma^2 : \delta^2) = \mathcal{H}(a^2 : b^2 : c^2 : d^2).$$

Given a point $P = (x : y : z : w)$ expressed in theta coordinates, **Alg. 29** (called **ThetaDBL**) describes how to compute $[2]P$, which requires certain auxiliary constants as input; these constants depend only on the theta null point and are precomputed by **Alg. 30**.

Algorithm 29: ThetaDBL(P, consts)

Input: The theta coordinates of P on A with theta null point $O_A := (a : b : c : d)$, and the auxiliary constants $\text{consts} := \text{ThetaPrecomp}(O_A)$.

Output: The theta coordinates of the point $[2]P$.

```

1  $x_P, y_P, z_P, w_P \leftarrow P$ ;
2  $c_1, c_2, c_3, c_4, c_5, c_6, c_7, c_8 \leftarrow \text{consts}$ ;
3  $X_{2P}, Y_{2P}, Z_{2P}, W_{2P} \leftarrow \mathcal{S} \circ \mathcal{H} \circ \mathcal{S}(x_P, y_P, z_P, w_P)$ ;
4  $X_{2P} \leftarrow X_{2P} \cdot c_8$ ;
5  $Y_{2P} \leftarrow Y_{2P} \cdot c_7$ ;
6  $Z_{2P} \leftarrow Z_{2P} \cdot c_6$ ;
7  $W_{2P} \leftarrow W_{2P} \cdot c_5$ ;
8  $X_{2P}, Y_{2P}, Z_{2P}, W_{2P} \leftarrow \mathcal{H}(X_{2P}, Y_{2P}, Z_{2P}, W_{2P})$ ;
9  $X_{2P} \leftarrow X_{2P} \cdot c_4$ ;
10  $Y_{2P} \leftarrow Y_{2P} \cdot c_3$ ;
11  $Z_{2P} \leftarrow Z_{2P} \cdot c_2$ ;
12  $W_{2P} \leftarrow W_{2P} \cdot c_1$ ;
13 return  $(X_{2P} : Y_{2P} : Z_{2P} : W_{2P})$ ; // Total cost: 8S + 8M + 16a
```

Algorithm 30: ThetaPrecomp(O_A)

Input: Theta null point $O_A := (a : b : c : d)$.

Output: Auxiliary constants consts used for arithmetic.

```

1  $(A, B, C, D) \leftarrow \mathcal{H} \circ \mathcal{S}(a, b, c, d)$ ;
2  $t_1 \leftarrow A \cdot B$ ;
3  $t_2 \leftarrow C \cdot D$ ;
4  $ABC \leftarrow t_1 \cdot C$ ;
5  $AB\delta^2 \leftarrow t_1 \cdot D$ ;
6  $ACD \leftarrow t_2 \cdot A$ ;
7  $BCD \leftarrow t_2 \cdot B$ ;
8  $t_1 \leftarrow a \cdot b$ ;
9  $t_2 \leftarrow c \cdot d$ ;
10  $abc \leftarrow t_1 \cdot c$ ;
11  $abd \leftarrow t_1 \cdot d$ ;
12  $acd \leftarrow t_2 \cdot a$ ;
13  $bcd \leftarrow t_2 \cdot b$ ;
14  $\text{consts} \leftarrow \{abc, abd, acd, bcd, ABC, ABD, ACD, BCD\}$ ;
15 return  $\text{consts}$ ; // Total cost: 4S + 12M(+8a)
```

7.1 Generic (2, 2)-isogeny computation

We first describe the computation of a generic (2, 2)-isogeny $\Phi: A \rightarrow B$ given torsion points that lie above $\ker(\Phi)$. For some $k > 1$, two points $T_1, T_2 \in A[2^k]$ are said to lie

above the kernel of a $(2, 2)$ -isogeny Φ if $\ker(\Phi) = \langle [2^{k-1}]T_1 \rangle \oplus \langle [2^{k-1}]T_2 \rangle$. Such points may be assumed to be accessible. For the formulas that follow to be valid, we require that $\ker(\Phi)$ satisfies a compatibility condition with the system of theta coordinates on A . When working with a chain of $(2, 2)$ -isogenies, it suffices to ensure that this compatibility condition holds at the beginning of the chain; it will then propagate throughout the chain automatically.

7.1.1 With isotropic 8-torsion lying above the kernel Assume we know the theta coordinates of two points $T_1'', T_2'' \in A[8]$ of order 8 that are isotropic, i.e., $e_8(T_1'', T_2'') = 1$, and such that $\ker(\Phi) = \langle [4]T_1'' \rangle \oplus \langle [4]T_2'' \rangle$. Using the algorithm **GenericCodomainWith8Torsion** (see **Alg. 31**), we can compute the theta null point $(a_2 : b_2 : c_2 : d_2)$ of the codomain B , along with its dual $(\alpha : \beta : \gamma : \delta)$ and the projective inverse $(\alpha^{-1} : \beta^{-1} : \gamma^{-1} : \delta^{-1})$. Note that in the generic case we expect $\alpha\beta\gamma\delta \neq 0$; indeed, otherwise A would correspond to a product of elliptic curves. Accordingly, we assume this non-vanishing in **GenericCodomainWith8Torsion**.

Algorithm 31: **GenericCodomainWith8Torsion**(T_1'', T_2'')

Input: Theta coordinates of T_1'' and T_2'' such that $\ker(\Phi) = \langle [4]T_1'' \rangle \oplus \langle [4]T_2'' \rangle$.
Output: Dual isogenous theta null point $(\alpha : \beta : \gamma : \delta)$, the projective inverse $(\alpha^{-1} : \beta^{-1} : \gamma^{-1} : \delta^{-1})$, and the theta null point O_B on B ; // Case $\alpha\beta\gamma\delta \neq 0$

```

1   $x_{T_1''}, y_{T_1''}, z_{T_1''}, w_{T_1''} \leftarrow T_1''$ ;
2   $x_{T_2''}, y_{T_2''}, z_{T_2''}, w_{T_2''} \leftarrow T_2''$ ;
3   $(x\alpha, x\beta, y\gamma, y\delta) \leftarrow \mathcal{H} \circ \mathcal{S}(x_{T_1''}, y_{T_1''}, z_{T_1''}, w_{T_1'')$ ;
4   $(z\alpha, w\beta, z\gamma, w\delta) \leftarrow \mathcal{H} \circ \mathcal{S}(x_{T_2''}, y_{T_2''}, z_{T_2''}, w_{T_2'')$ ;
5  if  $0 \in \{x\alpha, x\beta, z\alpha, w\beta, z\gamma, w\delta\}$  then
    | // Check only done in verification
6  |   raise Exception: "GenericCodomainWith8Torsion failed: unexpected splitting";
7  end
8   $x\alpha w\beta \leftarrow x\alpha \cdot w\beta$ ;
9   $z\alpha x\beta \leftarrow z\alpha \cdot x\beta$ ;
10  $\alpha \leftarrow z\alpha \cdot x\alpha w\beta$ ;
11  $\beta \leftarrow w\beta \cdot z\alpha x\beta$ ;
12  $\gamma \leftarrow z\gamma \cdot x\alpha w\beta$ ;
13  $\delta \leftarrow w\delta \cdot z\alpha x\beta$ ;
14  $z\gamma w\delta \leftarrow z\gamma \cdot w\delta$ ;
15  $\alpha^{-1} \leftarrow x\beta \cdot z\gamma w\delta$ ;
16  $\beta^{-1} \leftarrow x\alpha \cdot z\gamma w\delta$ ;
17  $\gamma^{-1} \leftarrow \delta$ ;
18  $\delta^{-1} \leftarrow \gamma$ ;
19  $P \leftarrow (x\alpha : x\beta : y\gamma : y\delta)$ ;
20  $Q \leftarrow (z\alpha : w\beta : z\gamma : w\delta)$ ;
21  $I \leftarrow (\alpha^{-1} : \beta^{-1} : \gamma^{-1} : \delta^{-1})$ ;
22 if not CHECKISOTROPIC( $P, Q, I$ ) then
    | // Check only done in verification
23 |   raise Exception: "GenericCodomainWith8Torsion failed:  $P, Q$  not isotropic";
24 end
25  $(a_2, b_2, c_2, d_2) \leftarrow \mathcal{H}(\alpha, \beta, \gamma, \delta)$ ;
26  $0_B \leftarrow (a_2 : b_2 : c_2 : d_2)$ ;
27 return  $(\alpha : \beta : \gamma : \delta), (\alpha^{-1} : \beta^{-1} : \gamma^{-1} : \delta^{-1}), 0_B$ ; // Total cost (without checks): 8S + 9M + 24a

```

7.1.2 With isotropic 4-torsion lying above the kernel When we cannot access 8-torsion points lying above $\ker(\Phi)$ but only 4-torsion points T_1', T_2' such that $\ker(\Phi) = \langle [2]T_1' \rangle \oplus \langle [2]T_2' \rangle$, we can still compute Φ using the algorithm **GenericCodomainWith4Torsion** (see **Alg. 32**). It requires only T_1' and the theta null point of A . However, this method

is considerably more expensive than `GenericCodomainWith8Torsion`, as it involves the extraction of two square roots.

Algorithm 32: `GenericCodomainWith4Torsion`(T'_1, O_A)

Input: Theta coordinates of T'_1 of order 4 such that $[2]T'_1 \in \ker(\Phi)$, and the theta null point $O_A := (a : b : c : d)$ of A .

Output: Dual isogenous theta null point $(\alpha : \beta : \gamma : \delta)$, the projective inverse $(\alpha^{-1} : \beta^{-1} : \gamma^{-1} : \delta^{-1})$, and the theta null point O_B on B . ; // Case $\alpha\beta\gamma\delta \neq 0$

- 1 $(x\alpha\beta, _, x\gamma\delta, _) \leftarrow \mathcal{H} \circ \mathcal{S}(x_{T'_1}, y_{T'_1}, z_{T'_1}, w_{T'_1})$; // the underscores denote discarded coordinates
- 2 $(\alpha^2, \beta^2, \gamma^2, \delta^2) \leftarrow \mathcal{H} \circ \mathcal{S}(a, b, c, d)$;
- 3 $\alpha\beta \leftarrow \text{SQUAREROOT}(\alpha^2 \cdot \beta^2)$;
- 4 $\alpha\gamma \leftarrow \text{SQUAREROOT}(\alpha^2 \cdot \gamma^2)$;
- 5 $\beta \leftarrow \alpha\beta \cdot \alpha\gamma$;
- 6 $\delta^{-1} \leftarrow \beta \cdot x\gamma\delta$;
- 7 $\beta \leftarrow \beta \cdot x\alpha\beta$;
- 8 $\delta \leftarrow x\gamma\delta \cdot \alpha\beta \cdot \alpha^2$;
- 9 $\alpha \leftarrow x\alpha\beta \cdot \alpha^2$;
- 10 $\gamma \leftarrow \alpha \cdot \gamma^2$;
- 11 $\alpha \leftarrow \alpha \cdot \alpha\gamma$;
- 12 $\alpha^{-1} \leftarrow x\alpha\beta \cdot \delta^2$;
- 13 $\gamma^{-1} \leftarrow \alpha^{-1} \cdot \beta^2$;
- 14 $\alpha^{-1} \leftarrow \alpha^{-1} \cdot \gamma^2$;
- 15 $\beta^{-1} \leftarrow \alpha^{-1} \cdot \alpha\beta$;
- 16 $\alpha^{-1} \leftarrow \alpha^{-1} \cdot \beta^2$;
- 17 $\gamma^{-1} \leftarrow \gamma^{-1} \cdot \alpha\gamma$;
- 18 $\delta^{-1} \leftarrow \delta^{-1} \cdot \beta^2$;
- 19 $(a_2, b_2, c_2, d_2) \leftarrow \mathcal{H}(\alpha, \beta, \gamma, \delta)$;
- 20 $O_B \leftarrow (a_2 : b_2 : c_2 : d_2)$;
- 21 **return** $(\alpha : \beta : \gamma : \delta), (\alpha^{-1} : \beta^{-1} : \gamma^{-1} : \delta^{-1}), O_B$; // Total cost: 8S + 17M + 24a + 2Sqrt

7.1.3 With kernel generators only When we only know generators T_1, T_2 of $\ker(\Phi)$, we can still compute Φ using `GenericCodomain`(see **Alg. 33**), given only the theta null point of A , at the expense of three square root extractions. Interestingly, due to the compatibility condition between $\ker(\Phi)$ and our system of theta coordinates, the points T_1 and T_2 are not actually needed for the computation.

7.1.4 Generic (2, 2)-isogeny evaluation Assume we have computed the codomain of a $(2, 2)$ -isogeny $\Phi: A \rightarrow B$ via one of the algorithms `GenericCodomainWith8Torsion`, `GenericCodomainWith4Torsion`, or `GenericCodomain`. That is, we possess the theta null point O_B of B , its dual $(\alpha : \beta : \gamma : \delta)$, and the dual projective inverse $(\alpha^{-1} : \beta^{-1} : \gamma^{-1} : \delta^{-1})$. We are then in a position to evaluate Φ at a point $P \in A$ using only the theta coordinates of P together with $(\alpha^{-1} : \beta^{-1} : \gamma^{-1} : \delta^{-1})$ via `GenericEval` (see **Alg. 34**).

7.2 Gluing (2, 2)-isogeny computation

We now turn to the computation of a gluing $(2, 2)$ -isogeny $\Phi: E_1 \times E_2 \rightarrow A$, where E_1, E_2 are elliptic curves and A is a principally polarized abelian surface (PPAS). The first step is to apply a change-of-basis matrix in order to express points $P = (P_1, P_2)$ on $E_1 \times E_2$ where each P_i is given in theta coordinates via `MontgomeryToTheta` (see **Alg. 35**) in the (non-product) theta coordinates corresponding to our chosen theta structure. This transformation is provided by `ProductToTheta` (see **Alg. 36**).

Algorithm 33: GenericCodomain(O_A)

Input: The theta constants $O_A := (a : b : c : d)$ of A .**Output:** Dual isogenous theta null point $(\alpha : \beta : \gamma : \delta)$, the projective inverse $(\alpha^{-1} : \beta^{-1} : \gamma^{-1} : \delta^{-1})$, and the theta null point O_B on B . ; // Case $\alpha\beta\gamma\delta \neq 0$

```

1   $(\alpha^2, \beta^2, \gamma^2, \delta^2) \leftarrow \mathcal{H} \circ \mathcal{S}(a, b, c, d);$ 
2   $\alpha \leftarrow \alpha^2;$ 
3   $\beta \leftarrow \alpha^2 \cdot \beta^2;$ 
4   $\gamma \leftarrow \alpha^2 \cdot \gamma^2;$ 
5   $\delta \leftarrow \alpha^2 \cdot \delta^2;$ 
6   $\beta \leftarrow \text{SquareRoot}(\beta);$  // compute  $\beta = \sqrt{\alpha^2 \beta^2}$ 
7   $\gamma \leftarrow \text{SquareRoot}(\gamma);$  // compute  $\gamma = \sqrt{\alpha^2 \gamma^2}$ 
8   $\delta \leftarrow \text{SquareRoot}(\delta);$  // compute  $\delta = \sqrt{\alpha^2 \delta^2}$ 
9   $\alpha^{-1} \leftarrow \gamma^2 \cdot \delta^2;$ 
10  $\beta^{-1} \leftarrow \alpha^{-1} \cdot \beta;$ 
11  $\alpha^{-1} \leftarrow \alpha^{-1} \cdot \beta^2;$ 
12  $\gamma^{-1} \leftarrow \delta^2 \cdot \beta^2 \cdot \gamma;$ 
13  $\delta^{-1} \leftarrow \gamma^2 \cdot \beta^2 \cdot \delta;$ 
14  $(a_2, b_2, c_2, d_2) \leftarrow \mathcal{H}(\alpha, \beta, \gamma, \delta);$ 
15  $0_B \leftarrow (a_2 : b_2 : c_2 : d_2);$ 
16 return  $(\alpha : \beta : \gamma : \delta), (\alpha^{-1} : \beta^{-1} : \gamma^{-1} : \delta^{-1}), 0_B;$  // Total cost: 4S + 10M + 16a + 3Sqrt

```

Algorithm 34: GenericEval(P, I)

Input: Theta coordinates of P , and the inverse of the dual isogenous theta null point $I := (\alpha^{-1} : \beta^{-1} : \gamma^{-1} : \delta^{-1})$ on B .**Output:** Theta coordinates of $P' := \Phi(P)$. ; // Case $\beta\gamma\delta \neq 0$

```

1   $x_P, y_P, z_P, w_P \leftarrow P;$ 
2   $\alpha^{-1}, \beta^{-1}, \gamma^{-1}, \delta^{-1} \leftarrow I;$ 
3   $(X_P, Y_P, Z_P, W_P) \leftarrow \mathcal{H} \circ \mathcal{S}(x_P, y_P, z_P, w_P);$ 
4   $X' \leftarrow \alpha^{-1} \cdot X_P;$ 
5   $Y' \leftarrow \beta^{-1} \cdot Y_P;$ 
6   $Z' \leftarrow \gamma^{-1} \cdot Z_P;$ 
7   $W' \leftarrow \delta^{-1} \cdot W_P;$ 
8   $(x_{P'}, y_{P'}, z_{P'}, w_{P'}) \leftarrow \mathcal{H}(X', Y', Z', W');$ 
9   $P' \leftarrow (x_{P'} : y_{P'} : z_{P'} : w_{P'});$ 
10 return  $P';$  // Total cost: 4S + 4M + 16a

```

Algorithm 35: MontgomeryToTheta(P, O)

Input: A point $P := (X : Z)$ in Montgomery coordinates, and the theta null point $O := (a : b)$.**Output:** Point P in theta coordinates $(\theta_0 : \theta_1)$.

```

1   $\theta_0 \leftarrow X - Z;$ 
2   $\theta_0 \leftarrow a \cdot \theta_0;$ 
3   $\theta_1 \leftarrow X + Z;$ 
4   $\theta_1 \leftarrow b \cdot \theta_1;$ 
5  return  $(\theta_0 : \theta_1);$  // Total cost: 2M + 2a

```

Algorithm 36: ProductToTheta(pts, N)

Input: A list of points pts, where each $P \in \text{pts}$ is given as $P := (P_1, P_2) \in E_1 \times E_2$, and N is the change-of-basis matrix computed by ThetaChangeOfBasis.

Output: The corresponding point P' in theta coordinates on $A \cong E_1 \times E_2$.

```

1 eval_pts  $\leftarrow []$ ;
2  $L \leftarrow \#\text{pts}$ ;
3 for  $j \leftarrow 1$  to  $L$  do
4    $P = (P_1, P_2) \leftarrow \text{pts}[j]$ ;
5   Write  $P_i = (\theta_{0,i} : \theta_{1,i})$  for  $i = 1, 2$ ;
6    $x \leftarrow \theta_{0,1} \cdot \theta_{0,2}$ ;
7    $y \leftarrow \theta_{0,1} \cdot \theta_{1,2}$ ;
8    $z \leftarrow \theta_{1,1} \cdot \theta_{0,2}$ ;
9    $w \leftarrow \theta_{1,1} \cdot \theta_{1,2}$ ;
10   $P' \leftarrow \mathbf{N} \cdot (x : y : z : w)$  ; // Cost: at most 20M + 12a
11  Append  $P'$  to eval_pts;
12 return eval_pts ; // Total cost: 20kM + 12ka with  $k = \#\text{pts}$ 
```

Algorithm 37: ActionByTranslation(P, Q)

Input: Four-torsion points $P = (P_1, P_2)$ and $Q = (Q_1, Q_2)$ on $E_1 \times E_2$.

Output: Array mats containing 2×2 matrices giving the action of translation by $P_1, Q_1 \in E_1$ and $P_2, Q_2 \in E_2$.

```

1  $P' := (P'_1, P'_2) \leftarrow [2]P$  ; // Cost: 4S + 8M + 8a using xDBL
2  $Q' := (Q'_1, Q'_2) \leftarrow [2]Q$  ; // Cost: 4S + 8M + 8a using xDBL
3 Write  $P_i = (X_i^{(P)} : Z_i^{(P)})$  and  $Q_i = (X_i^{(Q)} : Z_i^{(Q)})$  for  $i = 1, 2$ ;
4 Write  $P'_i = (U_i^{(P)} : W_i^{(P)})$  and  $Q'_i = (U_i^{(Q)} : W_i^{(Q)})$  for  $i = 1, 2$ ;
5 for  $i \leftarrow 1$  to 2 do
6    $WX_i^{(P)}, UZ_i^{(P)} \leftarrow W_i^{(P)} \cdot X_i^{(P)}, U_i^{(P)} \cdot Z_i^{(P)}$ ;
7    $\delta_i^{(P)} \leftarrow WX_i^{(P)} - UZ_i^{(P)}$ ;
8    $WX_i^{(Q)}, UZ_i^{(Q)} \leftarrow W_i^{(Q)} \cdot X_i^{(Q)}, U_i^{(Q)} \cdot Z_i^{(Q)}$ ;
9    $\delta_i^{(Q)} \leftarrow WX_i^{(Q)} - UZ_i^{(Q)}$ ;
10 Compute the inverses of  $\delta_i^{(P)}, \delta_i^{(Q)}, Z_i^{(P)}, Z_i^{(Q)}$  using batched inversions ; // Cost: 21M + 1I
11 Initialize array mats to store matrices;
12 pts  $\leftarrow [P, Q]$ ;
13 for  $i \leftarrow 1$  to 2 do
14   for  $j \leftarrow 1$  to 2 do
15      $R \leftarrow \text{pts}[j]$ ;
16      $M_{0,0} \leftarrow -U_i^{(R)} Z_i^{(R)} \cdot (\delta_i^{(R)})^{-1}$ ;
17      $M_{0,1} \leftarrow -W_i^{(R)} Z_i^{(R)} \cdot (\delta_i^{(R)})^{-1}$ ;
18      $M_{1,0} \leftarrow U_i^{(R)} X_i^{(R)} \cdot (\delta_i^{(R)})^{-1} - X_i^{(R)} \cdot (Z_i^{(R)})^{-1}$ ;
19      $M_{1,1} \leftarrow -M_{0,0}$ ;
20      $\mathbf{M} \leftarrow (M_{k,\ell})_{0 \leq k, \ell \leq 1}$ ;
21     Append  $\mathbf{M}$  to mats;
22 return mats ; // Total cost: 8S + 69M + 1I + 24a
```

Algorithm 38: ThetaChangeOfBasis(P, Q)

Input: The points $P = (P_1, P_2)$ and $Q = (Q_1, Q_2)$ in the 4-torsion of $E_1 \times E_2$ such that the kernel for the gluing isogeny is $\langle [2]P, [2]Q \rangle$.

Output: The 4×4 change-of-basis matrix $\mathbf{N}$.

```

1 if  $P_1, P_2, Q_1, Q_2$  do not have order 4 or  $[2]P_1 = [2]P_2$  or  $[2]Q_1 = [2]Q_2$  then
2   Error: “ThetaChangeOfBasis failed: gluing kernel is diagonal or not
   isotropic.”;
3  $\mathbf{G}, \mathbf{G}', \mathbf{H}, \mathbf{H}' \leftarrow \text{ActionByTranslation}(P, Q)$  ;           // Cost: 4S + 61M + 1I + 16a
4  $t_1 \leftarrow \mathbf{G}_{0,0} \cdot \mathbf{H}_{0,0} + \mathbf{G}_{0,1} \cdot \mathbf{H}_{1,0}$ ;
5  $t_2 \leftarrow \mathbf{G}_{1,0} \cdot \mathbf{H}_{0,0} + \mathbf{G}_{1,1} \cdot \mathbf{H}_{1,0}$ ;
6  $t_3 \leftarrow \mathbf{G}'_{0,0} \cdot \mathbf{H}'_{0,0} + \mathbf{G}'_{0,1} \cdot \mathbf{H}'_{1,0}$ ;
7  $t_4 \leftarrow \mathbf{G}'_{1,0} \cdot \mathbf{H}'_{0,0} + \mathbf{G}'_{1,1} \cdot \mathbf{H}'_{1,0}$ ;
8  $N_{0,0} \leftarrow \mathbf{G}_{0,0} \cdot \mathbf{G}'_{0,0} + \mathbf{H}_{0,0} \cdot \mathbf{H}'_{0,0} + t_1 \cdot t_3 + 1$ ;
9  $N_{0,1} \leftarrow \mathbf{G}_{0,0} \cdot \mathbf{G}'_{1,0} + \mathbf{H}_{0,0} \cdot \mathbf{H}'_{1,0} + t_1 \cdot t_4$ ;
10  $N_{0,2} \leftarrow \mathbf{G}_{1,0} \cdot \mathbf{G}'_{0,0} + \mathbf{H}_{1,0} \cdot \mathbf{H}'_{0,0} + t_2 \cdot t_3$ ;
11  $N_{0,3} \leftarrow \mathbf{G}_{1,0} \cdot \mathbf{G}'_{1,0} + \mathbf{H}_{1,0} \cdot \mathbf{H}'_{1,0} + t_2 \cdot t_4$ ;
12  $N_{1,0} \leftarrow \mathbf{H}'_{0,0} \cdot N_{0,0} + \mathbf{H}'_{0,1} \cdot N_{0,1}$ ;
13  $N_{1,1} \leftarrow \mathbf{H}'_{1,0} \cdot N_{0,0} + \mathbf{H}'_{1,1} \cdot N_{0,1}$ ;
14  $N_{1,2} \leftarrow \mathbf{H}'_{0,0} \cdot N_{0,2} + \mathbf{H}'_{0,1} \cdot N_{0,3}$ ;
15  $N_{1,3} \leftarrow \mathbf{H}'_{1,0} \cdot N_{0,2} + \mathbf{H}'_{1,1} \cdot N_{0,3}$ ;
16  $N_{2,0} \leftarrow \mathbf{G}_{0,0} \cdot N_{0,0} + \mathbf{G}_{0,1} \cdot N_{0,2}$ ;
17  $N_{2,1} \leftarrow \mathbf{G}_{0,0} \cdot N_{0,1} + \mathbf{G}_{0,1} \cdot N_{0,3}$ ;
18  $N_{2,2} \leftarrow \mathbf{G}_{1,0} \cdot N_{0,0} + \mathbf{G}_{1,1} \cdot N_{0,2}$ ;
19  $N_{2,3} \leftarrow \mathbf{G}_{1,0} \cdot N_{0,1} + \mathbf{G}_{1,1} \cdot N_{0,3}$ ;
20  $N_{3,0} \leftarrow \mathbf{G}_{0,0} \cdot N_{1,0} + \mathbf{G}_{0,1} \cdot N_{1,2}$ ;
21  $N_{3,1} \leftarrow \mathbf{G}_{0,0} \cdot N_{1,1} + \mathbf{G}_{0,1} \cdot N_{1,3}$ ;
22  $N_{3,2} \leftarrow \mathbf{G}_{1,0} \cdot N_{1,0} + \mathbf{G}_{1,1} \cdot N_{1,2}$ ;
23  $N_{3,3} \leftarrow \mathbf{G}_{1,0} \cdot N_{1,1} + \mathbf{G}_{1,1} \cdot N_{1,3}$ ;
24  $\mathbf{N} \leftarrow (N_{i,j})_{0 \leq i,j \leq 3}$ ;
25 return  $\mathbf{N}$  ;           // Total cost: 8S + 113M + 1I + 49a

```

In **GluingCodomain** we present the procedure for computing the codomain A of the gluing $(2, 2)$ -isogeny $\Phi: E_1 \times E_2 \rightarrow A$. Explicitly, due to the manner in which the theta structure is initialized in **ThetaChangeOfBasis** (see **Alg. 38**), we will always have $\delta = 0$; otherwise, gluing has failed (for instance, because A degenerates to a product of elliptic curves), and the algorithm aborts. Given 8-torsion points T_1'', T_2'' lying above the kernel generators as input, **GluingCodomain** (see **Alg. 39**) outputs the dual isogenous theta point $(\alpha : \beta : \gamma : 0)$, the “inverse” of the dual isogenous theta null point $(\alpha^{-1} : \beta^{-1} : \gamma^{-1} : 0)$, and the theta null point $(a_2 : b_2 : c_2 : d_2)$ on A .

As in the generic case, two types of failure may occur during the gluing process. First, the algorithm assumes that for points on the dual surface (i.e., after applying $\mathcal{H} \circ \mathcal{S}$) the final coordinate vanishes; this condition is checked in Line 9 of **GluingCodomain**. Second, it is necessary to ensure that the remaining projective coordinates of such points are non-zero, as verified in Line 12. Furthermore, analogously to the generic setting, we must confirm that the input points T_1'', T_2'' are isotropic and compatible with the theta structure. This final verification is carried out in Line 24 of **GluingCodomain**.

Assume that we have computed the codomain A of a gluing $(2, 2)$ -isogeny $\Phi: E_1 \times E_2 \rightarrow A$ via `GluingCodomain`. We are now in a position to evaluate Φ at a point $P \in E_1 \times E_2$ using the `GluingEval` method (see **Alg.40**). In addition, a faster variant, `GluingEvalSpecial` (see **Alg. 41**), is available for the special cases where $P = (P_1, 0)$ or $P = (0, P_2)$.

`GluingEval` takes as input the point $P \in E_1 \times E_2$, the 8-torsion point T_1'' lying above a generator of $\ker(\Phi)$, the point $J = (x : x : y : y)$ representing the dual of $\Phi(T_1'')$, and the change-of-basis matrix computed during `GluingCodomain`, and returns the theta point $\Phi(P)$.

`GluingEvalSpecial` takes as input a point $P \in E_1 \times E_2$ of the form $(P_1, 0)$ or $(0, P_2)$, together with the point $I = (\alpha^{-1} : \beta^{-1} : \gamma^{-1} : 0)$, the “inverse” of the dual isogenous theta null point over A , and returns the theta point $\Phi(P)$.

Algorithm 39: `GluingCodomain`(T_1'', T_2'')

Input: 8-torsion points $T_1'', T_2'' \in E_1 \times E_2$, with $\ker(\Phi) = \langle [4]T_1'' \rangle \oplus \langle [4]T_2'' \rangle$.

Output: Dual isogenous theta null point $(\alpha : \beta : \gamma : 0)$, the “inverse” of the dual isogenous theta null point $(\alpha^{-1} : \beta^{-1} : \gamma^{-1} : 0)$, the theta null point O_A on A , the dual isogenous theta point $\Phi(T_1'')$ on A , and the change-of-basis matrix $\mathbf{N}$ (re-used for point evaluation). ; // Case $\delta = 0$

```

1   $T_1'' \leftarrow [2](T_1'')$  ; // Cost: 2S + 4M + 4a using xDBL
2   $T_2'' \leftarrow [2](T_2'')$  ; // Cost: 2S + 4M + 4a using xDBL
3   $\mathbf{N} \leftarrow \text{ThetaChangeOfBasis}(T_1'', T_2'')$  ; // Total cost: 8S + 113M + 11I + 49a
4   $[P_1, P_2] \leftarrow \text{ProductToTheta}([T_1'', T_2''], \mathbf{N})$  ; // Total cost: 40M + 24a
5   $x_1, y_1, z_1, w_1 \leftarrow P_1$ ;
6   $x_2, y_2, z_2, w_2 \leftarrow P_2$ ;
7   $(X_1, Y_1, Z_1, W_1) \leftarrow \mathcal{H} \circ \mathcal{S}(x_1, y_1, z_1, w_1)$ ;
8   $(X_2, Y_2, Z_2, W_2) \leftarrow \mathcal{H} \circ \mathcal{S}(x_2, y_2, z_2, w_2)$ ;
9  if  $W_1 \neq 0$  or  $W_2 \neq 0$  then
10 |   raise Exception: “GluingCodomain failed: last coordinate is non-zero”;
11 end
12 if  $X_1 = 0$  or  $X_2 = 0$  or  $Y_1 = 0$  or  $Z_2 = 0$  then
13 |   raise Exception: “GluingCodomain failed: projective factors are zero”;
14 end
15  $\alpha \leftarrow X_1 \cdot X_2$ ;
16  $\beta \leftarrow Y_1 \cdot X_2$ ;
17  $\gamma \leftarrow X_1 \cdot Z_2$ ;
18  $\alpha^{-1} \leftarrow Y_1 \cdot Z_2$ ;
19  $\beta^{-1} \leftarrow \gamma$ ;
20  $\gamma^{-1} \leftarrow \beta$ ;
21  $x \leftarrow X_1 \cdot \alpha^{-1}$ ;
22  $y \leftarrow Z_1 \cdot \gamma^{-1}$ ;
23  $J \leftarrow (x : x : y : y)$ ;
24 if  $(Y_1 \cdot \beta^{-1} \neq x)$  or  $(X_2 \cdot \alpha^{-1} \neq Y_2 \cdot \beta^{-1})$  then
25 |   // Check only done in verification
26 |   raise Exception: “GluingCodomain failed:  $[2]T_1'', [2]T_2''$  not isotropic”;
27 end
28  $(a_2, b_2, c_2, d_2) \leftarrow \mathcal{H}(\alpha, \beta, \gamma, 0)$ ;
29  $O_A \leftarrow (a_2 : b_2 : c_2 : d_2)$ ;
30 return  $(\alpha : \beta : \gamma : 0), (\alpha^{-1} : \beta^{-1} : \gamma^{-1} : 0), O_A, J, \mathbf{N}$  ; // Total cost (without checks):
12S + 159M + 11I + 105a

```

7.3 Splitting Change of Coordinates

We now describe how to compute a splitting $(2, 2)$ -isogeny $\Phi : A \rightarrow E_1 \times E_2$, where E_1, E_2 are elliptic curves and A is a principally polarized abelian surface (PPAS). Such an isogeny is computed in two steps:

- (1) Compute a “generic” $(2, 2)$ -isogeny using `GenericCodomainWith8Torsion`, `GenericCodomainWith4Torsion`, or `GenericCodomain` to obtain an isogeny $\Phi : A \rightarrow B$ where $B \cong E_1 \times E_2$.

Algorithm 40: GluingEval($P, T_1'', J, \mathbf{N}$)

Input: The point $P \in E_1 \times E_2$ and T_1'' an 8-torsion point such that $[4]T_1'' \in \ker(\Phi)$, both in Jacobian coordinates; the point $J = (x : x : y : y)$, the dual of $\Phi(T_1'')$ on A ; and the change-of-basis matrix $\mathbf{N}$ computed during GluingCodomain.

Output: Theta coordinates of $\Phi(P)$.

```

1  $P_1, P_2 \leftarrow P$ ;
2  $T_1, T_2 \leftarrow T_1''$ ;
3  $x, y \leftarrow J$ ;
4  $u_1, v_1, z_1 \leftarrow \text{ADDComponents}(P_1, T_1, E_1)$  ; // Cost: 11M + 5S + 7a
5  $u_2, v_2, z_2 \leftarrow \text{ADDComponents}(P_2, T_2, E_2)$  ; // Cost: 11M + 5S + 7a
6  $U \leftarrow (u_1 \cdot u_2 + v_1 \cdot v_2, u_1 \cdot w_1, w_1 \cdot u_2, w_1 \cdot w_2)$ ;
7  $V \leftarrow (v_1 \cdot u_2 + u_1 \cdot v_2, v_1 \cdot w_1, w_1 \cdot v_2, 0)$ ;
8  $U \leftarrow \mathbf{N} \cdot U$ ;
9  $V \leftarrow \mathbf{N} \cdot V$ ;
10  $U \leftarrow \mathcal{S}(U)$ ;
11  $V \leftarrow \mathcal{S}(V)$ ;
12  $(X_{\pm}, Y_{\pm}, Z_{\pm}, W_{\pm}) \leftarrow \mathcal{H}(U - V)$ ;
13  $(X_{\Phi(P)}, Y_{\Phi(P)}, Z_{\Phi(P)}, W_{\Phi(P)}) \leftarrow X_{\pm} \cdot y, Y_{\pm} \cdot y, Z_{\pm} \cdot x, W_{\pm} \cdot x$ ;
14  $(x_{\Phi(P)}, y_{\Phi(P)}, z_{\Phi(P)}, w_{\Phi(P)}) \leftarrow \mathcal{H}(X_{\Phi(P)}, Y_{\Phi(P)}, Z_{\Phi(P)}, W_{\Phi(P)})$ ;
15 return  $(x_{\Phi(P)} : y_{\Phi(P)} : z_{\Phi(P)} : w_{\Phi(P)})$  ; // Total cost: 62M + 18S + 50a

```

Algorithm 41: GluingEvalSpecial($P, I, \mathbf{N}$)

Input: The point $P \in E_1 \times E_2$ of the form $(P_1, 0)$ or $(0, P_2)$, the “inverse” of the dual isogenous theta null point $I := (\alpha^{-1} : \beta^{-1} : \gamma^{-1} : 0)$ on A , and the change-of-basis matrix $\mathbf{N}$ computed during GluingCodomain.

Output: Theta coordinates of $\Phi(P)$.

```

1  $P \leftarrow \text{ProductToTheta}(P, \mathbf{N})$  ; // Cost: 20M + 12a
2  $x_P, y_P, z_P, w_P \leftarrow P$ ;
3  $\alpha^{-1}, \beta^{-1}, \gamma^{-1}, - \leftarrow I$ ;
4  $(X_P, Y_P, Z_P, 0) \leftarrow \mathcal{H} \circ \mathcal{S}(x_P, y_P, z_P, w_P)$ ;
5  $(X_{\Phi(P)}, Y_{\Phi(P)}, Z_{\Phi(P)}) \leftarrow (X_P \cdot \alpha^{-1}, Y_P \cdot \beta^{-1}, Z_P \cdot \gamma^{-1})$ ;
6  $(x_{\Phi(P)}, y_{\Phi(P)}, z_{\Phi(P)}, w_{\Phi(P)}) \leftarrow \mathcal{H}(X_{\Phi(P)}, Y_{\Phi(P)}, Z_{\Phi(P)}, 0)$ ;
7 return  $(x_{\Phi(P)} : y_{\Phi(P)} : z_{\Phi(P)} : w_{\Phi(P)})$  ; // Total cost: 4S + 23M

```

- (2) Compute the isomorphism whose action on the theta null point O_B recovers the product theta structure of $E_1 \times E_2$.

Alg. 43 (called **SplittingIsomorphism**) performs the latter step. It invokes a sub-algorithm **GetIndexSplitting** (see **Alg. 44**) to determine an index (i, j) such that $U_{i,j}(0) = 0$, where $U_{i,j}$ are the level-4 theta coordinates. To evaluate $U_{i,j}$, we refer the reader to [DMPR25].

After computing a splitting $(2, 2)$ -isogeny $\Phi : A \rightarrow B$ with $B \cong E_1 \times E_2$, we wish to recover the Montgomery coefficients of E_1 and E_2 from the theta null point O_B of B . This procedure is detailed in **ThetaToProduct** (see **Alg. 45**). Additionally, points must be converted from theta product coordinates back to Montgomery coordinates; this is handled by **ThetaProductPointToMontgomery** (see **Alg. 42**).

Finally, note that the Montgomery coordinates obtained after the entire $(2, 2)$ -isogeny chain may still leak information about the underlying one-dimensional isogenies. To prevent any potential leakage during signing, the matrix output by **SplittingIsomorphism** (see **Alg. 43**) is randomized using the following strategy.

For every Montgomery curve E , there exist exactly six isomorphic Montgomery curves. Given a theta-null point associated with E , the theta-null points of the five other isomorphic curves can be obtained via a suitable change of variables. For efficiency, we apply this change of variables to both codomain curves resulting from **SplittingIsomorphism**. Concretely, this operation reduces to left-multiplying the matrix returned by **SplittingIsomorphism** by one of six precomputed change-of-basis matrices. This procedure is summarized in **RandomizedProduct** (see **Alg. 46**).

Algorithm 42: **ThetaProductPointToMontgomery**(P, O_A)

Input: A theta point $P := (x : y : z : t)$ and the theta null point $O_A := (a : b : c : d)$ of PPAS A with theta product structure.

Output: The Montgomery coordinates $(X(P_1) : Z(P_1))$ and $(X(P_2) : Z(P_2))$ of $P = (P_1, P_2) \in E_1 \times E_2$.

```

1  $(a, b, c, d) \leftarrow O_A$ ;
2  $(x, y, z, w) \leftarrow P$ ;
3  $X_1 \leftarrow a \cdot z + c \cdot x$ ;
4  $Z_1 \leftarrow a \cdot z - c \cdot x$ ;
5  $X_2 \leftarrow a \cdot y + b \cdot x$ ;
6  $Z_2 \leftarrow a \cdot y - b \cdot x$ ;
7 return  $(X_1 : Z_1), (X_2 : Z_2)$ ;

```

7.4 Computing a $(2, 2)$ -isogeny chain between products of elliptic curves

Alg. 47 (called **2X2IsogenyChain**) outlines the procedure for evaluating a $(2, 2)$ -isogeny chain $\Phi : E_1 \times E_2 \rightarrow E_3 \times E_4$ between products of elliptic curves. Given two input points $P, Q \in E_1 \times E_2$ of order 2^e , the algorithm employs a balanced strategy to efficiently compute the isogeny while simultaneously evaluating the image $\Phi(P')$ of some point $P' \in E_1 \times E_2$. This routing strategy is specifically tailored to mitigate the significant computational overhead associated with the initial gluing step.

Algorithm 43: SplittingIsomorphism(O_A)**Input:** Theta null point O_A of $A \cong E_1 \times E_2$.**Output:** Isomorphism given by matrix $\mathbf{M}$, whose action on O_A recovers the theta null point associated with the product theta structure.

```

1   $(i, j) \leftarrow \text{GetIndexSplitting}(O_A)$ ;
2  if  $(i, j) = (0, 0)$  then
3       $\mathbf{M} \leftarrow \begin{pmatrix} 1 & \sqrt{-1} & 1 & \sqrt{-1} \\ 1 & -\sqrt{-1} & -1 & \sqrt{-1} \\ 1 & \sqrt{-1} & -1 & -\sqrt{-1} \\ -1 & \sqrt{-1} & -1 & \sqrt{-1} \end{pmatrix}$ ;
4  else if  $(i, j) = (1, 0)$  then
5       $\mathbf{M} \leftarrow \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & -1 & 1 \\ 1 & 1 & -1 & -1 \\ -1 & 1 & -1 & 1 \end{pmatrix}$ ;
6  else if  $(i, j) = (2, 0)$  then
7       $\mathbf{M} \leftarrow \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & -1 & -1 & 1 \\ -1 & -1 & 1 & 1 \end{pmatrix}$ ;
8  else if  $(i, j) = (3, 0)$  then
9       $\mathbf{M} \leftarrow \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ -1 & 1 & 1 & -1 \end{pmatrix}$ ;
10 else if  $(i, j) = (0, 1)$  then
11      $\mathbf{M} \leftarrow \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & -1 & 0 & 0 \end{pmatrix}$ ;
12 else if  $(i, j) = (2, 1)$  then
13      $\mathbf{M} \leftarrow \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & -1 & -1 & 1 \\ 1 & 1 & -1 & -1 \end{pmatrix}$ ;
14 else if  $(i, j) = (0, 2)$  then
15      $\mathbf{M} \leftarrow \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & -1 & 0 \end{pmatrix}$ ;
16 else if  $(i, j) = (1, 2)$  then
17      $\mathbf{M} \leftarrow \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$ ;
18 else if  $(i, j) = (0, 3)$  then
19      $\mathbf{M} \leftarrow \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \end{pmatrix}$ ;
20 else if  $(i, j) = (3, 3)$  then
21      $\mathbf{M} \leftarrow \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$ ;
22  $\mathbf{M} \leftarrow \text{RandomizedProduct}(\mathbf{M})$ ; // Only performed during signing
23 return  $\mathbf{M}$ 

```

Algorithm 44: GetIndexSplitting(O_A)

Input: Theta null point O_A of a theta structure $A \cong E_1 \times E_2$.
Output: Index (i, j) such that $U_{i,j}(0) = 0$.

```

1 inds  $\leftarrow \{(0, 0), (0, 1), (0, 2), (0, 3), (1, 0), (1, 2), (2, 0), (2, 1), (3, 0), (3, 3)\}$ ;
2 count  $\leftarrow 0$ ;
3 got_index  $\leftarrow$  false;
4 for  $k \leftarrow 1$  to #inds do
5    $(i, j) \leftarrow$  inds[ $k$ ];
6    $U \leftarrow \sum_{t=0}^3 \chi(i, t) \cdot 0_A[t] \cdot 0_A[j \oplus t]$ ; // Here,  $\oplus$  denotes bitwise XOR
7   if  $U = 0$  then
8     count  $\leftarrow$  count + 1;
9      $(\text{idx}_i, \text{idx}_j) \leftarrow (i, j)$ ;
10    got_index  $\leftarrow$  true;
11 if count  $\neq 1$  or not got_index then
12   Error: "GetIndexSplitting failed: zero or multiple zero indices found";
13   return  $\perp$ ;
14 return  $i, j$ 
```

Algorithm 45: ThetaToProduct(O_A)

Input: The theta null point $O_A := (a : b : c : d)$ of PPAS A with theta product structure.
Output: The Montgomery coefficients of E_1, E_2 , namely $(A_1 : C_1)$ and $(A_2 : C_2)$, respectively, such that $A \cong E_1 \times E_2$.

```

1  $(a, b, c, d) \leftarrow 0_A$ ;
2  $t_1 \leftarrow ad$ ;
3  $t_2 \leftarrow bc$ ;
4 if  $t_1 \neq t_2$  then
5   Raise "ThetaToProduct failed:  $O_A$  does not come from a product theta structure"
6  $x \leftarrow a^4$ ;
7  $y \leftarrow b^4$ ;
8  $A_2 \leftarrow x + y$ ;
9  $C_2 \leftarrow x - y$ ;
10  $A_2 \leftarrow -2A_2$ ;
11  $z \leftarrow c^4$ ;
12  $A_1 \leftarrow x + z$ ;
13  $C_1 \leftarrow x - z$ ;
14  $A_1 \leftarrow -2A_1$ ;
15 if  $C_1 = 0$  or  $C_2 = 0$  then
16   Raise "ThetaToProduct failed"
17 return  $(A_1 : C_1), (A_2 : C_2)$ 
```

Algorithm 46: RandomizedProduct(M)

Input: The splitting matrix M
Output: The randomized matrix N

```

1  $\mathcal{M} \leftarrow$ 
    $\left\{ \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \begin{pmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{pmatrix}, \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{pmatrix}, \begin{pmatrix} 1 & -1 & -1 & 1 \\ -1 & 1 & 1 & 1 \\ -1 & 1 & -1 & 1 \\ 1 & 1 & 1 & 1 \end{pmatrix}, \begin{pmatrix} -1 & \sqrt{-1} & \sqrt{-1} & 1 \\ \sqrt{-1} & -1 & 1 & \sqrt{-1} \\ \sqrt{-1} & 1 & -1 & \sqrt{-1} \\ 1 & \sqrt{-1} & \sqrt{-1} & -1 \end{pmatrix}, \begin{pmatrix} 1 & \sqrt{-1} & \sqrt{-1} & -1 \\ \sqrt{-1} & 1 & -1 & \sqrt{-1} \\ \sqrt{-1} & -1 & 1 & \sqrt{-1} \\ -1 & \sqrt{-1} & \sqrt{-1} & 1 \end{pmatrix} \right\}$ ;
2 Sample a random  $N$  in  $\mathcal{M}$ ;
3  $N \leftarrow N \cdot M$ ;
4 return  $N$ ;
```

Algorithm 47: 2X2IsogenyChain(P, Q, e, pts)

Input: Points $P, Q \in E_1 \times E_2$ of order 2^e , a positive integer e , and an array **pts** containing points on $E_1 \times E_2$.

Output: The codomain $E_3 \times E_4$ of the $(2, 2)$ -isogeny chain $\Phi = \Phi_e \circ \dots \circ \Phi_1$ such that $\ker(\Phi) = \langle P, Q \rangle$ with $\deg(\Phi_k) = 2$ for all $k = 1, \dots, e$, and the evaluated points $[\Phi(P) \mid P \in \text{pts}]$.

```

1 Initialize a list strat_pts  $\leftarrow [(P, Q)]$ ;
2 Initialize a list orders  $\leftarrow [e - 2]$ ;
3  $k \leftarrow 0$ ;
4 while orders[ $k$ ]  $\neq 1$  do
5    $k \leftarrow k + 1$ ;
6   if orders[ $k - 1$ ]  $\geq 16$  then
7      $n \leftarrow \lfloor \text{orders}[k - 1] / 2 \rfloor$ ;
8   else
9      $n \leftarrow \text{orders}[k - 1] - 1$ ;
10   $(R, S) \leftarrow \text{strat\_pts}[k - 1]$ ;
11  for  $i \leftarrow 0$  to  $n - 1$  do
12     $R \leftarrow \text{DBL}(R, A_1)$ ;
13     $S \leftarrow \text{DBL}(S, A_2)$ ;
14  strat_pts[ $k$ ]  $\leftarrow (R, S)$ ;
15  orders[ $k$ ]  $\leftarrow \text{orders}[k - 1] - n$ ;
16  $\rightarrow, I, O_A, J, N \leftarrow \text{GluingCodomain}(\text{strat\_pts}[k])$ ; // Computing  $\Phi_1 : E_1 \times E_2 \rightarrow A_1$ 
17 pts  $\leftarrow [\text{GluingEvalSpecial}(P, I, N) \mid P \in \text{pts}]$ ;
18 for  $i \leftarrow 0$  to  $k - 1$  do
19   strat_pts[ $i$ ]  $\leftarrow \text{GluingEval}(\text{strat\_pts}[i], I, J, N)$ ;
20   orders[ $i$ ]  $\leftarrow \text{orders}[i] - 1$ ;
21  $k \leftarrow k - 1$ ;
22 consts  $\leftarrow \text{ThetaPrecomp}(O_A)$ ;
23  $j \leftarrow 0$ ;
24 while  $k \geq 0$  and orders[ $k$ ]  $\neq 0$  do
25    $j \leftarrow j + 1$ ;
26   while orders[ $k$ ]  $\neq 1$  do
27      $k \leftarrow k + 1$ ;
28      $n \leftarrow \lfloor \text{orders}[k - 1] / 2 \rfloor$ ;
29     for  $i \leftarrow 0$  to  $n - 1$  do
30       strat_pts[ $k$ ]  $\leftarrow \text{ThetaDBL}(\text{strat\_pts}[k - 1], \text{consts})$ ;
31     orders[ $k$ ]  $\leftarrow \text{orders}[k - 1] - n$ ;
32   $O'_A, I, O_A \leftarrow \text{GenericCodomainWith8Torsion}(\text{strat\_pts}[k])$ ; // Computing  $\Phi_k : A_{k-1} \rightarrow A_k$ 
33  pts  $\leftarrow [\text{GenericEval}(P, I) \mid P \in \text{pts}]$ ;
34  consts  $\leftarrow \text{ThetaPrecomp}(O_A)$ ;
35  for  $i \leftarrow 0$  to  $k - 1$  do
36    strat_pts[ $i$ ]  $\leftarrow \text{GenericEval}(\text{strat\_pts}[i], I)$ ;
37    orders[ $i$ ]  $\leftarrow \text{orders}[i] - 1$ ;
38   $k \leftarrow k - 1$ ;
39 if  $e \geq 3$  then
40   strat_pts[0]  $\leftarrow \text{GenericEval}(\text{strat\_pts}[0], I)$ ;
41    $\rightarrow, I, O_A \leftarrow \text{GenericCodomainWith4Torsion}(\text{strat\_pts}[0], O_A)$ ; // Computing  $\Phi_{e-1} : A_{e-2} \rightarrow A_{e-1}$ 
42   pts  $\leftarrow [\text{GenericEval}(P, I) \mid P \in \text{pts}]$ ;
43   $\rightarrow, I, O_A \leftarrow \text{GenericCodomain}(O_A)$ ; // Computing  $\Phi_e : A_{e-1} \rightarrow A_e$ 
44  pts  $\leftarrow [\text{GenericEval}(P, I) \mid P \in \text{pts}]$ ;
45   $M \leftarrow \text{SplittingIsomorphism}(O_A)$ ;
46   $O_A \leftarrow M \cdot O_A$ ;
47  pts  $\leftarrow [M \cdot P \mid P \in \text{eval\_pts}]$ ;
48   $(A_3 : C_3), (A_4 : C_4) \leftarrow \text{ThetaToProduct}(O_A)$ ;
49  pts  $\leftarrow [\text{ThetaProductPointToMontgomery}(P, O_A) \mid P \in \text{pts}]$ ;
50  Let  $E_3, E_4$  be the elliptic curves defined by Montgomery coefficients  $(A_3 : C_3), (A_4 : C_4)$ ;
51 return  $E_3 \times E_4, \text{pts}$ ;

```

8 Performance Analysis

The submission package includes a reference implementation written in portable C language, which is actually based on the library provided in SQIsign Verison 2.0.1. It consists of the following important sub-modules

- **signature**: implementing the signature key-generation and signing protocols.
- **verification**: implementing the signature verification protocol.
- **ec**: module for the elliptic curve and isogeny computations.
- **quaternion**: module for quaternion computation. This module employs arbitrary precision arithmetic based on GMP, and floating-point arithmetic using DPE.
- **precomp**: module to pre-compute constants for the code package.
- **gf**: implementation of the finite field arithmetic over $\mathbb{F}_p$ and $\mathbb{F}_{p^2}$.
- **hd**: computations with $(2, 2)$ -isogenies in the theta model.
- **mp**: routines for arbitrary precision arithmetic of integers with no dependency of GMP.
- **common**: common dependencies and symmetric primitive implementations.

In detail, we retain the original code in the **ec**, **quaternion**, **hd**, **mp**, and **common** sub-modules. The **signature** and **verification** sub-modules are newly created. In the **gf** sub-module, we include prime field arithmetic generated using the **fiat-crypto** library [FIA25].

8.1 Key and signature sizes

In Table 1, we list the key and signature sizes of $\text{SQIsign2d-push}^{1/2}$ across different security levels.

Table 1: The key and signature sizes in bytes of $\text{SQIsign2d-push}^{1/2}$.

Parameter set	Public key	Secret key	Signature
Level-1	64	518	150
Level-2	96	768	218
Level-3	128	1024	293
Level-4	262	2096	593

8.2 Computational cost

In this subsection, we compare the computational costs of SQIsign2DEast , SQIsign2DWest , SQIsign2D^2 , SQIsign2Dpush and $\text{SQIsign2d-push}^{1/2}$. As shown in Table 2, $\text{SQIsign2d-push}^{1/2}$ requires the fewest $(2, 2)$ -isogeny computations among the existing schemes during the signing process.

8.3 Reference implementation

For benchmarking, the reference implementation is built with two configurations: (1) using the GMP system installation on Ubuntu 22.04 LTS and (2) using a custom built version with disabled assembly code. The CMake build options were:

- (1) `-DSQISIGN_BUILD_TYPE=ref -DCMAKE_BUILD_TYPE=Release`
- (2) `-DSQISIGN_BUILD_TYPE=ref -DCMAKE_BUILD_TYPE=Release
-DGMP_LIBRARY:STRING=BUILD -DGMP_BUILD_CONFIG_ARGS:STRING="-disable-assembly"`

Table 2: The number of isogeny computations cross different isogeny-based digital signature schemes. The parameters L for our scheme are given in **Sect 5.2**.

Protocol		2	3	(2, 2)	L
SQIsign-West[BDF ⁺ 25]	Gen	-	-	$2 \log p$	-
	Sign	$\log p$	-	$4 \log p$	-
	Verf	$\log p$	-	$\frac{1}{2} \log p$	-
SQIsign2D-East[NOC ⁺ 25]	Gen	-	-	$\log p$	-
	Sign	$\frac{1}{2} \log p$	2	$3 \log p$	-
	Verf	$\frac{1}{2} \log p$	2	$\frac{1}{2} \log p$	-
SQIsign2D ² [XLZO26]	Gen	$\frac{1}{4} \log p$	$\frac{1}{2} \log_3 p$	$\frac{1}{4} \log p$	-
	Sign	-	$\frac{5}{2} \log_3 p$	$\log p$	-
	Verf	-	$\frac{1}{2} \log_3 p$	$\frac{1}{2} \log p$	-
SQIsign2DPush[NO25]	Gen	$\frac{3}{2} \log p$	$\frac{3}{2} \log_3 p$	-	-
	Sign	$\log p$	$3 \log_3 p$	$\log p$	-
	Verf	-	$\frac{1}{2} \log_3 p$	$\frac{1}{2} \log p$	-
SQIsign2d-push ^{1/2}	Gen	$\frac{3}{2} \log p$	$\frac{3}{2} \log_3 p$	-	-
	Sign	$\log p$	$\frac{5}{2} \log_3 p$	$\frac{1}{2} \log p$	1
	Verf	-	$\frac{1}{2} \log_3 p$	$\frac{1}{2} \log p$	-

8.4 Performance evaluation

Table 3 shows the computational times of the implementation. The computational times are measured on a computer with an Intel(R) Core(TM) i7-1360P CPU@2.60Hz without Turbo Boost.

It should be noted that the norm of the response ideal constructed during the signing process must be coprime to 3. If this condition is not satisfied, resampling is required, which results in a comparatively higher signing time for Level-2.

Table 3: Computational times (ms) of SQIsign2d-push^{1/2}

Parameter set	KeyGen	Sign	Verify
Level-1	45.16	64.04	12.54
Level-2	162.52	437.22	46.12
Level-3	386.17	466.88	96.01
Level-4	5097.91	5557.96	1110.14

References

- AAA⁺25a. Marius A Aardal, Gora Adj, Arwa Alblooshi, Diego F Aranha, Isaac A Canales-Martínez, Jorge Chávez-Saab, Décio Luiz Gazzoni Filho, Krijn Reijnders, and Francisco Rodríguez-Henríquez. Optimized one-dimensional sqsign verification on intel and cortex-m4. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2025(1):497–522, 2025.
- AAA⁺25b. Marius A. Aardal, Gora Adj, Diego F. Aranha, Andrea Basso, Isaac Andrés Canales Martínez, Jorge Chávez-Saab, Maria Corte-Real Santos, Pierrick Dartois, Luca De Feo, Max Duparc, Jonathan Komada Eriksen, Tako Boris Fouotsa, Décio Luiz Gazzoni Filho, Basil Hess, David Kohel, Antonin Leroux, Patrick Longa, Luciano Maino, Michael Meyer, Kohei Nakagawa, Hiroshi Onuki, Lorenz Panny, Sikhar Patranabis, Christophe Petit, Giacomo Pope, Krijn Reijnders, Damien Robert, Francisco Rodríguez-Henríquez, Sina Schaeffler, and Benjamin Wesolowski. SQIsign. Technical report, National Institute of Standards and Technology, 2025.
- BDF⁺25. Andrea Basso, Pierrick Dartois, Luca De Feo, Antonin Leroux, Luciano Maino, Giacomo Pope, Damien Robert, and Benjamin Wesolowski. SQIsign2D–West: The Fast, the Small, and the Safer. In Kai-Min Chung and Yu Sasaki, editors, *Advances in Cryptology – ASIACRYPT 2024*, pages 339–370, Singapore, 2025. Springer Nature Singapore.
- BFLS20. Daniel Bernstein, Luca De Feo, Antonin Leroux, and Benjamin Smith. Faster computation of isogenies of large prime degree. *Open Book Series*, 4(1):39–55, 2020.
- Can87. David Geoffrey Cantor. Computing in the jacobian of a hyperelliptic curve. *Mathematics of Computation*, 48:95–101, 1987.
- CCH⁺19. Matthew Campagna, Craig Costello, Basil Hess, Amir Jalali, Brian Koziel, Brian LaMacchia, Patrick Longa, Michael Naehrig, Joost Renes, David Urbanik, et al. Supersingular isogeny key encapsulation, 2019.
- Coh93. Henri Cohen. *A course in computational algebraic number theory*, volume 138 of *Graduate Texts in Mathematics*. Springer-Verlag, Berlin, 1993.
- CS18. Craig Costello and Benjamin Smith. Montgomery curves and their arithmetic: The case of large characteristic fields. *Journal of Cryptographic Engineering*, 8(3):227–240, 2018.
- Deu41. Max Deuring. Die Typen der Multiplikatorenringe elliptischer Funktionenkörper. *Abh. Math. Sem. Hansischen Univ.*, 14:197–272, 1941.
- DF25. Max Duparc and Tako Boris Fouotsa. Sqprime: A dimension 2 variant of sqsignhd with non-smooth challenge isogenies. In Kai-Min Chung and Yu Sasaki, editors, *Advances in Cryptology – ASIACRYPT 2024*, pages 396–429, Singapore, 2025. Springer Nature Singapore.
- DFKL⁺20. Luca De Feo, David Kohel, Antonin Leroux, Christophe Petit, and Benjamin Wesolowski. Sqsign: Compact post-quantum signatures from quaternions and isogenies. In Shiho Moriai and Huaxiong Wang, editors, *Advances in Cryptology – ASIACRYPT 2020*, pages 64–93, Cham, 2020. Springer International Publishing.
- DLRW24. Pierrick Dartois, Antonin Leroux, Damien Robert, and Benjamin Wesolowski. Sqsignhd: new dimensions in cryptography. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 3–32. Springer, 2024.
- DMPR25. Pierrick Dartois, Luciano Maino, Giacomo Pope, and Damien Robert. An algorithmic approach to (2, 2)-isogenies in the theta model and applications to isogeny-based cryptography. In Kai-Min Chung and Yu Sasaki, editors, *Advances in Cryptology – ASIACRYPT 2024*, pages 304–338, Singapore, 2025. Springer Nature Singapore.
- EHL⁺18a. Kirsten Eisenträger, Sean Hallgren, Kristin Lauter, Travis Morrison, and Christophe Petit. Supersingular isogeny graphs and endomorphism rings: reductions and solutions. In *Advances in cryptology—EUROCRYPT 2018. Part III*, volume 10822 of *Lecture Notes in Comput. Sci.*, pages 329–368. Springer, Cham, 2018.
- EHL⁺18b. Kirsten Eisenträger, Sean Hallgren, Kristin Lauter, Travis Morrison, and Christophe Petit. Supersingular isogeny graphs and endomorphism rings: Reductions and solutions. In Jesper Buus Nielsen and Vincent Rijmen, editors, *Advances in Cryptology – EUROCRYPT 2018*, pages 329–368, Cham, 2018. Springer International Publishing.
- FIA25. Fiat-crypto: Synthesizing correct-by-construction code for cryptographic primitives, 2025.
- Kan97. Ernst Kani. The number of curves of genus two with elliptic differentials. *Journal für die reine und angewandte Mathematik*, 485:93–122, 1997.
- Koh96. David Russell Kohel. *Endomorphism rings of elliptic curves over finite fields*. ProQuest LLC, Ann Arbor, MI, 1996. Thesis (Ph.D.)—University of California, Berkeley.
- Lyu09. Vadim Lyubashevsky. Fiat-shamir with aborts: Applications to lattice and factoring-based signatures. In Mitsuru Matsui, editor, *Advances in Cryptology – ASIACRYPT 2009*, pages 598–616, Berlin, Heidelberg, 2009. Springer Berlin Heidelberg.
- Mil86. J. S. Milne. *Jacobian Varieties*, pages 167–212. Springer New York, New York, NY, 1986.
- NO25. Kohei Nakagawa and Hiroshi Onuki. SQIsign2DPush: Faster signature scheme using 2-dimensional isogenies. Cryptology ePrint Archive, Paper 2025/897, 2025.

- NOC⁺25. Kohei Nakagawa, Hiroshi Onuki, Wouter Castryck, Mingjie Chen, Riccardo Invernizzi, Gioella Lorenzon, and Frederik Vercauteren. Sqsign2d-east: A new signature scheme using 2-dimensional isogenies. In Kai-Min Chung and Yu Sasaki, editors, *Advances in Cryptology – ASIACRYPT 2024*, pages 272–303, Singapore, 2025. Springer Nature Singapore.
- NS09. Phong Q. Nguyen and Damien Stehlé. An lll algorithm with quadratic complexity. *SIAM Journal on Computing*, 39(3):874–903, 2009.
- OU73. Frans Oort and Kenji Ueno. Principally polarized abelian varieties dimension two or three are jacobian varieties. (38), 1973.
- PW24. Aurel Page and Benjamin Wesolowski. The supersingular endomorphism ring and one endomorphism problems are equivalent. In Marc Joye and Gregor Leander, editors, *Advances in Cryptology – EUROCRYPT 2024*, pages 388–417, Cham, 2024. Springer Nature Switzerland.
- Tat62. John Tate. Duality theorems in galois cohomology over number fields. In *Proc. Internat. Congr. Mathematicians (Stockholm, 1962)*, pages 288–295, 1962.
- Vél71. Jacques Vélú. Isogénies entre courbes elliptiques. *Comptes-Rendus de l’Académie des Sciences*, 273:238–241, 1971.
- Voi21. John Voight. *Quaternion algebras*, volume 288 of *Graduate Texts in Mathematics*. Springer, Cham, 2021.
- Wat69. William C. Waterhouse. Abelian varieties over finite fields. *Annales scientifiques de l’École Normale Supérieure*, 2(4):521–560, 1969.
- Wei40. André Weil. Sur les fonctions algébriques a corps de constantes fini. *CR Acad. Sci. Paris*, 210(592-594):149, 1940.
- Wes22. Benjamin Wesolowski. The supersingular isogeny path and endomorphism ring problems are equivalent. In *2021 IEEE 62nd Annual Symposium on Foundations of Computer Science (FOCS)*, pages 1100–1111, 2022.
- XLZO26. Zheng Xu, Kaizhan Lin, Chang-An Zhao, and Yi Ouyang. Sqsign2d²: New sqsign2d variant by leveraging power smooth isogenies in dimension one. In Goichiro Hanaoka and Bo-Yin Yang, editors, *Advances in Cryptology – ASIACRYPT 2025*, pages 341–371, Singapore, 2026. Springer Nature Singapore.